РАЗБОР ОФИЦИАЛЬНОГО ОБРАЗЦА

Демонстрационный экзамен 2027

Информационная система мебельного производства

WPF + PostgreSQL

ФИО преподавателя _____

Должность _____

Кафедра / лаборатория _____



Как устроен экзамен

3 ч 30 мин
210 минут на выполнение

Базовый
Уровень ГИА ДЭ

75
Максимум баллов

Разбираем официальный образец КИМ 09.02.07-5-2027. Предприятие выпускает мебель по спецификациям. Нужно связать исходные документы, базу данных, программу для пользователей и сервис.

Шесть заданий и время

Задание	Время
1. Проектирование ER-диаграммы	20 мин
2. Создание базы и импорт	30 мин
3. Запрос стоимости заказа	20 мин
4. Информационная система	60 мин
5. API и тестовая коллекция	60 мин
6. Документация API	20 мин

Содержание

Раздел	Слайд
Подготовка	7
1. Проектирование ER	24
2. Создание БД и импорт	60
3. Расчёт стоимости	90
4. Формы и WPF	125
5. API и проверки	243
6. Документация API	302
Готовый пример	323

Содержание · Информационная система

Блок задания 4	Слайд
Окно входа	127
Учётные записи	141
SQL-запросы	157
Пазл и блокировка	184
Рабочее окно	203

Что открыть перед занятием

1. Откройте КИМ на страницах 30–32 и скачайте приложения к официальному образцу. Проверьте, что открываются Excel, JSON, PDF и шаблон DOCX.
2. Проверьте Visual Studio с поддержкой .NET, PostgreSQL с pgAdmin, draw.io и Postman.
3. Распакуйте готовый пример отдельно от рабочей папки. В Visual Studio откройте решение или файл проекта. Перед входом подготовьте базу и учебные записи по инструкции запуска.

ЧТО ПРОВЕРИТЬ

Исходные документы и готовый пример открываются с одного учебного компьютера.

КИМ · PDF Исходные файлы · ZIP Готовый пример · ZIP

ПЕРЕД НАЧАЛОМ

Документы и задача

Перед вами пример про мебельное производство. Мы начнём с заказа на два стола «Самобранка» и дойдём до приложения, сервиса и документации.

ЧТО ПРОВЕРИТЬ

На рабочем столе есть папка **Surname + Subject Area**. Исходные файлы остаются отдельно от результатов.

ШАГ 1 ИЗ 9 · 1/2

Проверьте Windows и программы

Проверка среды заранее помогает понять, можно ли на этом компьютере собрать приложение и проверить API.

1. Откройте рабочий стол **Windows**. Если вы работаете на **Mac** через **Parallels**, сначала запустите свою виртуальную машину **Windows**.
2. Откройте «Пуск», введите **Visual Studio** и запустите программу.
3. В стартовом окне нажмите **Create a new project** («Создать проект»). Найдите шаблон **WPF Application** для **C#** на современном **.NET**. Перейдите по кнопке «Далее» до выбора платформы. Если в списке есть **.NET 10**, закройте мастер крестиком. На этом шаге достаточно увидеть платформу в списке.

ШАГ 1 ИЗ 9 · 2/2

Проверьте Windows и программы

1. В меню «Пуск» откройте **pgAdmin**. Слева раскройте **Servers**, дважды нажмите на настроенный учебный сервер **PostgreSQL** и введите пароль, если появится запрос. Раскройте **Databases**. Если сервера в списке нет или подключение не удалось, уточните параметры у преподавателя.
2. Откройте **Postman** через меню «Пуск». Если не удалось открыть программу или найти нужный шаблон или платформу, сообщите преподавателю, на каком действии возникла проблема.

ЧТО ПРОВЕРИТЬ

В **Visual Studio** доступен нужный шаблон **C#** и **.NET 10**. **Postman** открывается. Сервер базы данных отвечает в **phpMyAdmin** или **pgAdmin**, в зависимости от выбранного стека.

ШАГ 2 ИЗ 9 · 1/2

Определите задачу предприятия

Предприятие хранит заказ, состав изделия, цены и план выпуска в отдельных документах. Информационная система связывает эти сведения и поддерживает работу пользователей.

1. Откройте «КИМ · стр. 30» по ссылке в начале шага. Прочитайте задания на страницах 30–32.
2. Найдите в условии задания 1, что производит предприятие и на основании каких документов предстоит спроектировать систему.
3. Для каждого из шести заданий назовите, какой результат должен получиться. Сверьтесь со списком ниже.

ЧТО ПРОВЕРИТЬ

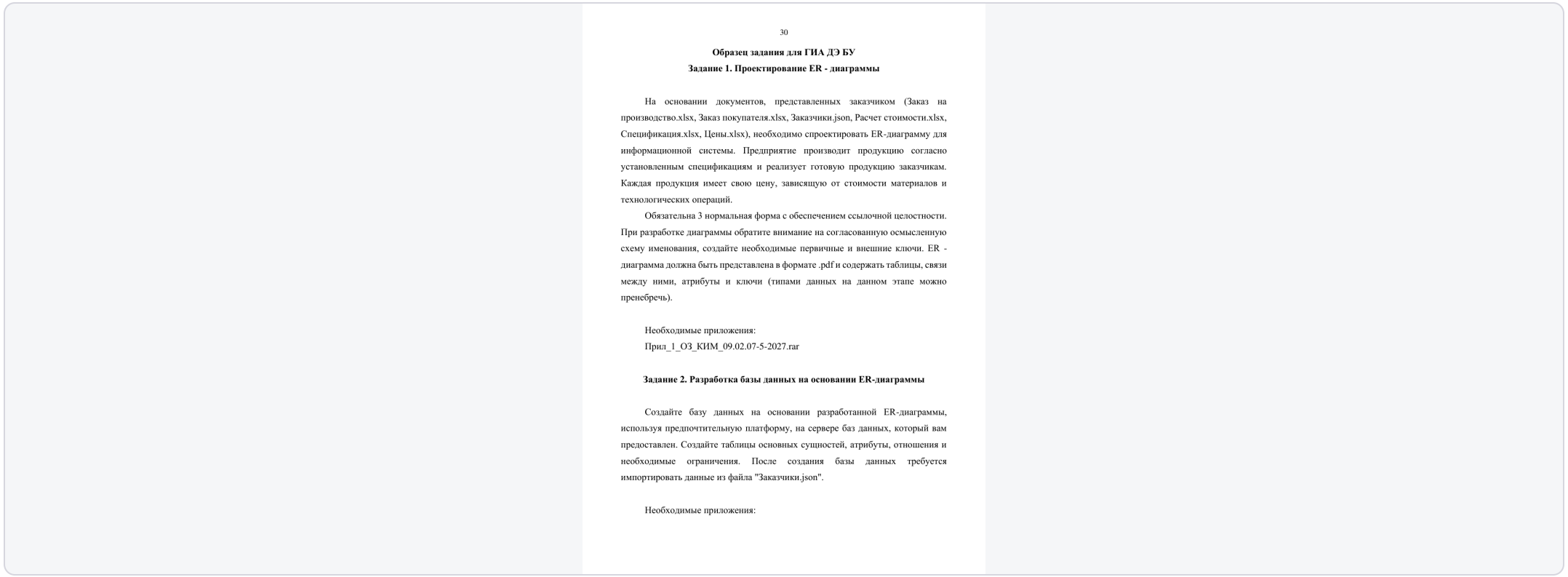
1 — ER-диаграмма в PDF. 2 — база данных с импортированными заказчиками. 3 — запрос расчёта стоимости заказа. 4 — настольное приложение с авторизацией и ролями. 5 — API и коллекция запросов с тестами. 6 — документация API по шаблону.

КИМ, стр. 30–32

ШАГ 2 ИЗ 9 · 2/2

Определите задачу предприятия

Покажите на экране



30

Образец задания для ГИА ДЭ БУ

Задание 1. Проектирование ER - диаграммы

На основании документов, представленных заказчиком (Заказ на производство.xlsx, Заказ покупателя.xlsx, Заказчики.json, Расчет стоимости.xlsx, Спецификация.xlsx, Цены.xlsx), необходимо спроектировать ER-диаграмму для информационной системы. Предприятие производит продукцию согласно установленным спецификациям и реализует готовую продукцию заказчикам. Каждая продукция имеет свою цену, зависящую от стоимости материалов и технологических операций.

Обязательна 3 нормальная форма с обеспечением ссылочной целостности. При разработке диаграммы обратите внимание на согласованную осмысленную схему именования, создайте необходимые первичные и внешние ключи. ER - диаграмма должна быть представлена в формате .pdf и содержать таблицы, связи между ними, атрибуты и ключи (типами данных на данном этапе можно пренебречь).

Необходимые приложения:
Прил_1_О3_КИМ_09.02.07-5-2027.rar

Задание 2. Разработка базы данных на основании ER-диаграммы

Создайте базу данных на основании разработанной ER-диаграммы, используя предпочтительную платформу, на сервере баз данных, который вам предоставлен. Создайте таблицы основных сущностей, атрибуты, отношения и необходимые ограничения. После создания базы данных требуется импортировать данные из файла "Заказчики.json".

Необходимые приложения:

КИМ 09.02.07-5-2027, страница 30: начало заданий ГИА БУ. Изображение оригинальной страницы PDF.

КИМ, стр. 30–32

ШАГ 3 ИЗ 9

Подготовьте собственное место работы

В одной рабочей папке проще найти все файлы, которые вы создадите на следующих этапах.

1. Перейдите на рабочий стол Windows. Если открыты окна, нажмите Win+D.
2. Нажмите правой кнопкой на свободном месте рабочего стола → «Создать» → «Папку».
3. Введите фамилию латиницей, знак «+» и название предметной области на английском. Например, Ivanov + Furniture Manufacturing. Нажмите Enter.
4. Откройте созданную папку двойным щелчком.

ЧТО ПРОВЕРИТЬ

На рабочем столе Windows есть папка, названная по шаблону Surname + Subject Area.

ШАГ 4 ИЗ 9

Сохраните исходные документы

Эти документы понадобятся для схемы, импорта заказчиков и расчёта стоимости.

1. Откройте свою рабочую папку на рабочем столе Windows. Нажмите Ctrl+Shift+N, введите «Исходные данные» и нажмите Enter.
2. Нажмите «Приложения · ZIP» в начале этого шага.
3. В Проводнике откройте папку «Загрузки» или папку, куда сохранили архив. Нажмите правой кнопкой по basic-inputs.zip и выберите «Извлечь всё».
4. Нажмите «Обзор», выберите созданную папку «Исходные данные» и нажмите «Извлечь». Откройте эту папку в Проводнике.

ЧТО ПРОВЕРИТЬ

В папке «Исходные данные» видны папки «Задание 1» — «Задание 6». Внутри «Задание 1» открывается «Заказ покупателя.xlsx».

Откройте: archive

ШАГ 5 ИЗ 9

Подготовьте папку для результатов

Готовые файлы будет проще найти, если хранить их отдельно от исходных документов.

1. Откройте свою рабочую папку на рабочем столе **Windows**.
2. Нажмите **Ctrl+Shift+N**, введите «Результаты» и нажмите **Enter**. Сюда будете сохранять диаграмму, SQL-файлы и другие результаты.
3. Убедитесь, что в рабочей папке рядом видны «Исходные данные» и «Результаты». Откройте «Результаты». Папка пока пуста.

ЧТО ПРОВЕРИТЬ

В рабочей папке рядом лежат «Исходные данные» с материалами КИМ и пустая папка «Результаты».

ШАГ 6 ИЗ 9 · 1/2

Прочитайте заказ покупателя

Номер, дата и покупатель относятся ко всему заказу. Наименование изделия, количество и цена относятся к отдельной строке товара.

1. Откройте «Заказ покупателя.xlsx» по ссылке в начале шага. Ищите значения по заголовкам столбцов. Если навести указатель на ячейку, появится её адрес.
2. В верхней части документа найдите номер и дату заказа, исполнителя и заказчика.
3. Найдите строку со столом «Самобранка». Прочитайте значения в столбцах «Код», «Кол-во», «Цена», «Скидка» и «Сумма». Если часть столбцов скрыта, прокрутите таблицу вправо.

ЧТО ПРОВЕРИТЬ

В заказе два стола по **14 120**. Скидка на позицию — **1 412**, итог к оплате — **26 828**.

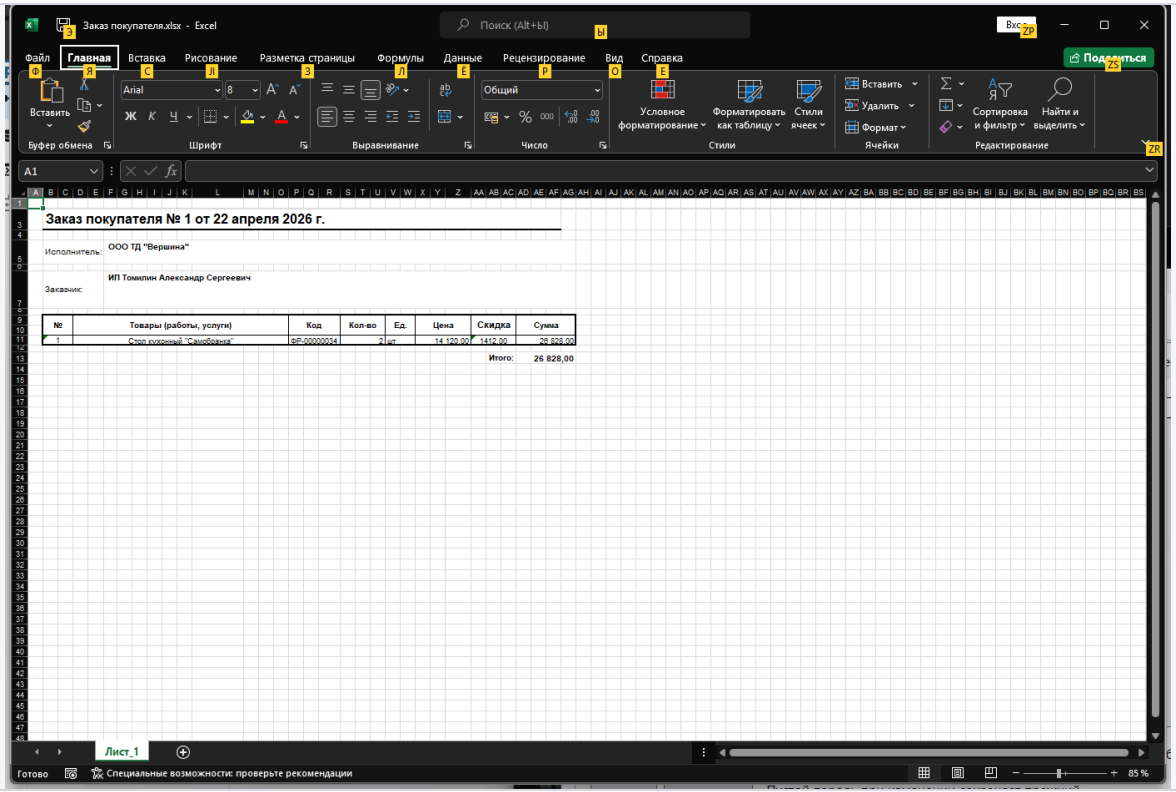
Заказ покупателя.xlsx, B3:F7, D11:AD11

Откройте: Заказ покупателя.xlsx

ШАГ 6 ИЗ 9 · 2/2

Прочитайте заказ покупателя

Покажите на экране



Заказ покупателя из приложения открыт в **Excel**. Маршрут: **WPF / PostgreSQL**.

Заказ покупателя.**xlsx**, **B3:F7**, **D11:AD11**

ШАГ 7 ИЗ 9 · 1/2

Прочитайте состав и цены

Спецификация показывает, сколько ресурсов нужно на один стол. Прайс показывает, сколько стоит единица каждого ресурса.

1. Откройте «Спецификация.xlsx». В разделе «Состав» найдите «Евровинт 6,5x5». Прочитайте его единицу измерения и количество.
2. Откройте «Цены.xlsx». Найдите то же наименование в столбце «Номенклатура» и прочитайте цену в столбце D.
3. Вернитесь к спецификации. В разделе «Технологическая операция» найдите «Сборка модулей» и прочитайте два значения: «Норма времени» и «Количество».

ЧТО ПРОВЕРИТЬ

На один стол требуется **0,012** тыс. шт. евровинтов, цена в прайсе — **595**. У сборки модулей норма времени **0,75** и количество **1**.

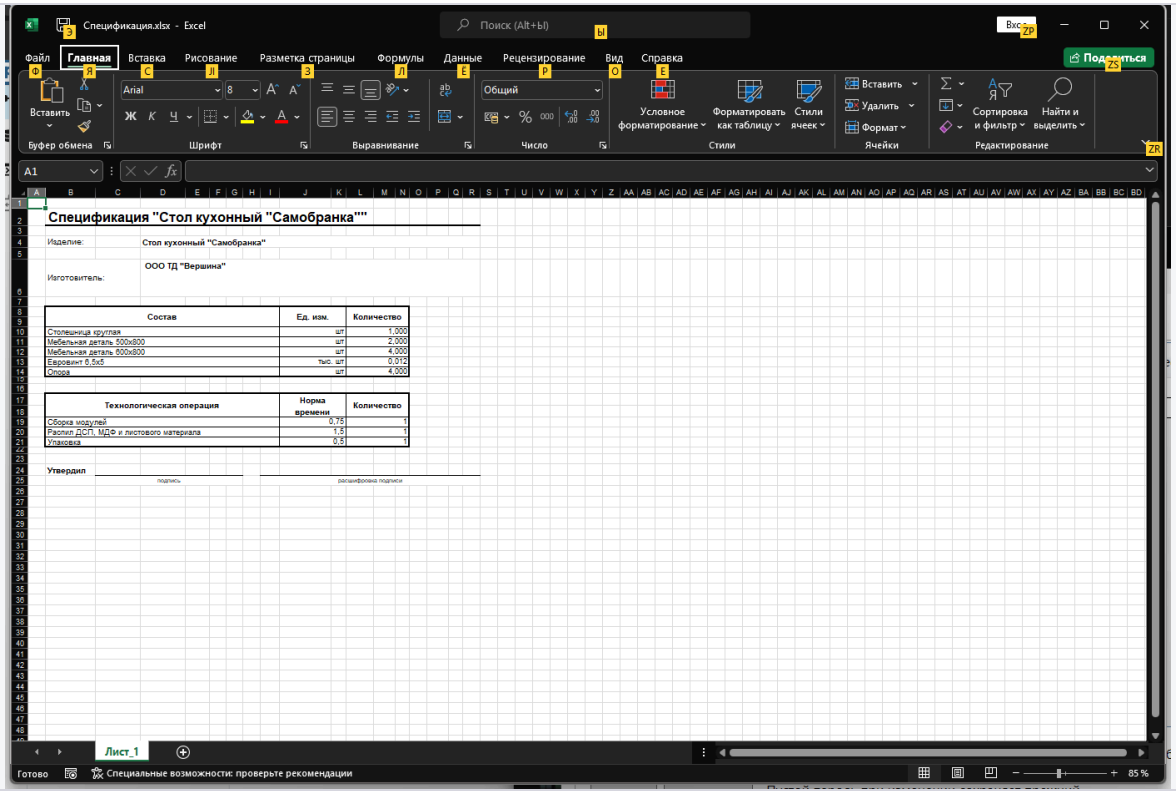
Спецификация.xlsx, B13:L13 и B19:L19 · Цены.xlsx, A2:D2

Откройте: Спецификация.xlsx · Цены.xlsx

ШАГ 7 ИЗ 9 · 2/2

Прочитайте состав и цены

Покажите на экране



Спецификация: материалы, количества и нормы технологических операций в **Excel**. Маршрут: **WPF / PostgreSQL**.

Спецификация.xlsx, B13:L13 и B19:L19 · Цены.xlsx, A2:D2

ШАГ 8 ИЗ 9 · 1/2

Проверьте JSON заказчиков

JSON — текстовый файл со списком записей. Каждая запись станет отдельной строкой справочника контрагентов.

1. Откройте «Заказчики.json». Каждая запись между фигурными скобками описывает одного контрагента.
2. В первой записи найдите **id** (код), **name** (название), **inn** (ИНН), **addres** (адрес), **phone** (телефон) и **type** (тип контрагента).
3. Посмотрите, что записано в **inn** у первого контрагента, и сравните значения **type** у разных записей. При импорте сохраняйте код с ведущими нулями как текст.

ЧТО ПРОВЕРИТЬ

В файле шесть записей. Первый код — **000000001**, первый ИНН пустой. В поле **type** встречаются «Поставщик» и «Покупатель».

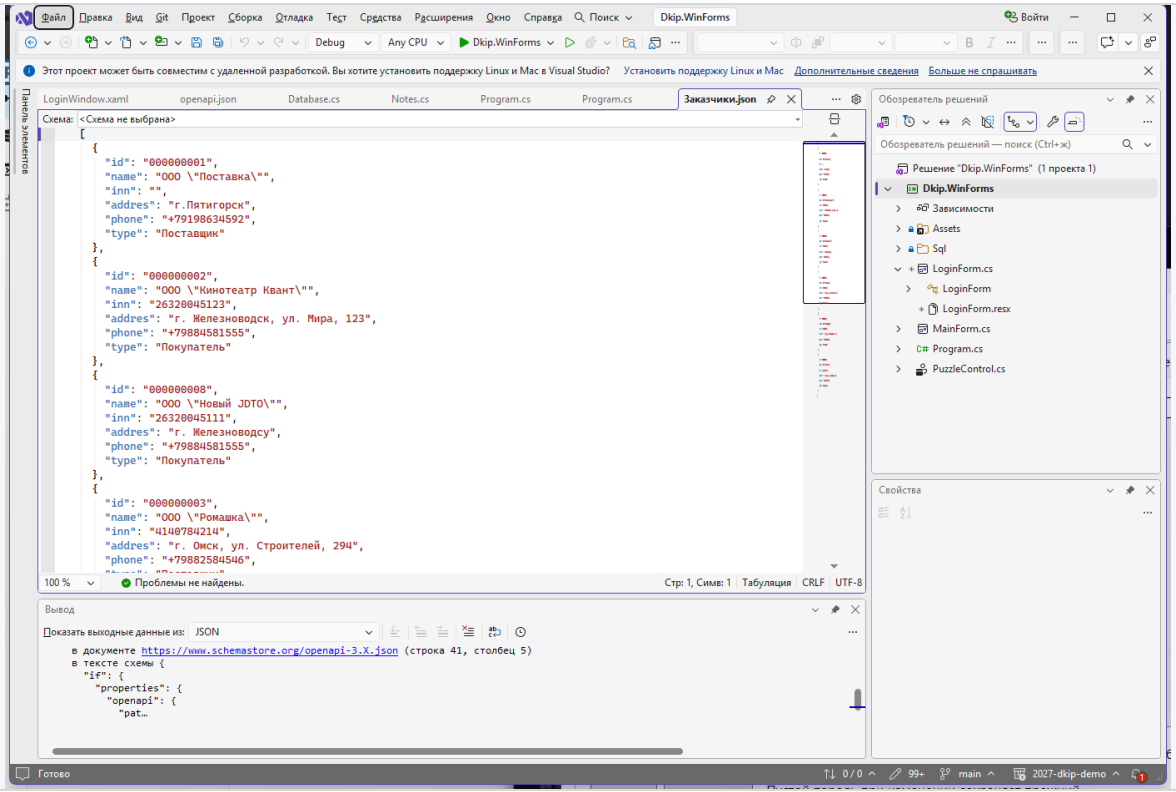
Заказчики.json

Откройте: Заказчики.json

ШАГ 8 ИЗ 9 · 2/2

Проверьте JSON заказчиков

Покажите на экране



Исходный Заказчики.**json** открыт в редакторе без изменения значений. Маршрут: **WPF / PostgreSQL**.

Заказчики.**json**

ШАГ 9 ИЗ 9 · 1/2

Сравните документы

Название, код и единица измерения могут различаться между документами. При проектировании важно сохранить значения из каждого источника и пояснить, как вы их сопоставили.

1. Откройте «Заказ покупателя.xlsx» и «Заказ на производство.xlsx». В каждом найдите строку со столом «Самобранка» и сравните столбцы «Код» и «Количество». В обоих документах указано два стола, но коды разные.
2. Откройте «Спецификация.xlsx». Сравните единицы измерения евровинта и опоры в разделе «Состав» с их единицами в разделе «Потребность в материалах» заказа на производство.

ШАГ 9 ИЗ 9 · 2/2

Сравните документы

1 Откройте «Расчет стоимости.xlsx». Найдите «Итого» и сравните с суммой к оплате из заказа покупателя. В первом файле указан исходный расчёт затрат, во втором — сумма продажи со скидкой. Свой расчёт затрат вы выполните в задании 3.

ЧТО ПРОВЕРИТЬ

В заказе покупателя код ФР-**00000034**, в заказе на производство — НФ-**00000006**. В учебном примере это одно изделие по совпадению названия, оба исходных кода сохраняются. Евровинт и опора в спецификации указаны в тыс. шт. и шт. соответственно, в заказе на производство — наоборот. Исходный расчёт затрат — **11 448,12**, сумма к оплате — **26 828**.

Откройте: Заказ покупателя.**xlsx** · Заказ на производство.**xlsx** · Спецификация.**xlsx** · Расчет стоимости.**xlsx**

РЕЗУЛЬТАТ ЭТАПА

Что должно остаться после этапа

На рабочем столе есть папка **Surname + Subject Area**. Исходные файлы остаются отдельно от результатов.

ЧТО ПРОВЕРИТЬ

Рабочая папка с исходными документами и отдельным местом для результатов.

ЗАДАНИЕ 1 · 20 МИН

Проектирование ER

Из документов выделим сущности — то, о чём надо хранить отдельные записи. Затем свяжем их ключами.

ЧТО ПРОВЕРИТЬ

ER-диаграмма.**pdf** открывается и читается. ER-диаграмма.**drawio** остаётся для правок.

УСЛОВИЕ ИЗ КИМ · 1/2

Проектирование ER-диаграммы

На основании документов, представленных заказчиком (Заказ на производство.xlsx, Заказ покупателя.xlsx, Заказчики.json, Расчет стоимости.xlsx, Спецификация.xlsx, Цены.xlsx), необходимо спроектировать ER-диаграмму для информационной системы.

Предприятие производит продукцию согласно установленным спецификациям и реализует готовую продукцию заказчикам. Каждая продукция имеет свою цену, зависящую от стоимости материалов и технологических операций.

УСЛОВИЕ ИЗ КИМ · 2/2

Проектирование ER-диаграммы

Обязательна 3 нормальная форма с обеспечением ссылочной целостности. При разработке диаграммы обратите внимание на согласованную осмысленную схему именования, создайте необходимые первичные и внешние ключи.

ER-диаграмма должна быть представлена в формате .pdf и содержать таблицы, связи между ними, атрибуты и ключи (типами данных на данном этапе можно пренебречь).

Приложение: Прил_1_О3_КИМ_09.02.07-5-2027.rar

КИМ · стр. 30 Исходные файлы · ZIP

ШАГ 1 ИЗ 12

Откройте draw.io

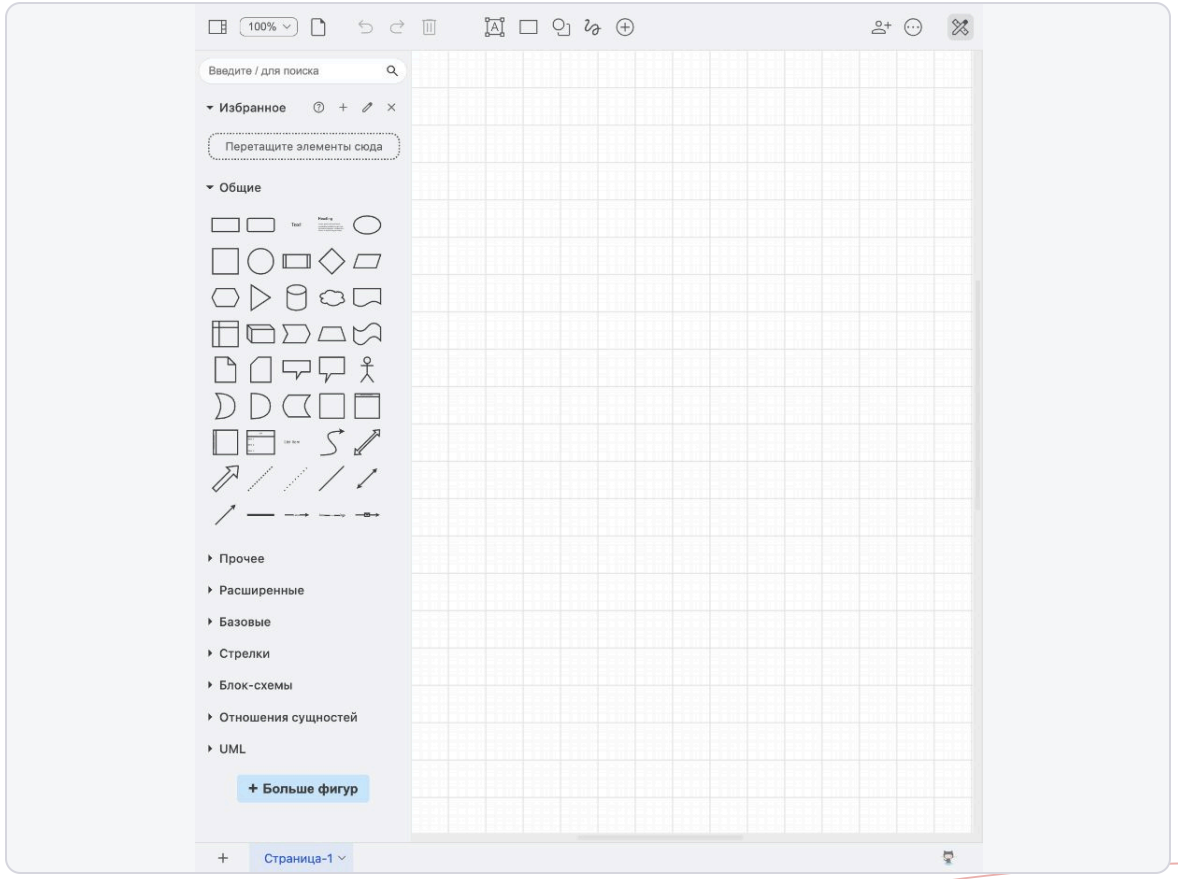
Найдите в исходниках сведения об объектах и повторяющиеся строки. Постройте свою схему в draw.io и сверьте ключи и связи с учебным примером.

1. Для тренировки откройте draw.io по ссылке выше. На экзамене будут установлены настольный draw.io и Microsoft Visio.
2. Если редактор предлагает выбрать место хранения, нажмите «Устройство» (Device). Создайте пустую диаграмму.

ЧТО ПРОВЕРИТЬ

Открыт редактор draw.io. На холсте пока нет фигур, слева видна панель фигур.

Откройте: drawio



Браузерный draw.io с пустой диаграммой. Слева видна панель фигур, внизу кнопка «Больше фигур».

ШАГ 2 ИЗ 12 · 1/2

Подключите библиотеку ER

Найдите в исходниках сведения об объектах и повторяющиеся строки. Постройте свою схему в **draw.io** и сверьте ключи и связи с учебным примером.

1. Внизу левой панели нажмите «Больше фигур» (**More Shapes**).
2. В разделе «Программное обеспечение» (**Software**) найдите и отметьте «Отношения сущностей» (**Entity Relation**). Если галочка уже стоит, оставьте её.
3. Нажмите «Применить» (**Apply**). В левой панели раскройте «Отношения сущностей», чтобы увидеть фигуры таблиц и связей.

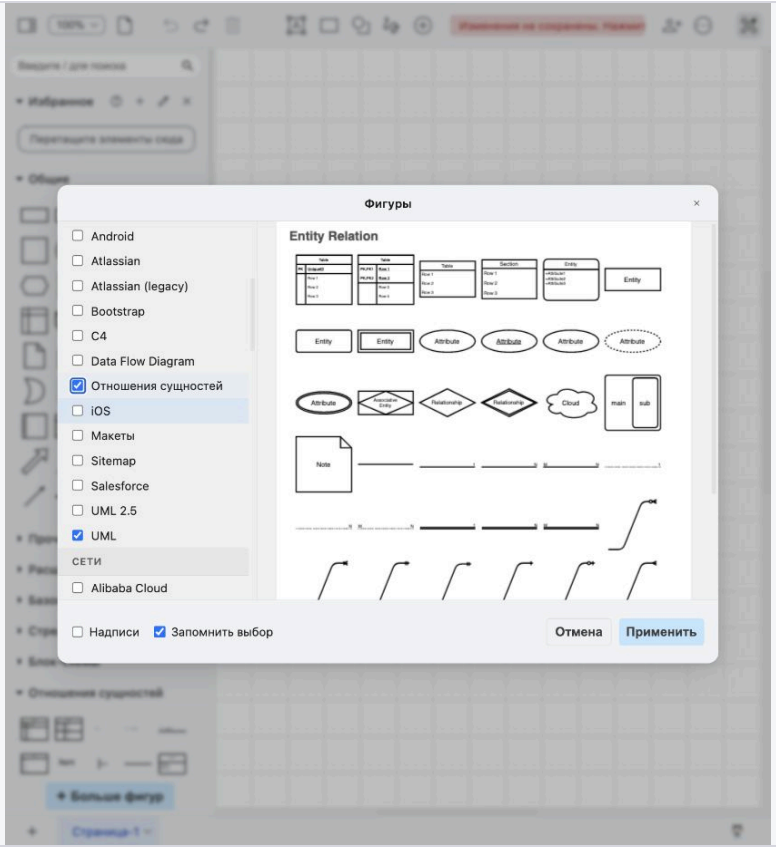
ЧТО ПРОВЕРИТЬ

В левой панели раскрыт раздел «Отношения сущностей». В нём видны фигуры таблиц и линий связи.

ШАГ 2 ИЗ 12 · 2/2

Подключите библиотеку ER

Покажите на экране



В окне «Фигуры» выбрана библиотека «Отношения сущностей». Внизу справа кнопка «Применить».

ШАГ 3 ИЗ 12

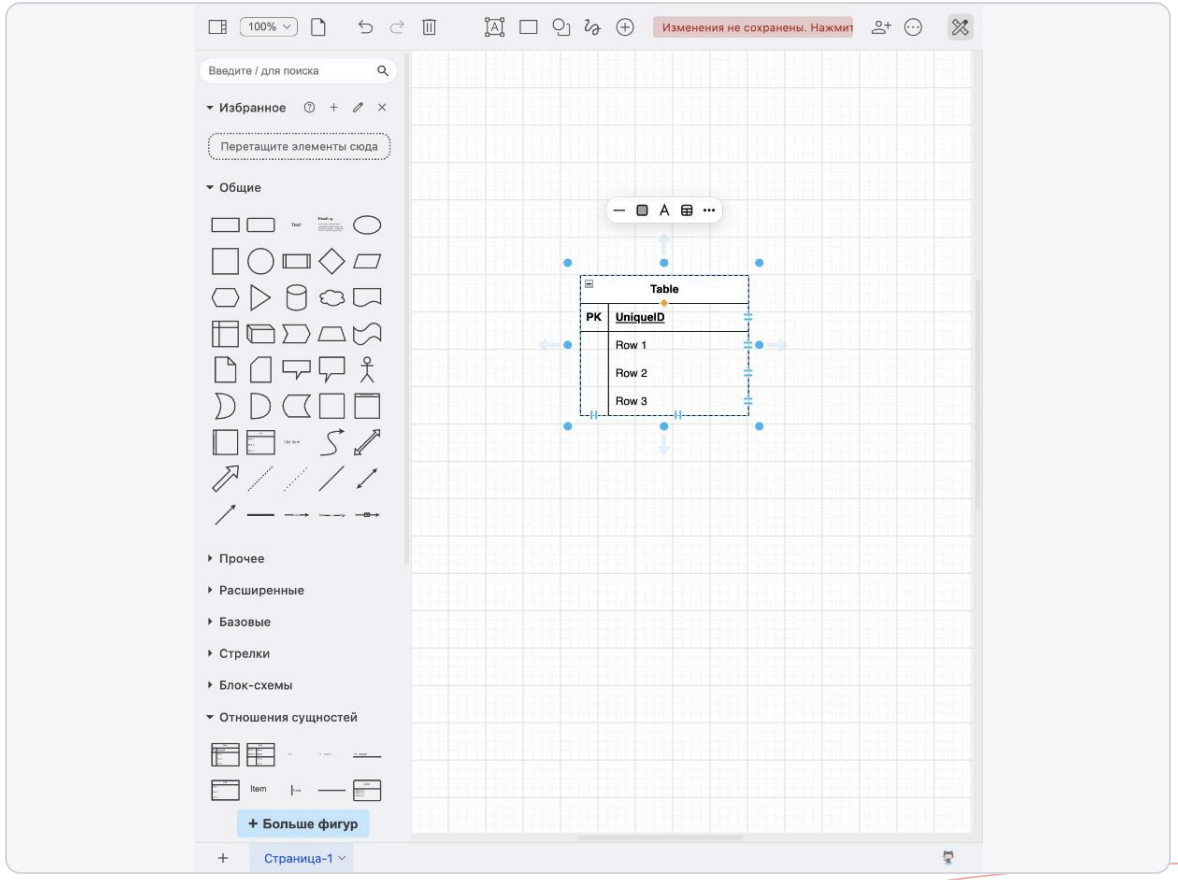
Добавьте фигуру таблицы

Найдите в исходниках сведения об объектах и повторяющиеся строки. Постройте свою схему в draw.io и сверьте ключи и связи с учебным примером.

1. В библиотеке «Отношения сущностей» найдите первую фигуру **Table**. На значке видны заголовок таблицы и строки.
2. Перетащите фигуру на пустой холст. На следующем шаге замените заголовок **Table** и названия строк своими.

ЧТО ПРОВЕРИТЬ

На холсте появилась фигура **Table** с заголовком и строками, включая строку **PK**.



Фигура **Table** из библиотеки «Отношения сущностей» на холсте. Под заголовком видна строка **PK**.

ШАГ 4 ИЗ 12 · 1/3

Отделите сущности от документов

Из одного документа могут получиться несколько таблиц. Сначала выясните, к чему относится каждое поле.

1. Откройте «Заказ покупателя.xlsx». Номер и дата в заголовке и имя покупателя относятся ко всему заказу. Название стола, количество и цена относятся к одной строке товара.
2. Откройте «Заказчики.json». Дважды нажмите заголовок фигуры таблицы в **draw.io** и введите **counterparty**. По первой записи заполните строки. Отметьте **id** как **PK**, ниже добавьте **name**, **inn**, **address**, **phone** и **party_type**.

ШАГ 4 ИЗ 12 · 2/3

Отделите сущности от документов

Покажите на экране

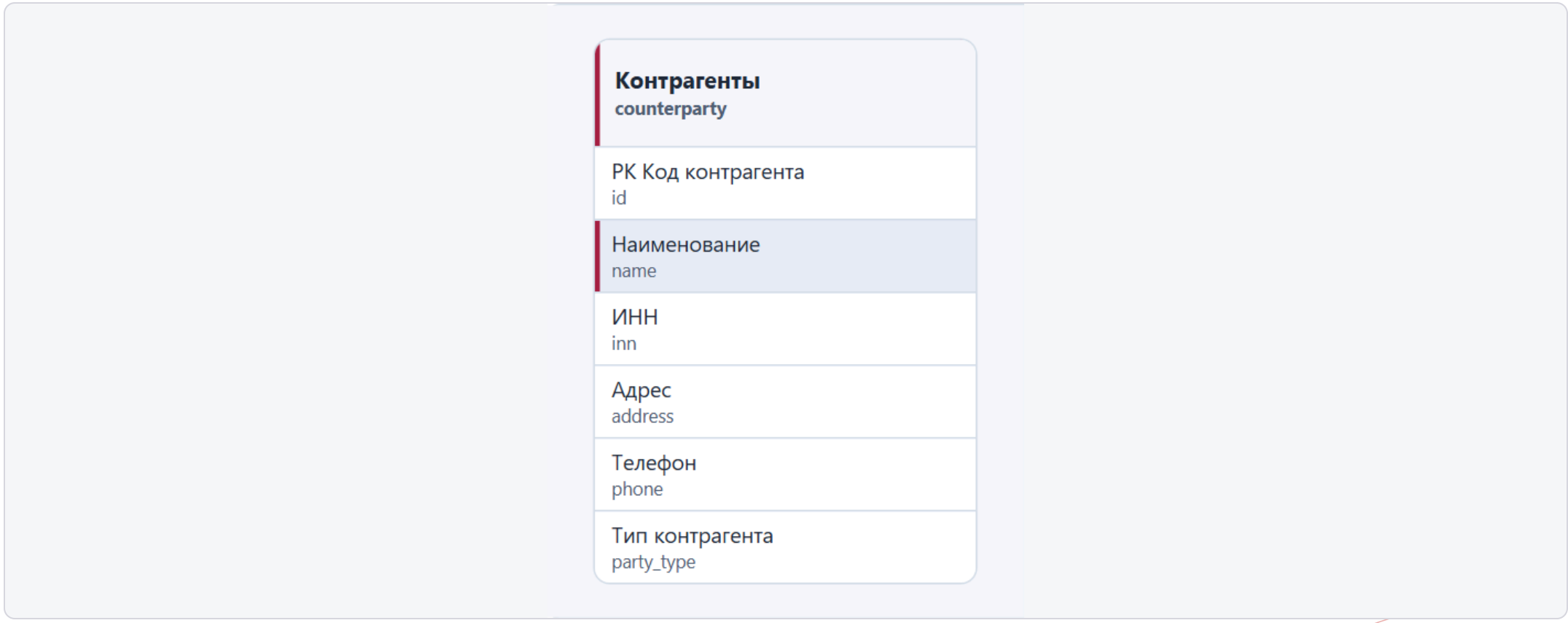


Таблица **counterparty**. Выделено поле **name** из «Заказчики.**json**».

ШАГ 4 ИЗ 12 · 3/3

Отделите сущности от документов

1. Если строк не хватает, нажмите правой кнопкой по существующей строке и выберите «Дублировать» (Duplicate). Затем измените название в новой строке.
2. Сверьте имена с JSON. Поле **addres** соответствует **address**, а **type** — **party_type**. Значения, в том числе код с ведущими нулями и пустой ИНН, остаются в исходном файле.

ЧТО ПРОВЕРИТЬ

В **counterparty** шесть полей. Ключ **id** обозначает одного контрагента. Вы можете показать, где в заказе шапка, а где строка товара.

Откройте: Заказ покупателя.**xlsx** · Заказчики.**json**

ШАГ 5 ИЗ 12 · 1/3

Добавьте номенклатуру и цены

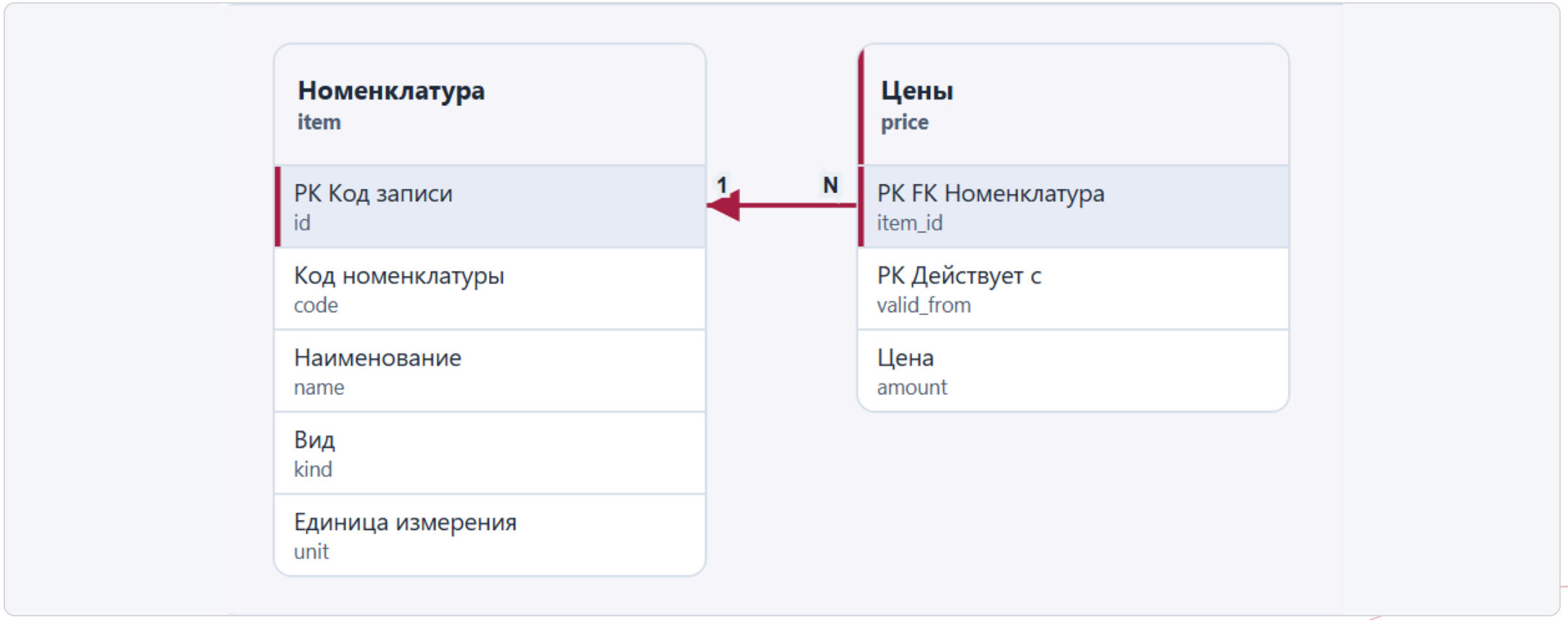
Один ресурс может встречаться в разных спецификациях. Его название хранится в номенклатуре, а цена может меняться со временем.

1. Откройте «Заказ на производство.xlsx». Найдите евровинт в разделе «Потребность в материалах» и его код ФР-00000027. Добавьте таблицу **item** с PK **id** и полями **code**, **name**, **kind**, **unit**.
2. Поле **kind** различает изделие (**product**), материал (**material**) и операцию (**operation**). У стола вид **product**, у евровинта **material**, у сборки **operation**.
3. В «Цены.xlsx» найдите «Евровинт 6,5x5» и цену 595. Добавьте таблицу **price** с полями **item_id**, **valid_from** и **amount**. Проведите связь **price.item_id** → **item.id**.

ШАГ 5 ИЗ 12 · 2/3

Добавьте номенклатуру и цены

Покажите на экране



Цена ссылается на номенклатуру. Ключ **price** состоит из **item_id** и **valid_from**.

ШАГ 5 ИЗ 12 · 3/3

Добавьте номенклатуру и цены

1 Пометьте item_id и valid_from как части одного PK. Даты в прайсе нет. В учебном расчёте используем 01.04.2026 как дату начала действия цен.

ЧТО ПРОВЕРИТЬ

У каждой цены есть ссылка на ресурс. Для одного ресурса можно записать цены с разными датами, и пара «ресурс + дата» уникальна.

Откройте: Заказ на производство.xlsx · Цены.xlsx

ШАГ 6 ИЗ 12 · 1/3

Добавьте спецификацию и состав

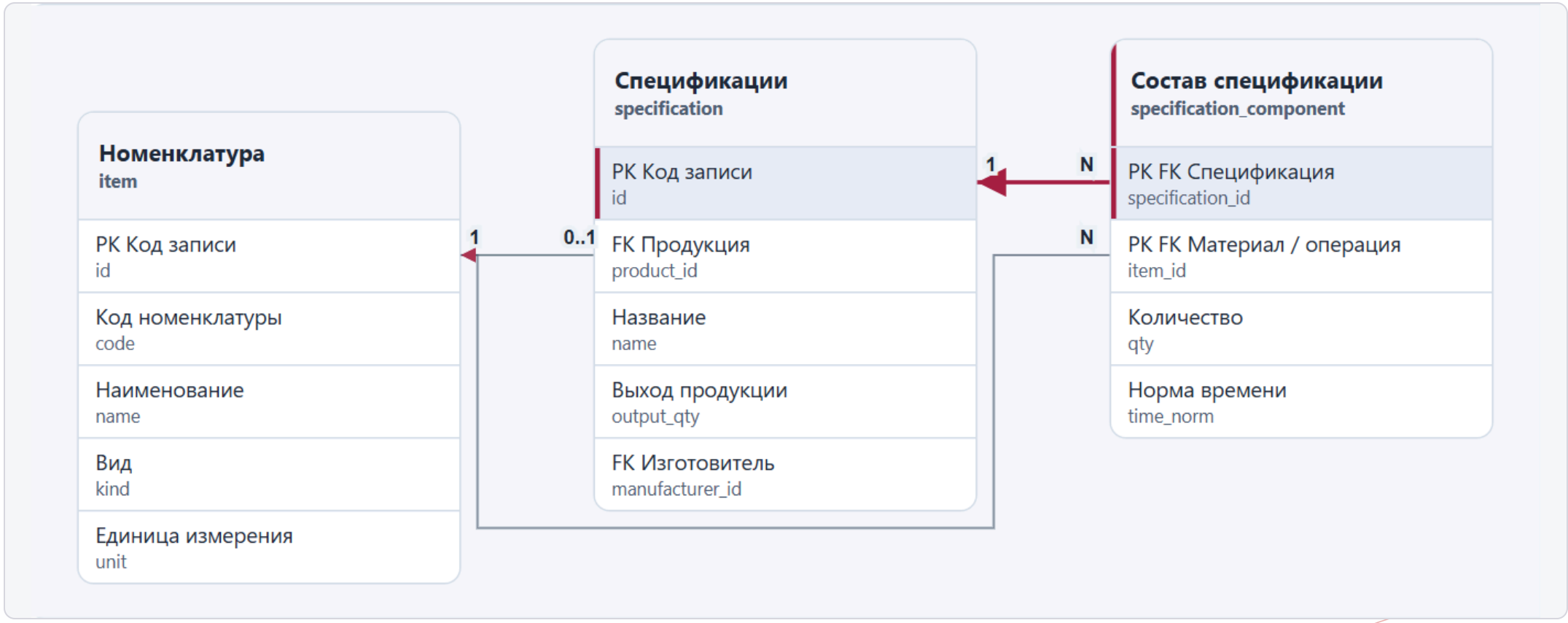
Шапка описывает спецификацию целиком. В её составе каждая строка связывает спецификацию с одним материалом или операцией.

1. Откройте «Спецификация.xlsx». В шапке найдите изделие, изготовителя и название. Добавьте **specification** с PK **id**, полями **name** и **output_qty**. В примере нормы заданы на один стол.
2. Свяжите **product_id** → **item.id** и **manufacturer_id** → **counterparty.id**. Пометьте **product_id** как **UNIQUE**, чтобы у изделия могла быть максимум одна спецификация.
3. В разделе «Состав» найдите евровинт. Добавьте **specification_component**. Проведите связи **specification_id** → **specification.id** и **item_id** → **item.id**. Эти два поля вместе образуют PK.

ШАГ 6 ИЗ 12 · 2/3

Добавьте спецификацию и состав

Покажите на экране



Спецификация, номенклатура и состав. Выделена связь **specification_component.specification_id → specification.id.**

ШАГ 6 ИЗ 12 · 3/3

Добавьте спецификацию и состав

1 Добавьте **qty** для количества по норме и **time_norm** для нормы времени. Для «Сборки модулей» количество равно 1, норма времени — 0,75. Для материалов в примере используем множитель времени 1.

ЧТО ПРОВЕРИТЬ

Строка с евровинтом относится к спецификации стола и ссылается на евровинт в **item**. В другой спецификации у того же евровинта может быть своё количество.

Откройте: Спецификация.**xlsx**

ШАГ 7 ИЗ 12 · 1/4

Добавьте заказ и план производства

Заказ покупателя

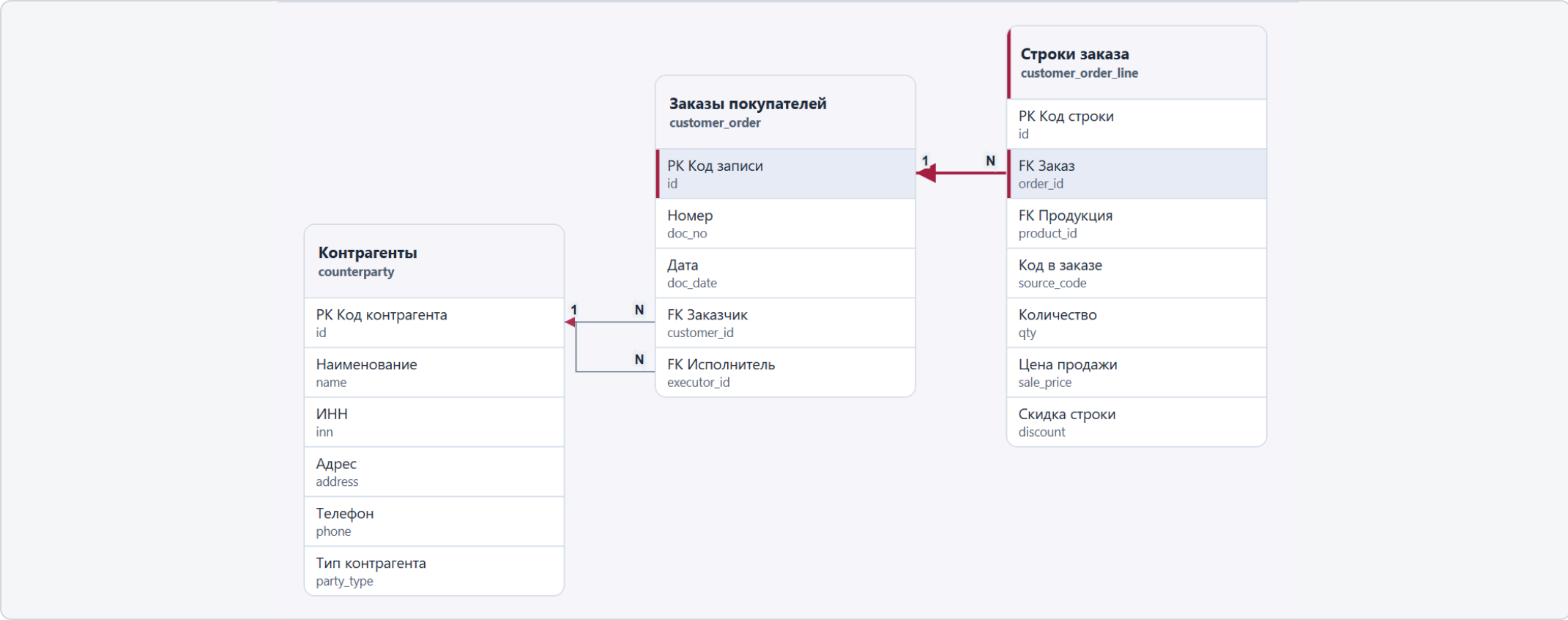
Шапка хранит сведения о документе. Отдельные строки хранят изделия и ресурсы, которые можно связать с номенклатурой.

1. По заголовку заказа добавьте **customer_order** с PK **id**. В шапке разместите номер и дату заказа. Поля **customer_id** и **executor_id** свяжите с **counterparty.id**.
2. По строке с двумя столами добавьте **customer_order_line** с PK **id**. Свяжите **order_id** → **customer_order.id** и **product_id** → **item.id**. Здесь же разместите количество, продажную цену, скидку и исходный код ФР-00000034 в **source_code**.

ШАГ 7 ИЗ 12 · 2/4

Добавьте заказ и план производства

Заказ покупателя



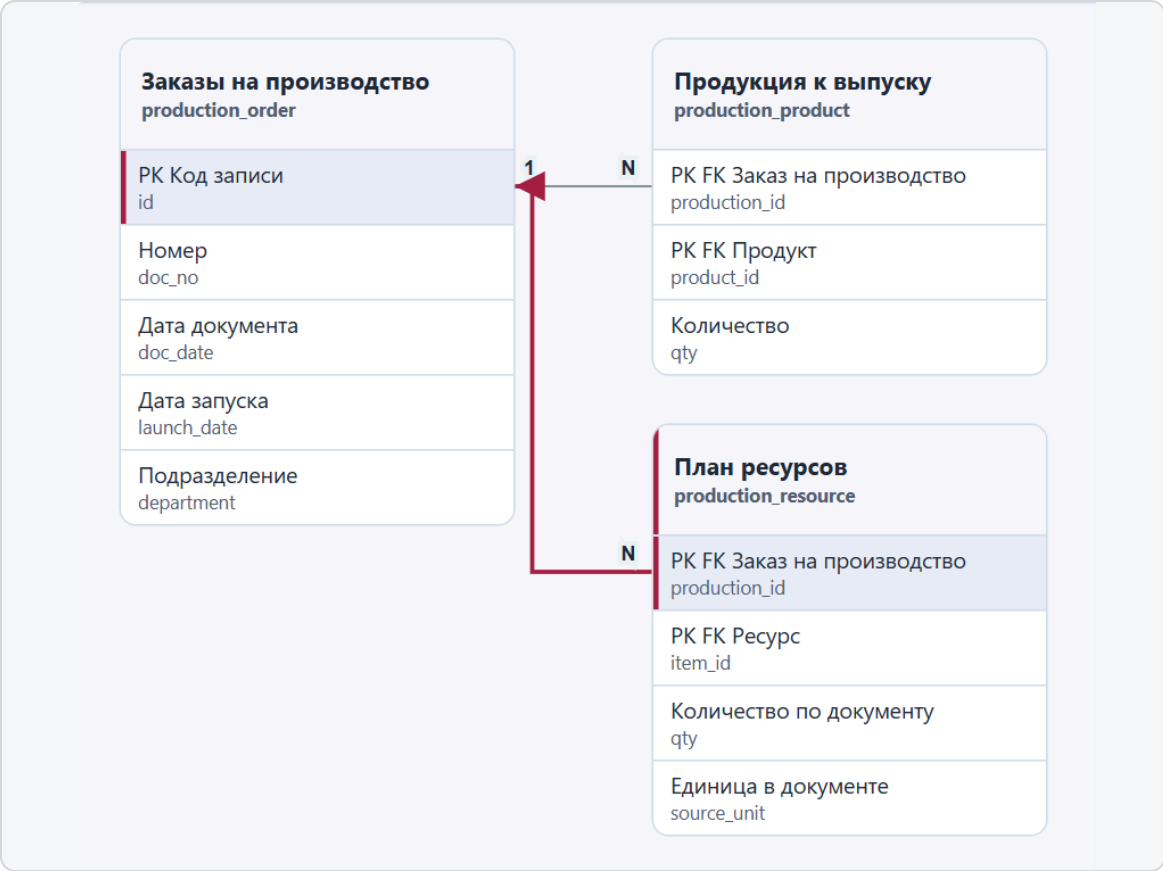
Контрагенты, заказ покупателя и его строки. Выделена связь **customer_order_line.order_id → customer_order.id**.

ШАГ 7 ИЗ 12 · 3/4

Добавьте заказ и план производства

Заказ на производство

1. По заголовку и шапке добавьте production_order с PK id, номером документа, датой, датой запуска и подразделением.
2. Раздел «Продукция, работа» перенесите в модель как production_product, разделы материалов и операций — как production_resource. В обеих таблицах свяжите production_id с production_order.id, а изделие или ресурс — с item.id. Каждая пара внешних ключей образует PK своей таблицы.



Заказ на производство, продукция к выпуску и план ресурсов. Выделена связь production_resource.production_id → production_order.id.

ШАГ 7 ИЗ 12 · 4/4

Добавьте заказ и план производства

Заказ на производство

1. Покажите количество в строках выпуска и плана. В **production_resource.source_unit** сохраните единицу из плана. Например, для упаковки в плане указаны штуки, а в учебном расчёте тариф принят за час.

ЧТО ПРОВЕРИТЬ

Строка продажи ссылается на заказ покупателя. Строка выпуска и строки ресурсов ссылаются на свой производственный заказ. Оба документа о столе связаны с одной записью в **item**.

Откройте: Заказ покупателя.**xlsx** · Заказ на производство.**xlsx**

ШАГ 8 ИЗ 12 · 1/2

Добавьте пользователей

Таблица **users** дополняет учебную схему для входа и ролей в задании 4. У автора заметки будет ссылка на эту учётную запись.

1. Откройте Приложение 1 к заданию 4. Найдите требования к входу, ролям и блокировке.
2. Добавьте **users** с PK **id**. Для входа отметьте уникальный **login** и поле **password_hash** для хеша пароля.
3. Добавьте **role** для роли, **failed_attempts** для числа неудачных попыток и **is_locked** для состояния блокировки.

ЧТО ПРОВЕРИТЬ

На схеме видны ключ **id**, уникальный логин, хеш пароля, роль и два поля, по которым приложение ведёт блокировку.

Откройте: Приложение **1.pdf**

ШАГ 8 ИЗ 12 · 2/2

Добавьте пользователей

Покажите на экране

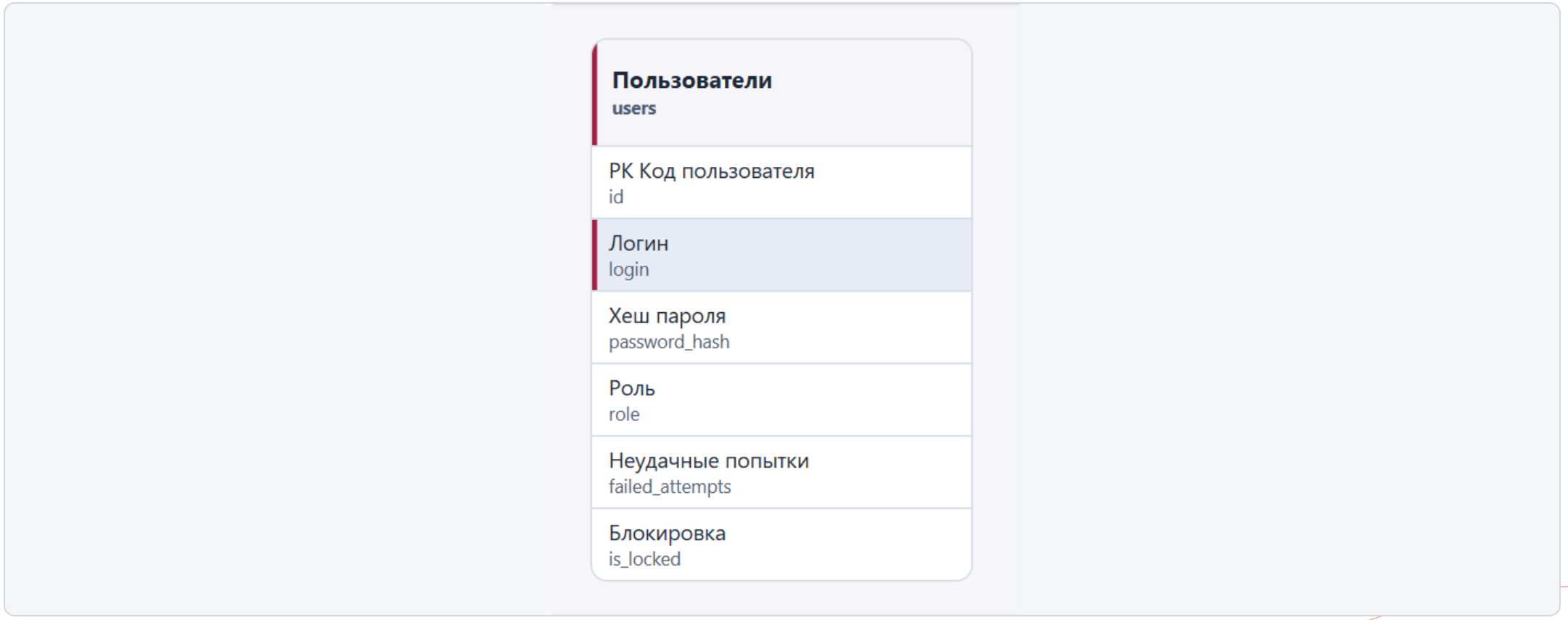


Таблица **users** с логином, хешем пароля, ролью и полями блокировки.

ШАГ 9 ИЗ 12 · 1/2

Добавьте заметки

Таблица **notes** дополняет учебную схему для задания 5. Через автора заметки API получает его логин.

1. В приложении к заданию 5 найдите раздел «Структура таблицы базы данных». В **notes** добавьте поля **id**, **title**, **content**, **id_user** и **created_at**.
2. Отметьте **id** как **PK**, **id_user** как **FK**. Проведите связь **notes.id_user** → **users.id**. Каждая заметка имеет одного автора. У пользователя может быть несколько заметок.
3. В ответе API **title_user** составляется из заголовка и логина, а **formatted_date** получается из даты создания. На схеме **notes** остаются исходные поля.

ЧТО ПРОВЕРИТЬ

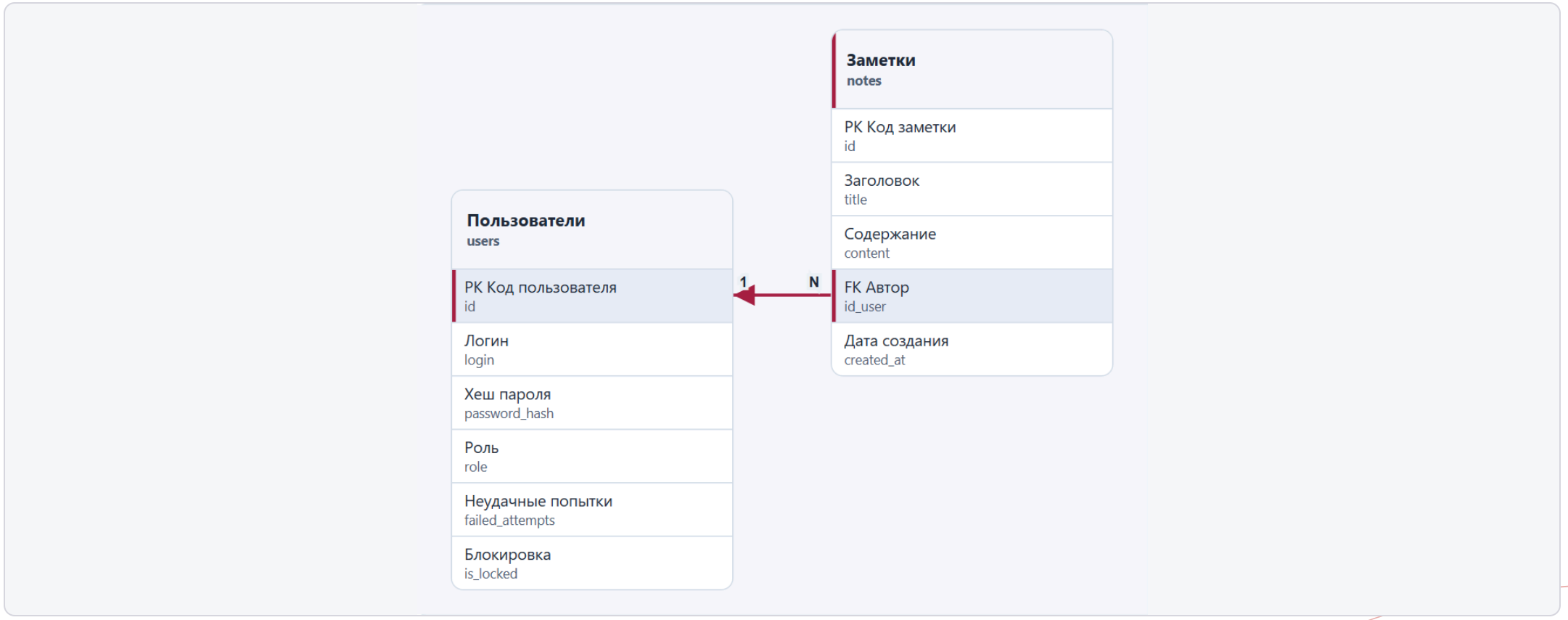
В **notes** пять полей, а связь автора заканчивается на **users.id**. Поля ответа API можно получить из **notes** и **users**.

Откройте: Прил_5_О3_КИМ_09.02.07-5-2027.pdf

ШАГ 9 ИЗ 12 · 2/2

Добавьте заметки

Покажите на экране



Связь **notes.id_user → users.id**. У каждой заметки один автор.

ШАГ 10 ИЗ 12

Сверьте свою ER-диаграмму

В разборе ниже выберите нужное поле в режиме «Поэтапное построение». Чтобы увидеть все таблицы, переключитесь на «Готовая схема».

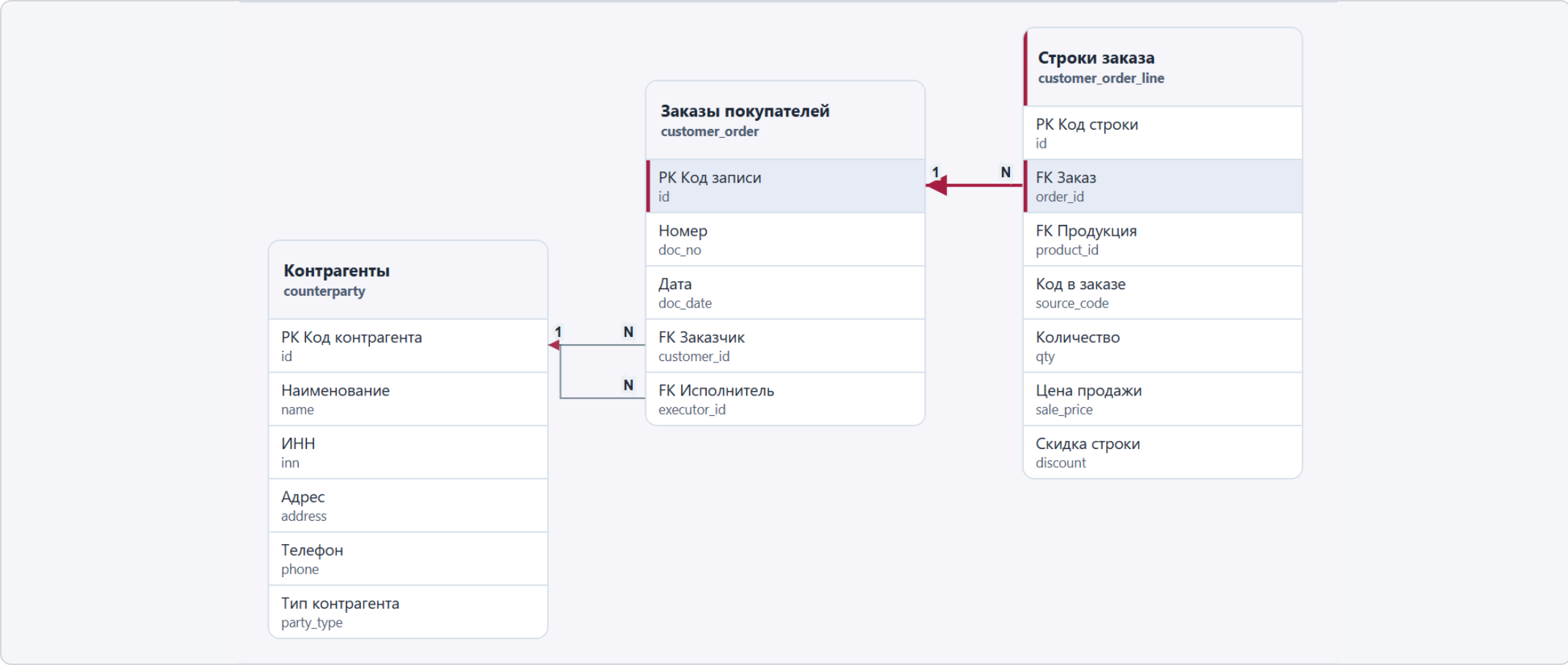
1. Строки заказа → Заказ. Проверьте, что позиция с товаром ведёт к шапке заказа.
2. Состав спецификации → Материал / операция. Проверьте, что строка состава ведёт к одному ресурсу из номенклатуры.
3. Заметки → Автор. Проверьте связь **id_user** с ключом пользователя.

ЧТО ПРОВЕРИТЬ

На своей схеме вы можете проследить эти три связи от внешнего ключа до первичного.

ШАГ 10 · ФРАГМЕНТ СХЕМЫ

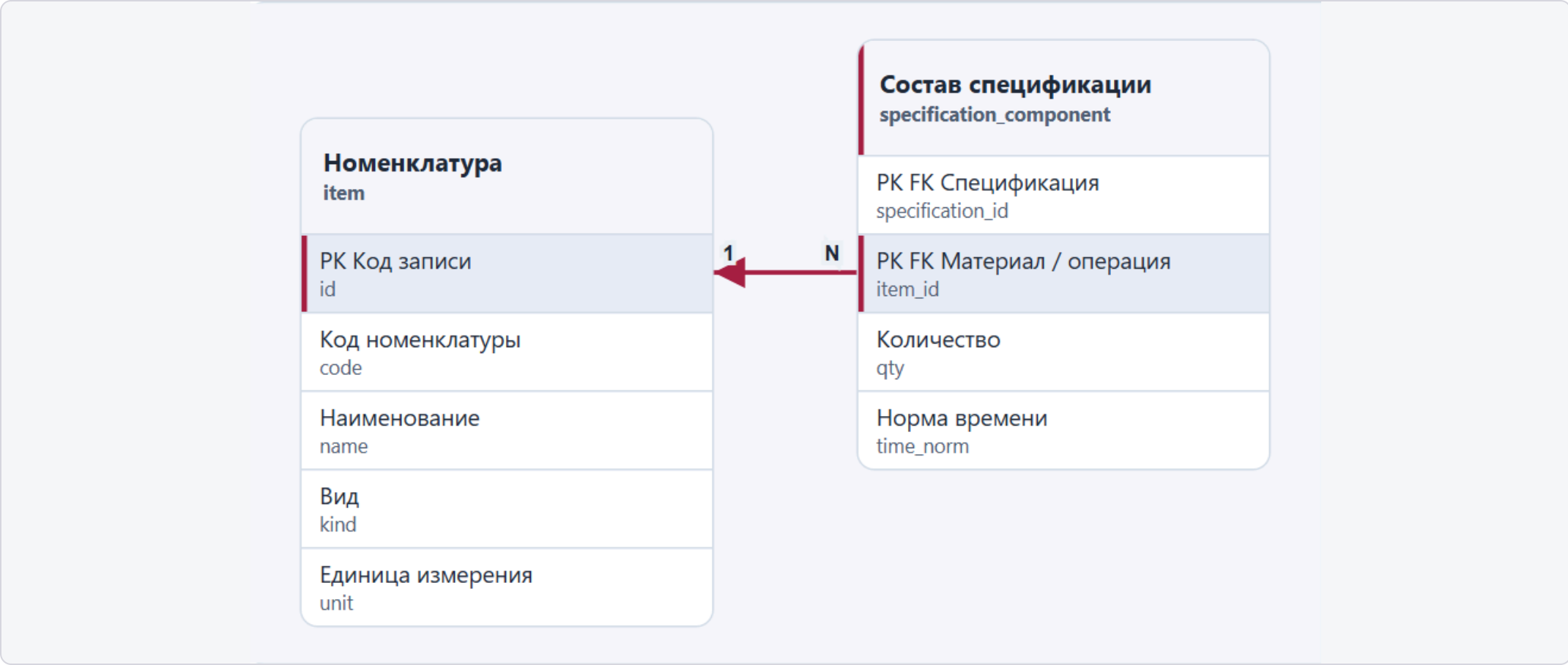
Строка заказа и его шапка



Контрагенты, заказ покупателя и его строки. Выделена связь **customer_order_line.order_id → customer_order.id**.

ШАГ 10 · ФРАГМЕНТ СХЕМЫ

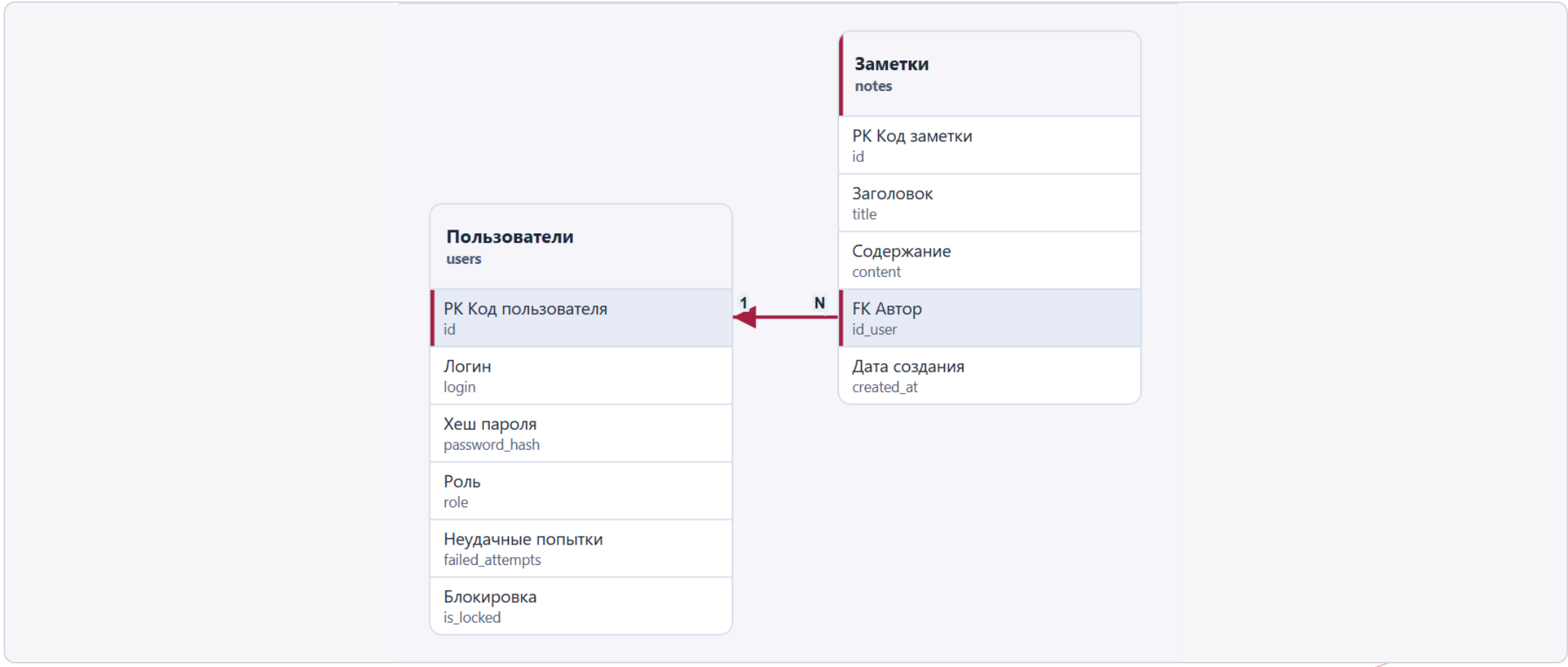
Строка состава и ресурс



Норма **qty** хранится в составе, а название ресурса — в **item**. Выделена связь **specification_component.item_id → item.id**.

ШАГ 10 · ФРАГМЕНТ СХЕМЫ

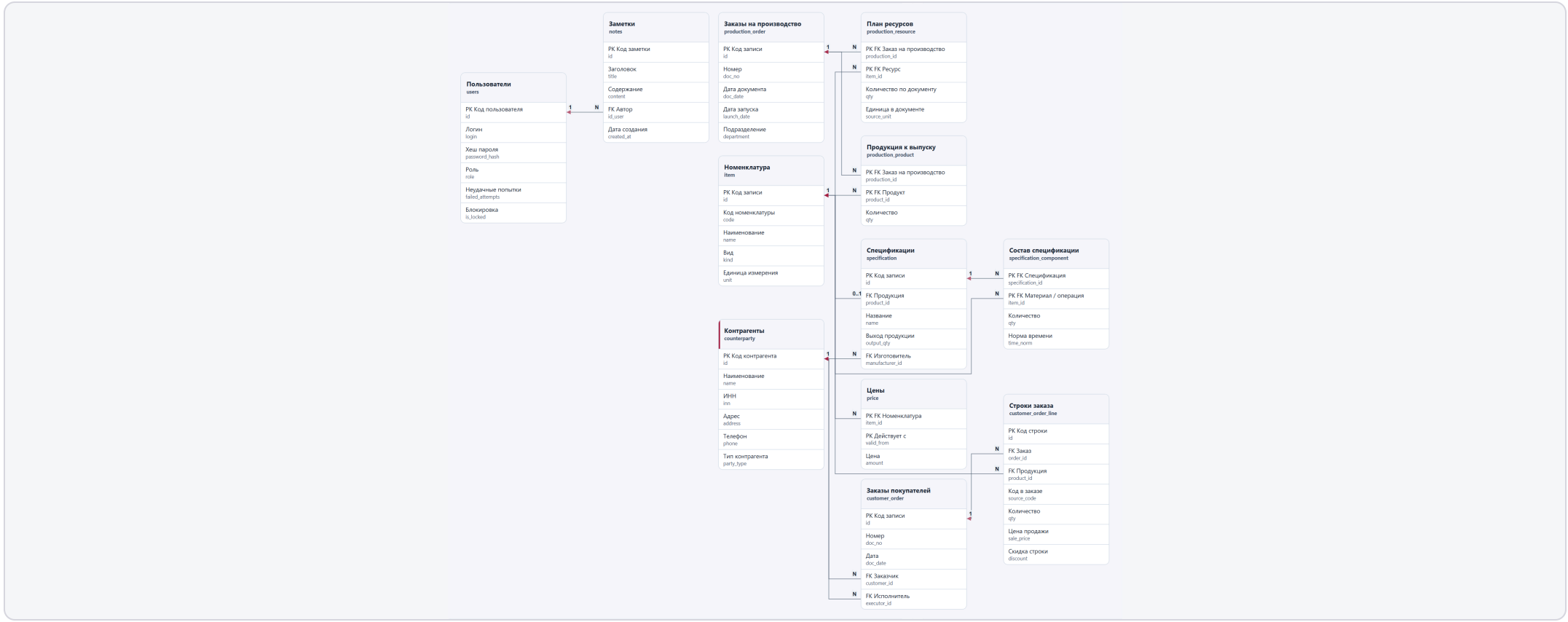
Заметка и её автор



Связь **notes.id_user → users.id**. У каждой заметки один автор.

ШАГ 10 · СХЕМА

Общая ER-диаграмма



Готовая схема с 12 таблицами, полями, ключами и связями.

ER-диаграмма · PDF

ШАГ 11 ИЗ 12 · 1/3

Проверьте нормализацию и связи

Разделите шапку заказа и его позиции

Место поля определяется тем, от какого ключа зависит его значение. Проверим это на заказе № 1 и на составе спецификации.

1. Представьте, что после двух столов в заказе № 1 добавилась ещё одна позиция. Если хранить всё в одном списке, номер, дата и покупатель повторятся. В **customer_order** оставьте одну шапку. В **customer_order_line** добавьте две позиции с одинаковым **order_id** и своими **id**, изделиями и количеством.
2. В **draw.io** соедините **customer_order_line.order_id** и **customer_order.id** линией из библиотеки **ER**. Поставьте «один» со стороны шапки и «много» со стороны позиций. На диаграмме проследите обе позиции до одной шапки.

КИМ, стр. 30

ШАГ 11 ИЗ 12 · 2/3

Проверьте нормализацию и связи

Проверьте зависимости для 2НФ и 3НФ

1. 2НФ. В specification_component ключ состоит из specification_id и item_id. Норма евровинтов может отличаться в другой спецификации, поэтому qty зависит от обеих частей. Название евровинта определяется одним item_id и остаётся в item.
2. 3НФ. В customer_order по id заказа находим customer_id, а по нему телефон покупателя. Проведите customer_order.customer_id → counterparty.id. Если телефон изменится, достаточно исправить одну запись в counterparty.

КИМ, стр. 30

ШАГ 11 ИЗ 12 · 3/3

Проверьте нормализацию и связи

Сверьте связи на своей схеме

1. Для каждой линии найдите FK, соответствующий ключ и кратность. У заказа может быть несколько позиций, у позиции один заказ. Между изделием и спецификацией в учебной схеме кратность «один к нулю или одному».

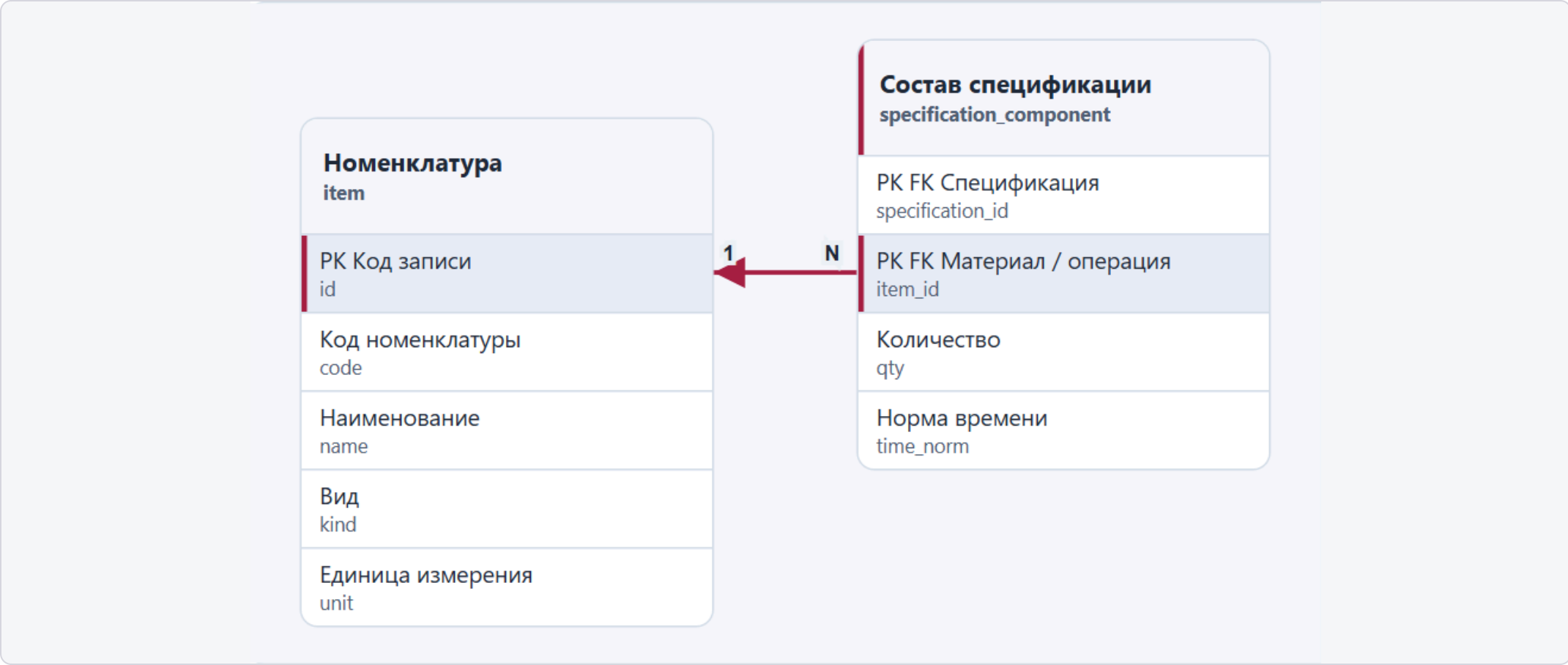
ЧТО ПРОВЕРИТЬ

Две позиции ведут к одной шапке заказа. Телефон хранится у контрагента, название материала — в **item**, а норма — у пары «спецификация + ресурс».

КИМ, стр. 30

ШАГ 11 · ФРАГМЕНТ СХЕМЫ

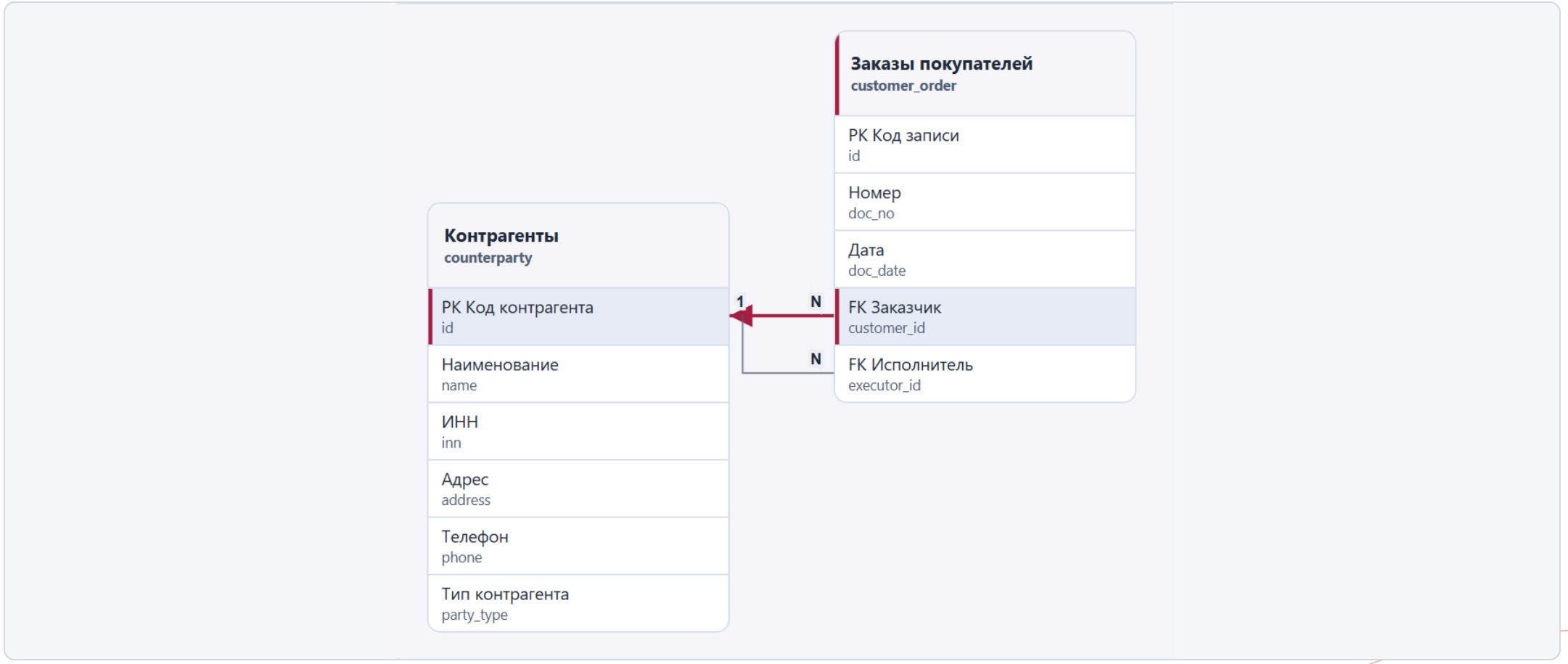
2НФ · Норма расхода зависит от пары ключей



Норма **qty** хранится в составе, а название ресурса — в **item**. Выделена связь **specification_component.item_id → item.id**.

ШАГ 11 · ФРАГМЕНТ СХЕМЫ

3НФ · Телефон хранится у контрагента



Заказ ссылается на покупателя по **customer_id**. Поле **phone** находится в **counterparty**.

ШАГ 12 ИЗ 12

Экспорт в draw.io

По итоговому PDF преподаватель проверяет таблицы, поля, ключи и связи. Типы данных на этом этапе необязательны.

1. Сохраните редактируемую схему в папку «Результаты» под именем **ER-диаграмма.drawio**. Выберите «Файл» → «Экспортировать как» → **PDF**.
2. Если схема занимает несколько страниц, выберите экспорт всех страниц. Сохраните **ER-диаграмма.pdf** в папку «Результаты».
3. Откройте PDF. Увеличьте масштаб и проверьте, что все таблицы на месте, названия полей и ключей читаются, а обозначения связей видны полностью.

ЧТО ПРОВЕРИТЬ

В папке «Результаты» открывается **ER-диаграмма.pdf**. В нём видны таблицы, поля, **PK**, **FK** и кратности.

КИМ, стр. 30

РЕЗУЛЬТАТ ЭТАПА

Что должно остаться после этапа

ER-диаграмма.pdf открывается и читается. ER-диаграмма.drawio остаётся для правок.

ЧТО ПРОВЕРИТЬ

ER-диаграмма.pdf с таблицами, полями, ключами и связями.

ЗАДАНИЕ 2 · 30 МИН

Создание БД и импорт

Схема из задания 1 становится таблицами в PostgreSQL. Импортируем клиентов после создания таблицы.

ЧТО ПРОВЕРИТЬ

В **counterparty** шесть строк. Первый **id** — **000000001**, ведущие нули сохранены. Есть файл выгрузки **dkip2027-postgresql.sql**.

УСЛОВИЕ ИЗ КИМ

Разработка базы данных на основании ER-диаграммы

Создайте базу данных на основании разработанной ER-диаграммы, используя предпочтительную платформу, на сервере баз данных, который вам предоставлен. Создайте таблицы основных сущностей, атрибуты, отношения и необходимые ограничения.

После создания базы данных требуется импортировать данные из файла "Заказчики.json".

Приложение: Прил_2_О3_КИМ_09.02.07-5-2027.rar

КИМ · стр. 30–31 Исходные файлы · ZIP

ШАГ 1 ИЗ 10 · 1/2

Откройте редактор базы данных

pgAdmin подключается к запущенному серверу PostgreSQL.

1. Откройте pgAdmin и раскройте Servers. Дважды нажмите настроенный учебный сервер. Введите пароль, если pgAdmin попросит.
2. Если подключения нет, нажмите правой кнопкой на Servers → Register → Server. На вкладке Connection укажите адрес, порт и имя пользователя, выданные для вашего сервера. Нажмите Save.
3. Раскройте Databases. Убедитесь, что соединение открыто.

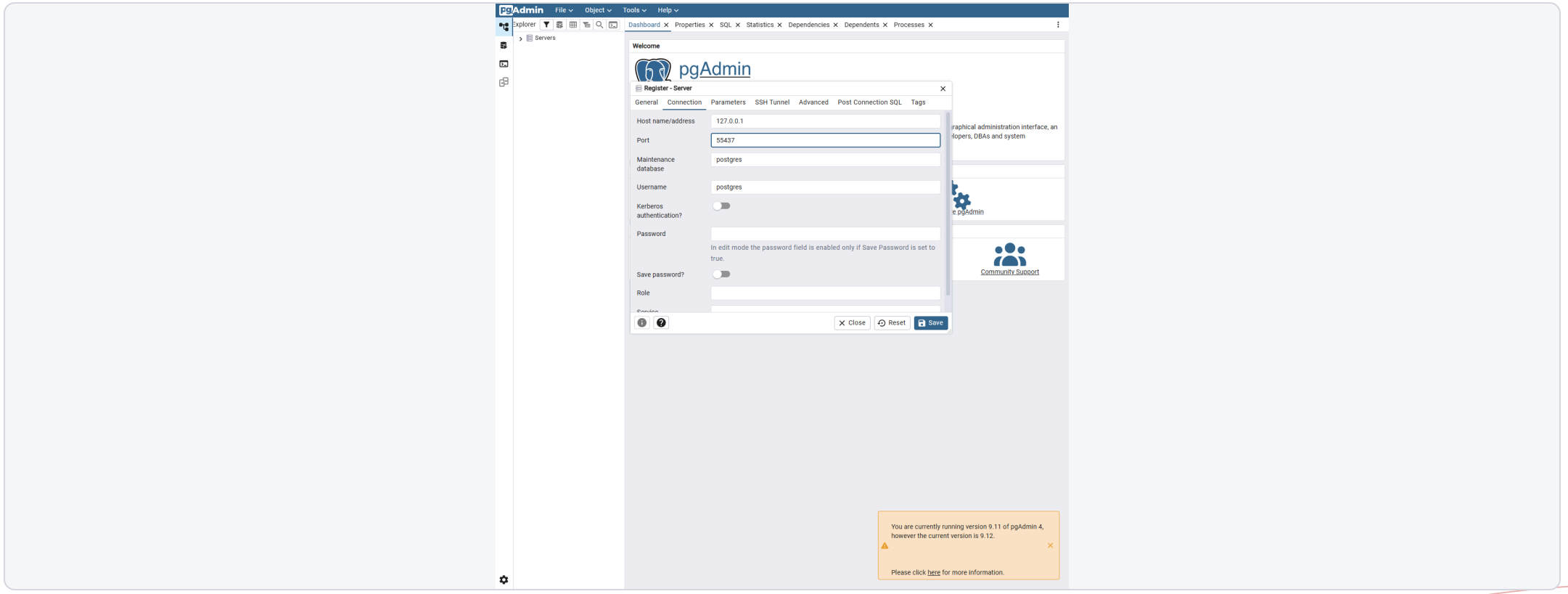
ЧТО ПРОВЕРИТЬ

Редактор показывает список баз данных. Теперь можно создать свою учебную базу.

ШАГ 1 ИЗ 10 · 2/2

Откройте редактор базы данных

Покажите на экране



pgAdmin: регистрация подключения к локальному **PostgreSQL**. На снимке отдельный учебный сервер с портом **55437**; укажите порт своего сервера, обычно **5432**.

ШАГ 2 ИЗ 10

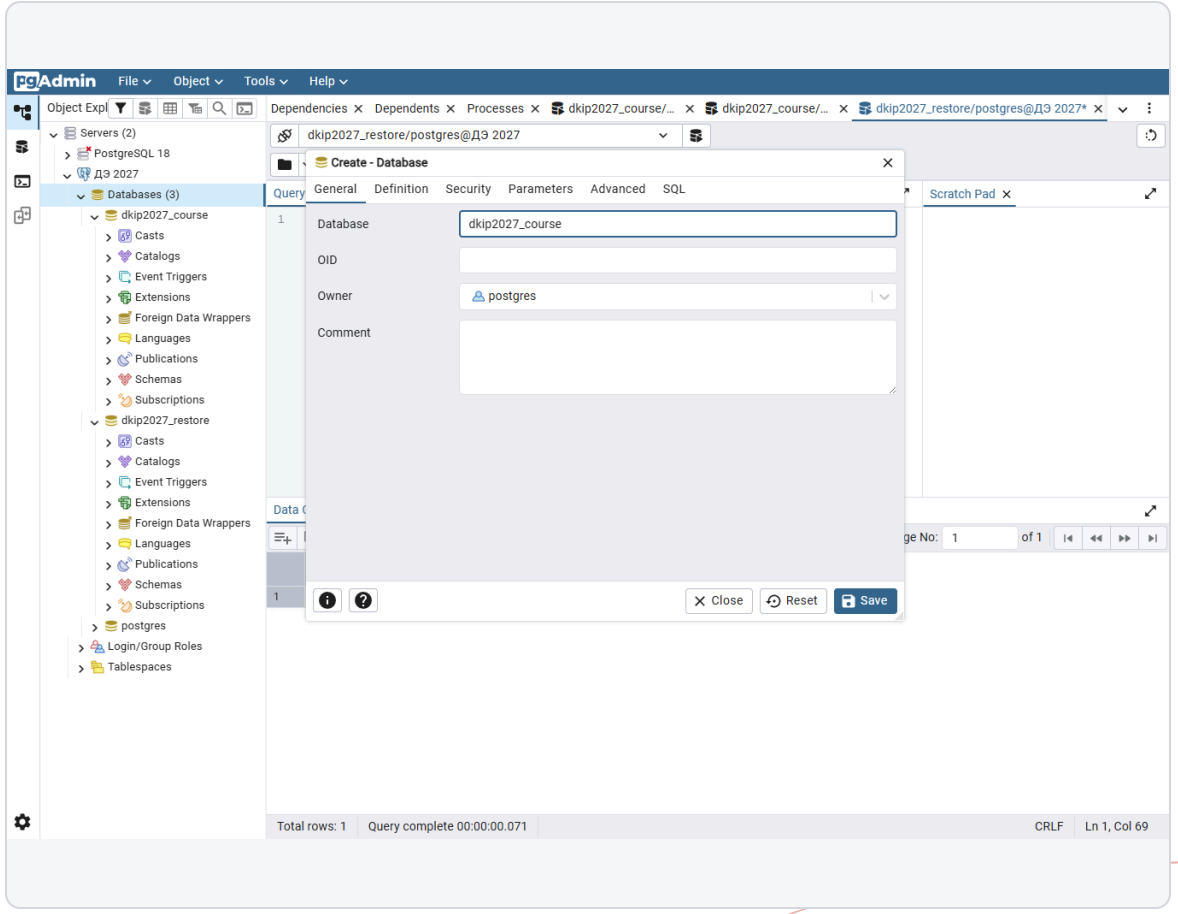
Создайте пустую учебную базу

Имя выбранной базы помогает проверить, куда попадут таблицы и данные.

1. В pgAdmin нажмите правой кнопкой по **Databases** → **Create** → **Database**. В поле **Database** введите **dkip2027_course**. Нажмите **Save**.
2. Если ваша роль не может создавать базы, откройте пустую учебную базу, которую выдал преподаватель. Раскройте её → **Schemas** → **public** → **Tables** и проверьте список.

ЧТО ПРОВЕРИТЬ

Выбрана **dkip2027_course**. Список таблиц в новой базе пуст.



Поля создания отдельной учебной базы данных. Маршрут: **WPF / PostgreSQL**.

ШАГ 2 · НЕОБЯЗАТЕЛЬНАЯ ПРОВЕРКА SQL

Создайте пустую учебную базу

```
SELECT current_database();
```

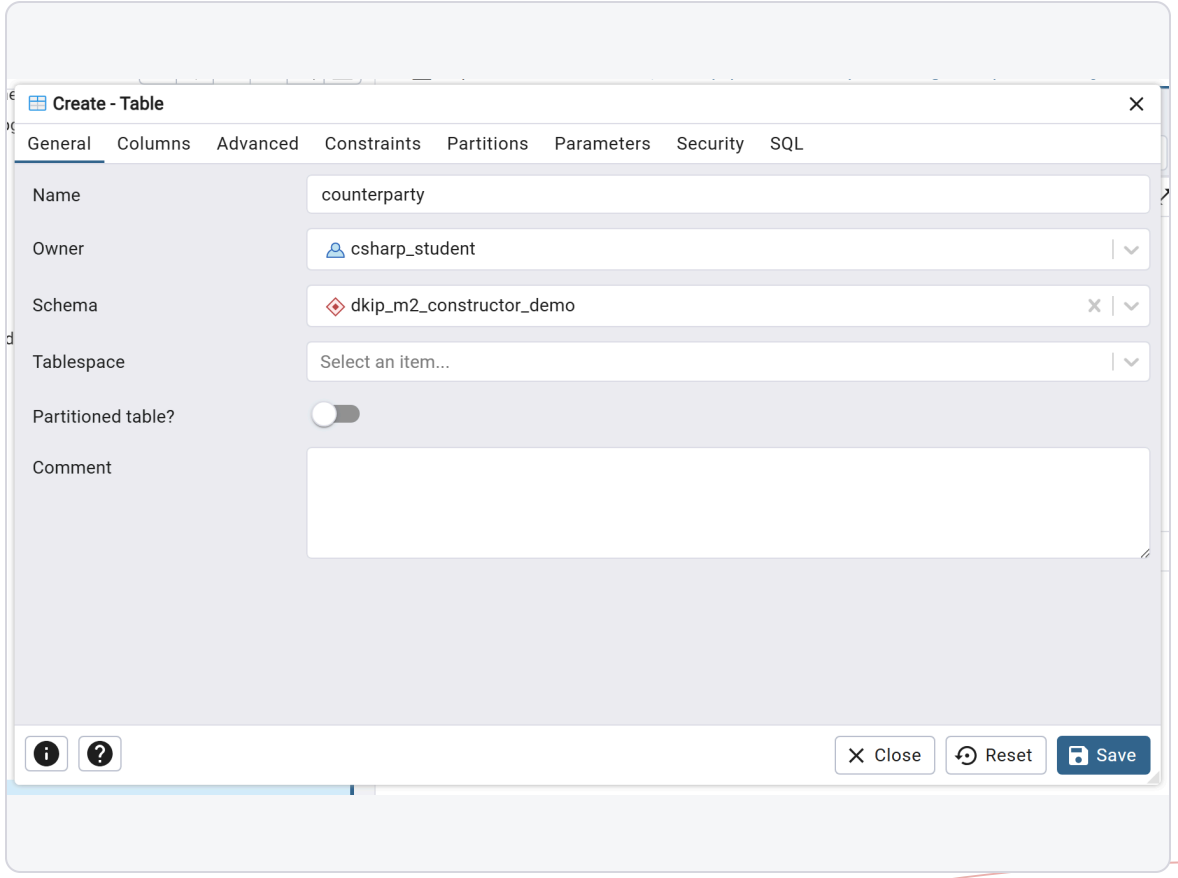
ШАГ 3 ИЗ 10 · 1/3

Создайте counterparty в конструкторе

Откройте форму

Первичный ключ отличает записи. Текстовый тип сохраняет ведущие нули в кодах.

1. В своей базе раскройте **Schemas** → нужная схема (обычно **public**). Нажмите правой кнопкой по **Tables** → **Create** → **Table**. На вкладке **General** введите **counterparty** и перейдите в **Columns**.



pgAdmin, форма **Create Table**. На вкладке **General** введено имя **counterparty** и выбрана учебная схема.

ШАГ 3 ИЗ 10 · 2/3

Создайте counterparty в конструкторе

Добавьте поля

1. Создайте шесть полей в таком порядке: id (32), name (255), inn (20), address (500), phone (64), party_type (24). Число в скобках — длина текстового поля.
2. Нажимайте «+» в **Columns** для добавления строки. Для каждого поля выберите **character varying** и укажите **Length**. Включите **Not NULL** для всех полей и **Primary key** для id.
3. Нажмите **Save**. При необходимости обновите **Tables** и раскройте **counterparty** → **Columns**.

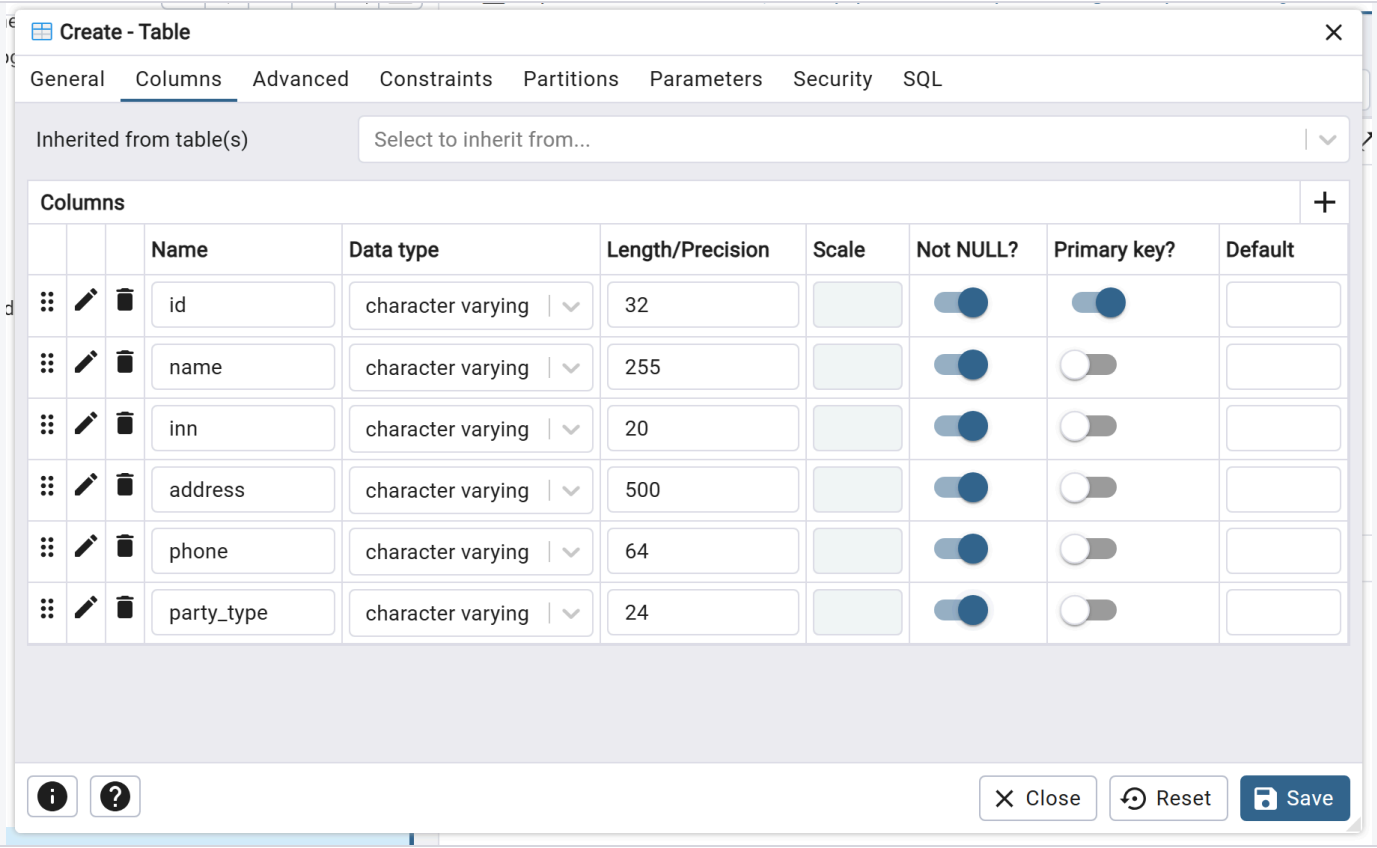
ЧТО ПРОВЕРИТЬ

В базе появилась пустая **counterparty**. Поле **id** имеет текстовый тип и отмечено как первичный ключ.

ШАГ 3 ИЗ 10 · 3/3

Создайте counterparty в конструкторе

Добавьте поля



Вкладка **Columns** в **pgAdmin**. Для шести полей **counterparty** заданы текстовые типы и длины, **id** отмечен как первичный ключ.

ШАГ 4 ИЗ 10 · 1/5

Создайте остальные таблицы и связи

Перенесите таблицы из ER

Таблицы для ссылок создают раньше таблиц с внешними ключами.

1. Выбирайте таблицы в разборе ниже по порядку. Для каждой откройте форму создания, перенесите поля из карточки и сохраните таблицу. Начните с **item**, если **counterparty** уже создана на предыдущем шаге.
2. Для составного ключа откройте **Constraints** → **Primary Key** в форме таблицы. Нажмите «+», откройте созданную строку карандашом и в **Definition** → **Columns** добавьте оба поля в один ключ. Для specification_component это specification_id и item_id.

ШАГ 4 ИЗ 10 · 2/5

Создайте остальные таблицы и связи

Переведите одну строку состава в поля таблицы

1. В разборе выберите specification_component. Поля specification_id и item_id получают тип integer, как ключи specification.id и item.id. Сохраните их в одном PK. Каждое поле может повторяться в разных строках, их пара — уникальна.
2. Для qty и time_norm возьмите numeric(14,3). Три знака после точки позволяют сохранить норму евровинтов 0,012. NOT NULL требует значение, а CHECK (qty > 0) и CHECK (time_norm > 0) проверяют положительную норму.

ШАГ 4 ИЗ 10 · 3/5

Создайте остальные таблицы и связи

Переведите одну строку состава в поля таблицы

1. Задайте два FK из карточки. Первый указывает, в какой спецификации находится строка. Второй указывает, какой ресурс используется. После восстановления выгрузки проверите составной ключ на двух строках.

ШАГ 4 ИЗ 10 · 4/5

Создайте остальные таблицы и связи

Проведите связи

1. В дереве зависимой таблицы нажмите правой кнопкой по **Constraints** → **Create** → **Foreign Key**. В **General** задайте имя ключа. В **Columns** для примера **notes** укажите **Local column** — **id_user**, **References** — **users** в своей схеме, **Referencing** — **id**. Нажмите «+» и **Save**. Повторите для каждой связи из карточки.
2. Выберите свою базу и откройте **Tools** → **ERD For Database**, если этот пункт доступен. Сверьте связи на схеме.

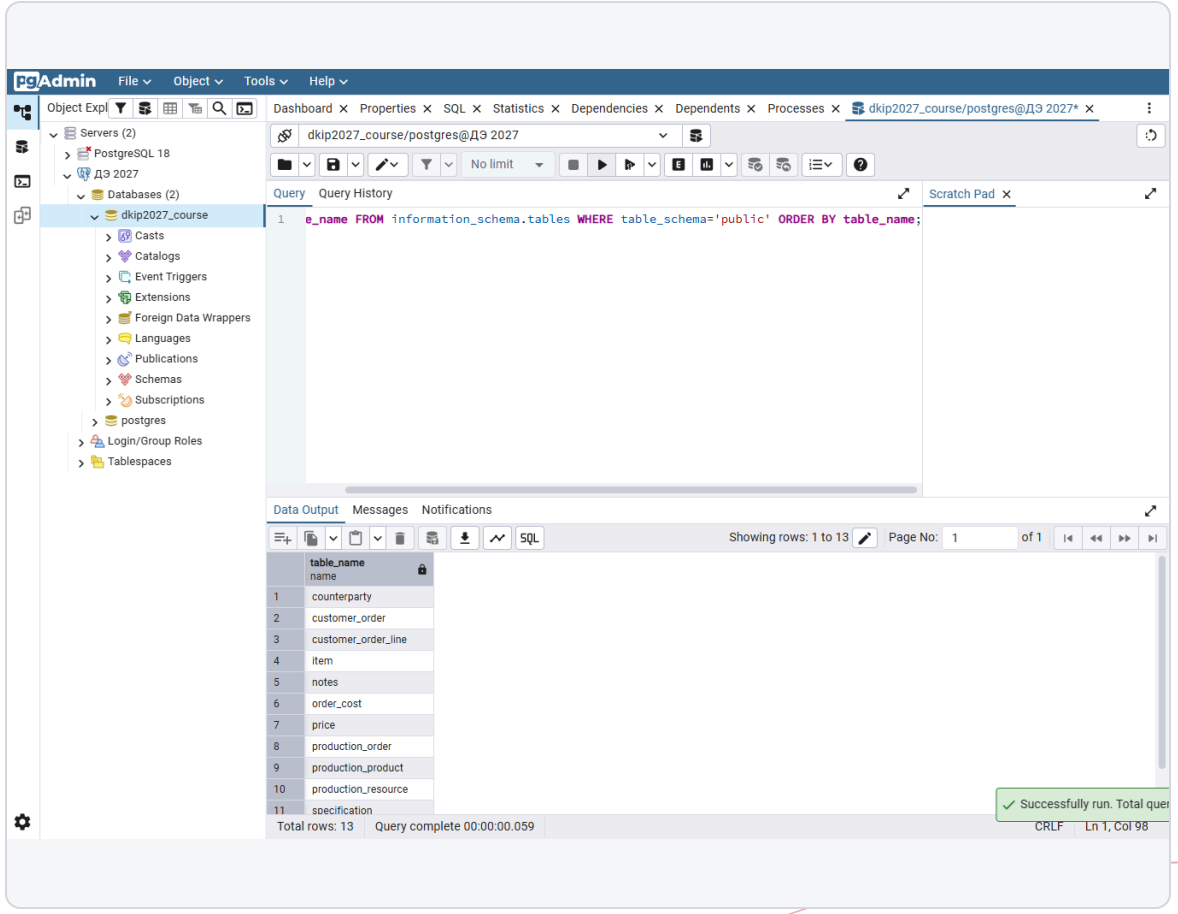
ШАГ 4 ИЗ 10 · 5/5

Создайте остальные таблицы и связи

Покажите результат на экране

ЧТО ПРОВЕРИТЬ

В редакторе видны таблицы с вашей ER-диаграммы и заданные связи. Учебный пример содержит 12 таблиц.



Созданные таблицы учебной базы в редакторе СУБД. Маршрут: **WPF / PostgreSQL**.

ШАГ 5 ИЗ 10 · 1/2

Проверьте первичные и внешние ключи

Первичный ключ отличает строки, а внешний показывает, на какую таблицу и поле ссылается запись.

1. Раскройте **specification_component** → **Constraints**. Откройте свойства **Primary Key** и проверьте столбцы **specification_id** и **item_id**.
2. Раскройте **Constraints** в каждой зависимой таблице и откройте внешние ключи. Начните с **notes.id_user** → **users.id**. Сверьте поля каждой связи по карточкам таблиц.

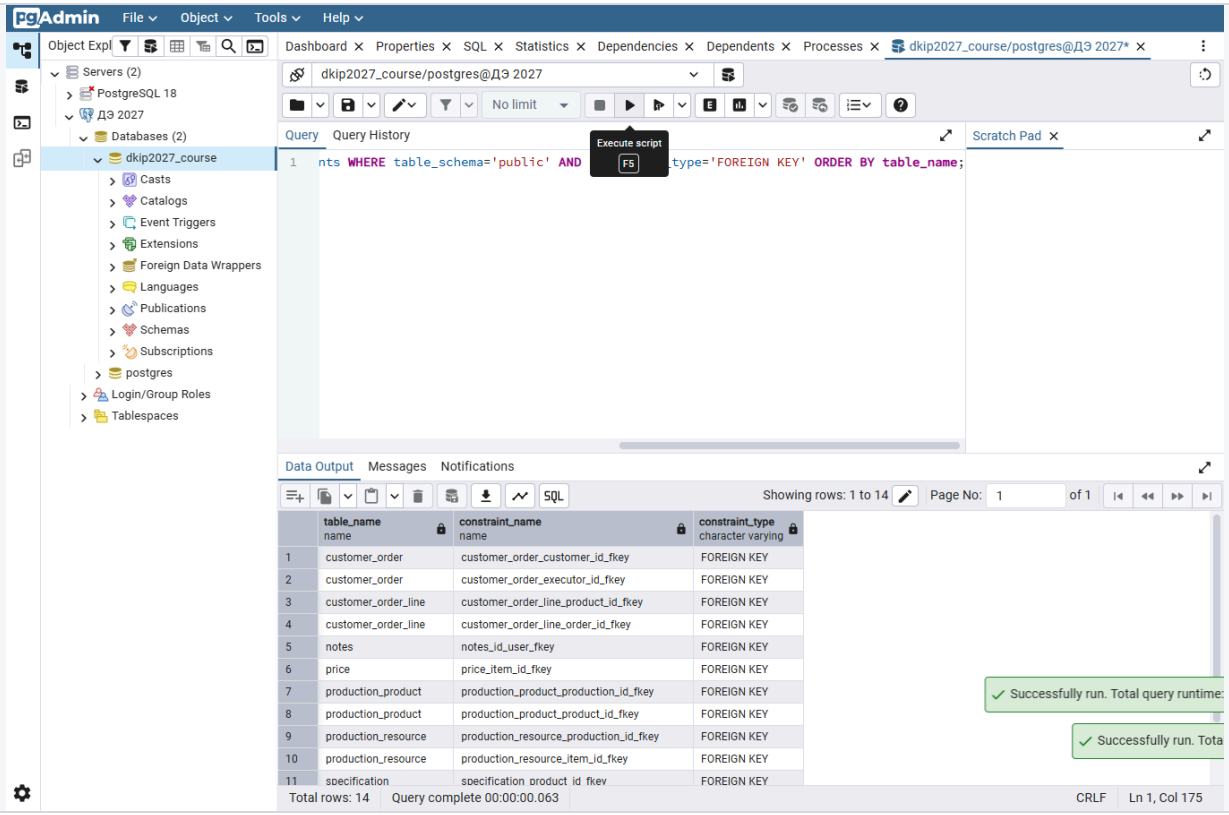
ЧТО ПРОВЕРИТЬ

В примере **12** таблиц, у каждой есть первичный ключ. В каталоге внешних ключей должно быть **14** пар полей.

ШАГ 5 ИЗ 10 · 2/2

Проверьте первичные и внешние ключи

Покажите на экране



Проверка по каталогу **PostgreSQL** показывает **14** внешних ключей учебной схемы. Поля отдельной связи открываются через **Constraints** соответствующей таблицы.

ШАГ 6 ИЗ 10

Подготовьте CSV с заказчиками

Каждый объект исходного JSON станет одной строкой CSV и затем одной строкой в **counterparty**.

1. Откройте «Заказчики.json» из папки «Исходные данные → Задание 2». В первой записи найдите код **000000001** и пустой ИНН.
2. Откройте «Справка» → «**CSV через Excel**». Загрузите JSON в **Power Query**, задайте полям тип «Текст» и сохраните «Заказчики.csv» в формате **CSV UTF-8**.
3. Откройте CSV в Блокноте. Проверьте порядок заголовков **id, name, inn, address, phone, party_type**. Запомните разделитель полей.

ЧТО ПРОВЕРИТЬ

В «Заказчики.csv» есть заголовок и шесть записей. Первый код сохранился как **000000001**.

Откройте: Заказчики.json

ШАГ 7 ИЗ 10 · 1/2

Импортируйте CSV в counterparty

Редактор базы переносит строки из подготовленного CSV в уже созданную таблицу.

1. Нажмите правой кнопкой по пустой **counterparty** → **Import/Export Data**. В **General** выберите **Import**, «Заказчики.csv», **csv** и **UTF8**.
2. В **Options** включите **Header**, выберите разделитель из файла и двойную кавычку (") в **Quote** и **Escape**. В **Columns** добавьте **inn** в **NOT NULL columns**.
3. Нажмите **OK** и дождитесь **Process completed**. Подробные настройки по шагам показаны в «Справке» → «**CSV через Excel**».

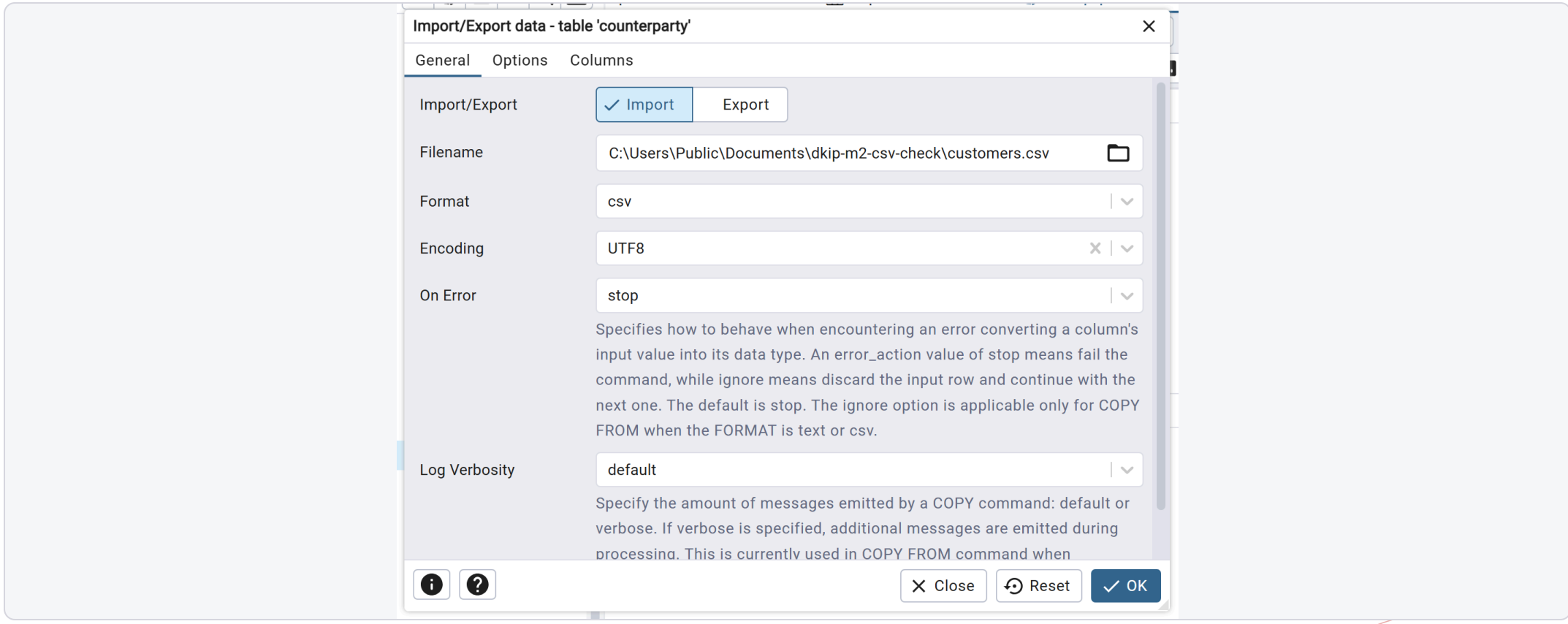
ЧТО ПРОВЕРИТЬ

Редактор завершил импорт. В **counterparty** должно быть шесть строк.

ШАГ 7 ИЗ 10 · 2/2

Импортируйте CSV в counterparty

Покажите на экране



pgAdmin, вкладка **General**. Для импорта в **counterparty** выбран учебный файл **CSV** в кодировке **UTF8**.

ШАГ 8 ИЗ 10

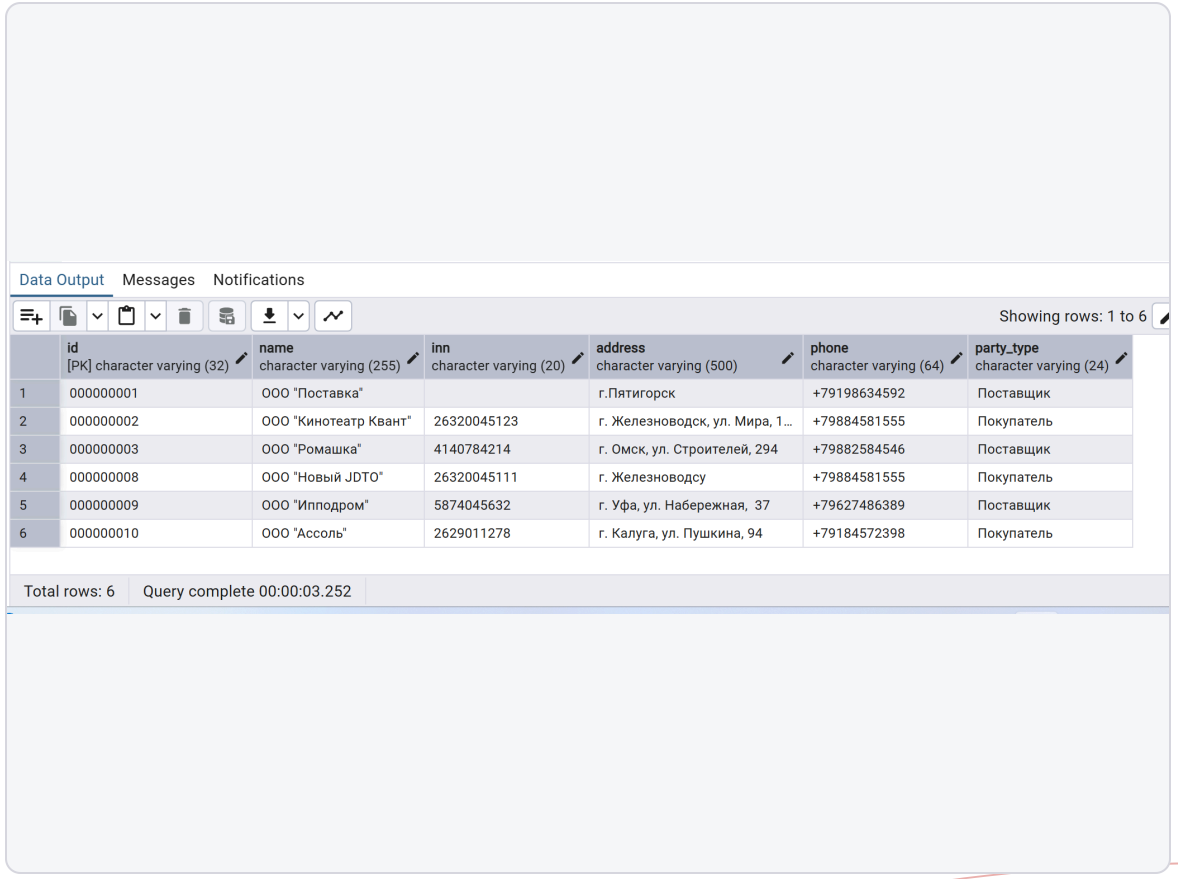
Сверьте импортированные значения

Проверьте, что в таблицу попали все записи и сохранились исходные значения.

1. Нажмите правой кнопкой на **counterparty** → **View/Edit Data** → **All Rows**. Сравните строки с «Заказчики.csv».
2. Проверьте первый **id** 000000001, пустой ИНН, кавычки в названиях, знак + в телефонах и значения **address** и **party_type**.

ЧТО ПРОВЕРИТЬ

В **counterparty** шесть строк. Значения совпадают с подготовленным **CSV**.



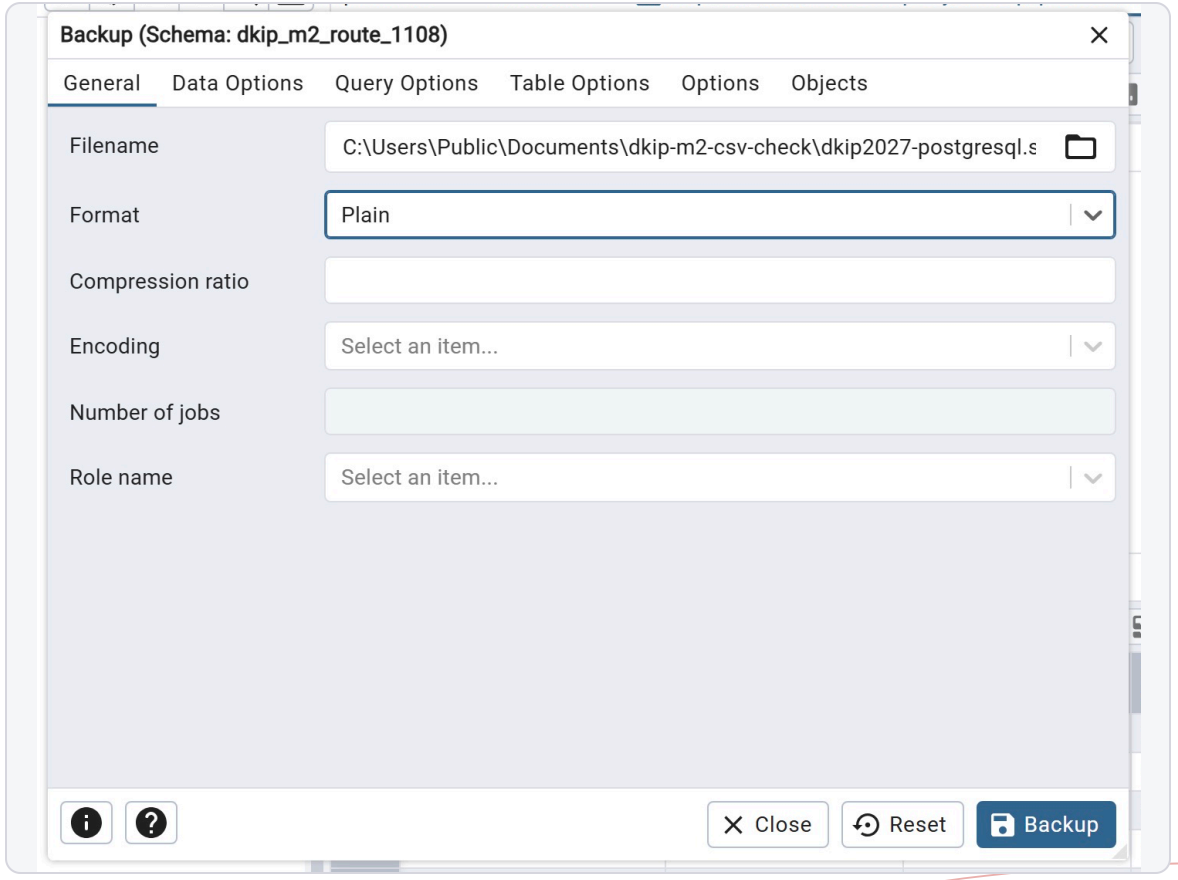
В **counterparty** загружены шесть строк. Видны ведущие нули, кавычки в названиях, пустой ИНН и знак + в телефонах.

ШАГ 9 ИЗ 10 · 1/2

Сохраните выгрузку

Выгрузка переносит структуру и загруженные данные в другую базу.

1. Нажмите правой кнопкой по своей базе → **Backup**. Если на учебном сервере вам выделили отдельную схему в общей базе, выберите **Backup** у своей схемы. В **Filename** укажите путь к **dkip2027-postgresql.sql** в папке «Результаты».
2. Выберите **Format** → **Plain**. В **Data Options** оставьте выключенными **Only data** и **Only schemas**, чтобы в файл вошли структура и данные. Нажмите **Backup** и дождитесь статуса **Finished** в **Processes**.



pgAdmin, окно **Backup** учебной схемы. Формат **Plain** создаёт SQL-файл со структурой и данными.

ШАГ 9 ИЗ 10 · 2/2

Сохраните выгрузку

1 Откройте сохранённый .sql в Блокноте. Найдите **CREATE TABLE** и блок данных **counterparty**. Расширение файла само по себе не определяет его формат.

ЧТО ПРОВЕРИТЬ

В папке «Результаты» есть **SQL**-файл со структурой и данными таблиц.

ШАГ 10 ИЗ 10 · 1/3

Проверьте восстановление

Восстановите структуру и данные

В восстановленной копии можно проверить данные и убедиться, что ограничения действительно работают.

1. Создайте пустую базу **dkip2027_restore**. Нажмите правой кнопкой по ней → **Restore**.
2. Выберите формат **Plain**, укажите свой **SQL**-файл и запустите **Restore**. В **Processes** дождитесь статуса **Finished**. Если **Restore** не предлагает **Plain**, для проверки сделайте отдельный архив **Backup** в формате **Custom** с расширением **.backup** и выберите **Custom** в **Restore**.
3. Обновите дерево восстановленной базы. Сверьте 12 таблиц и 14 внешних ключей в **Constraints**. Откройте **counterparty** → **View/Edit Data** → **All Rows** и сравните шесть строк с исходным файлом.

ШАГ 10 ИЗ 10 · 2/3

Проверьте восстановление

Проверьте составной ключ действием

1. Выберите восстановленную копию. Откройте **Tools → Query Tool**. Выполните первый блок один раз. Он добавит две тестовые спецификации и один общий ресурс.
2. Выполните второй блок. Увидите две строки с одним **item_id** и разными **specification_id**. Значение **qty = 0,012** сохранилось.
3. Выполните третий блок отдельно. База отклонит повтор той же пары ключей. Повторите выборку, чтобы увидеть те же две строки. Для задания 3 вернитесь к исходной учебной базе.

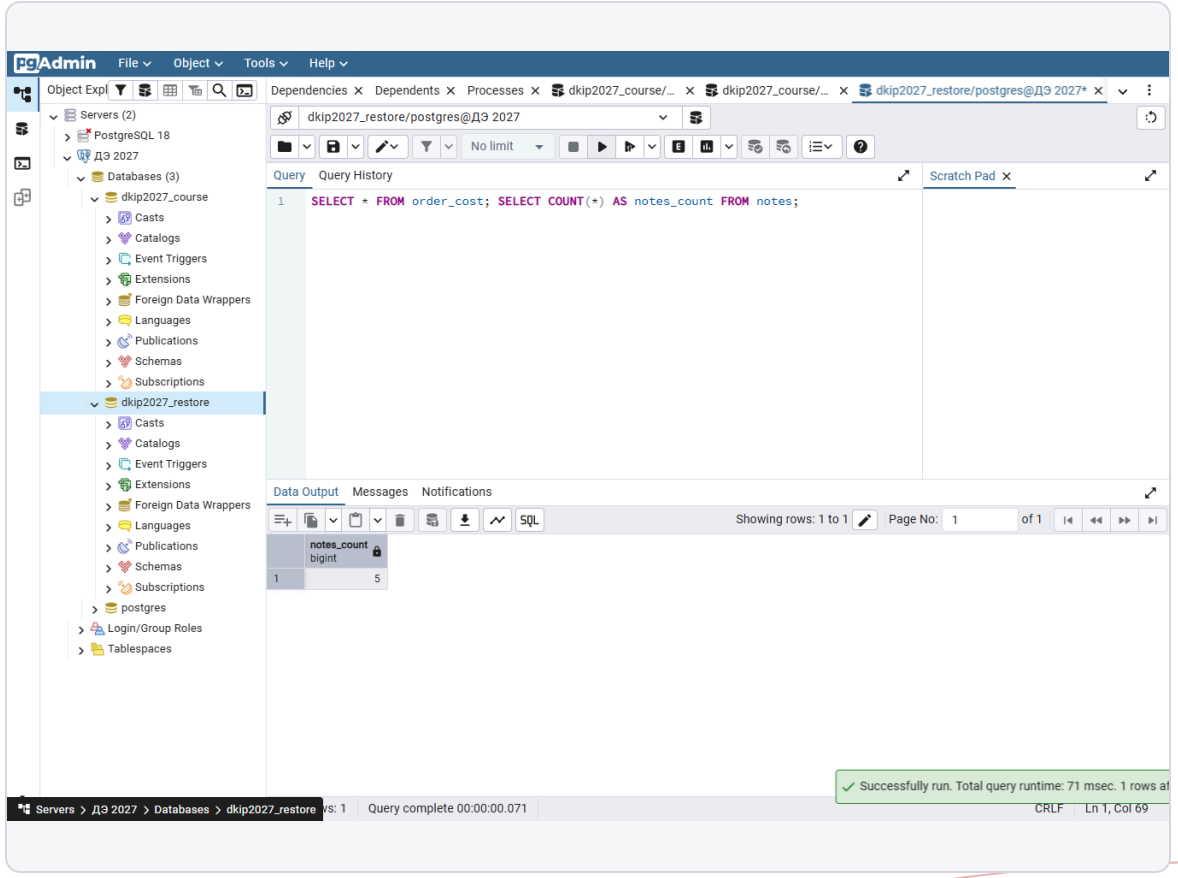
ШАГ 10 ИЗ 10 · 3/3

Проверьте восстановление

Покажите результат на экране

ЧТО ПРОВЕРИТЬ

Выгрузка восстановила структуру и шесть исходных заказчиков. Один ресурс вошёл в две спецификации. Дубликат пары отклонён.



Запрос к отдельной восстановленной базе **dkip2027_restore**. Маршрут: **WPF / PostgreSQL**.

ШАГ 10 · 1. ДВЕ СПЕЦИФИКАЦИИ И ОБЩИЙ РЕСУРС В ВОССТАНОВЛЕННОЙ КОПИИ · 1/2

Проверьте восстановление

```
INSERT INTO counterparty (id, name, inn, address, phone, party_type)
VALUES ('PK-TEST', 'Учебный изготовитель', '', '', '', 'Производитель');

INSERT INTO item (code, name, kind, unit) VALUES
  ('PK-PRODUCT-A', 'Учебное изделие А', 'product', 'шт.'),
  ('PK-PRODUCT-B', 'Учебное изделие Б', 'product', 'шт.'),
  ('PK-MATERIAL', 'Учебный евровинт', 'material', 'тыс. шт.');

INSERT INTO specification (product_id, name, output_qty, manufacturer_id)
SELECT id, name, 1, 'PK-TEST'
FROM item WHERE code IN ('PK-PRODUCT-A', 'PK-PRODUCT-B');

INSERT INTO specification_component (specification_id, item_id, qty, time_norm)
SELECT s.id, r.id, 0.012, 1
FROM specification s
JOIN item p ON p.id = s.product_id
```



ШАГ 10 · 1. ДВЕ СПЕЦИФИКАЦИИ И ОБЩИЙ РЕСУРС В ВОССТАНОВЛЕННОЙ КОПИИ · 2/2

Проверьте восстановление

```
JOIN item r ON r.code = 'PK-MATERIAL'
WHERE p.code IN ('PK-PRODUCT-A', 'PK-PRODUCT-B');
```

ШАГ 10 · 2. ВЫБОРКА ДВУХ СТРОК

Проверьте восстановление

```
SELECT c.specification_id, c.item_id, c.qty
FROM specification_component c JOIN item r ON r.id = c.item_id
WHERE r.code = 'PK-MATERIAL'
ORDER BY c.specification_id;
```



ШАГ 10 · 3. ПОПЫТКА ПОВТОРИТЬ ПЕРВУЮ ПАРУ

Проверьте восстановление

```
INSERT INTO specification_component (specification_id, item_id, qty, time_norm)
SELECT s.id, r.id, 0.024, 1
FROM specification s
JOIN item p ON p.id = s.product_id
JOIN item r ON r.code = 'PK-MATERIAL'
WHERE p.code = 'PK-PRODUCT-A';
```

РЕЗУЛЬТАТ ЭТАПА

Что должно остаться после этапа

В counterparty шесть строк. Первый id — 000000001, ведущие нули сохранены. Есть файл выгрузки dkip2027-postgresql.sql.

ЧТО ПРОВЕРИТЬ

Рабочая база с импортированными заказчиками и выгрузка.

ЗАДАНИЕ 3 · 20 МИН

Расчёт стоимости

Рассчитаем затраты на изготовление двух столов. Скидка и итог к оплате берутся из другого расчёта.

ЧТО ПРОВЕРИТЬ

03-cost.sql даёт **14 374,28**: материалы **9 974,28** и операции **4 400**.

УСЛОВИЕ ИЗ КИМ

Создание запроса

Создайте запрос, позволяющий вычислить полную стоимость заказа покупателя с учетом следующей информации:

- количество продукции в заказе;

- стоимость всех материалов и технологических операций, использованных для производства данной продукции (учитывая норму расхода).

Приложение: Прил_3_ОЗ_КИМ_09.02.07-5-2027.rar

КИМ · стр. 31 Исходные файлы · ZIP

ШАГ 1 ИЗ 9 · 1/7

Подготовьте строки для расчёта

Откройте свою базу из задания 2

Расчёт соединяет количество заказа, норму спецификации и цену ресурса. Сначала проверяйте одну строку вручную; затем стройте общий SQL.

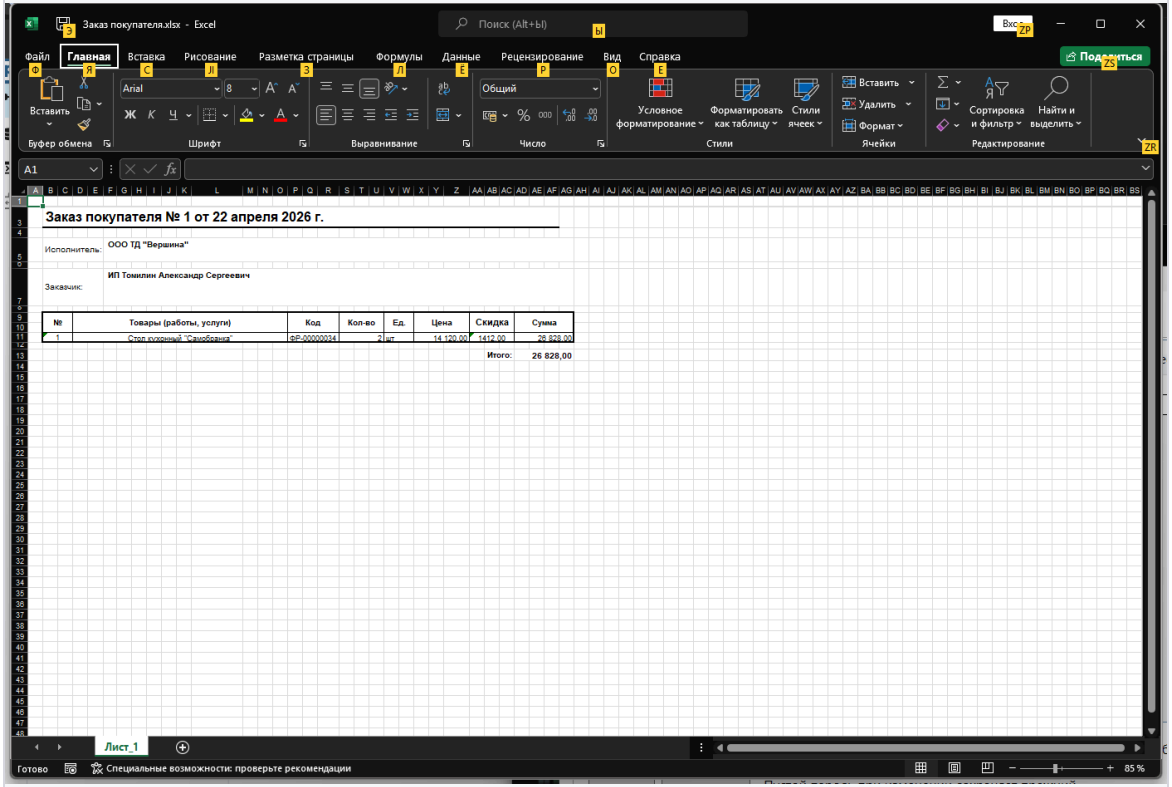
1. Откройте свою базу из задания 2. В **counterparty** уже есть шесть записей из **JSON**. Откройте четыре таблицы **Excel** по ссылкам шага.
2. В **pgAdmin** откройте **Schemas** → **public** → **Tables**. Правой кнопкой по таблице → **View/Edit Data** → **All Rows**. Нажмите **Add row**, заполните строку и нажмите **Save Data Changes**. Поля **Identity** оставляйте пустыми. После сохранения посмотрите присвоенный **id**.

ШАГ 1 ИЗ 9 · 2/7

Подготовьте строки для расчёта

Добавьте участников заказа

1. В «Заказ покупателя.xlsx» найдите исполнителя в F5 и покупателя в F7. Их нет среди шести записей из JSON. В counterparty добавьте покупателя ИП Томилин Александр Сергеевич и исполнителя ООО ТД «Вершина».
2. Откройте Query Tool и выполните блок «Контрагенты из заказа». Он добавит две строки с учебными кодами DOC-CUSTOMER и DOC-MAKER. ИНН, адрес и телефон в этих строках пустые, поскольку таких сведений в заказе нет.



Заказ покупателя из приложения открыт в Excel. Маршрут: WPF / PostgreSQL.

ШАГ 1 ИЗ 9 · 3/7

Подготовьте строки для расчёта

Заполните номенклатуру

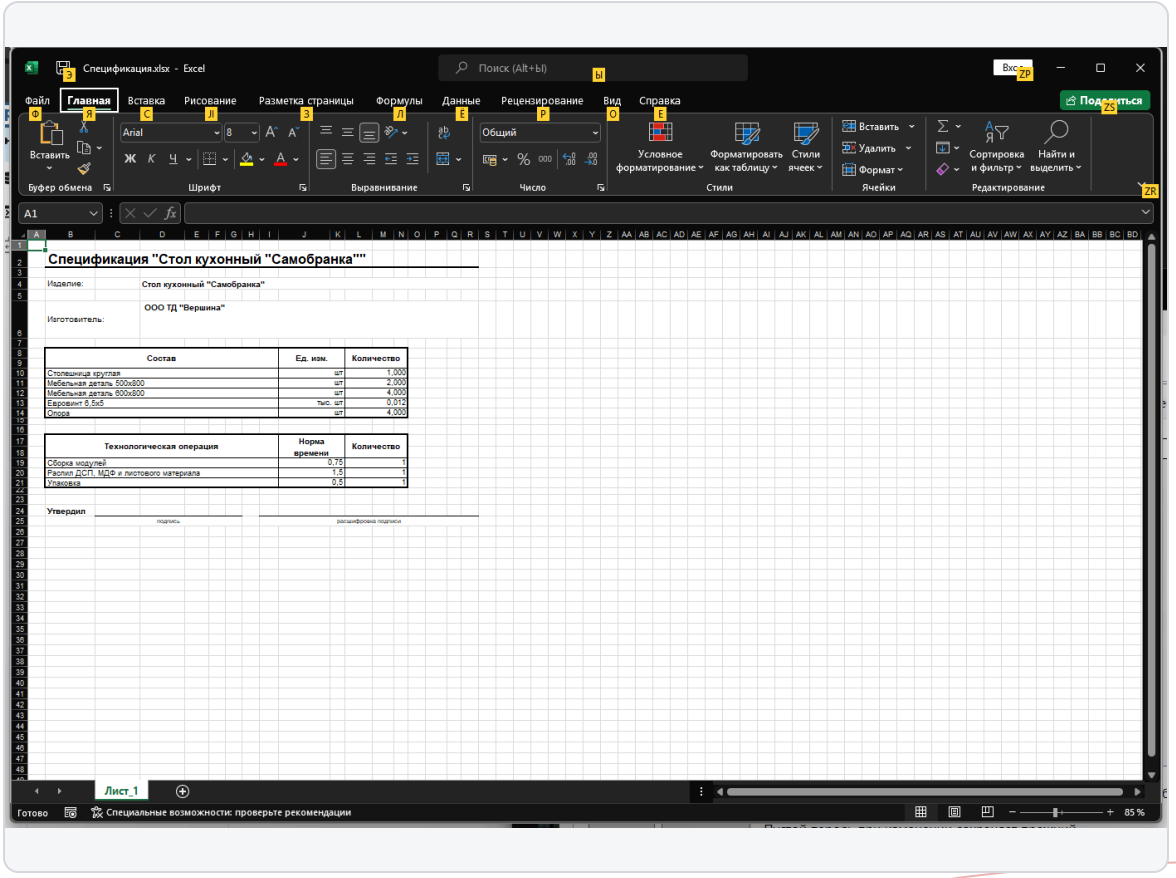
1. В item добавьте стол с кодом НФ-00000006 из S11 заказа на производство, названием из D11, kind = product и unit = шт. Запишите присвоенный id стола.
2. Добавьте пять материалов и три операции. Названия и коды возьмите из столбцов D и S заказа на производство, строки 17–21 и 27–29. Для материалов выберите kind = material, единицы возьмите из J10:J14 спецификации. Для операций укажите kind = operation и unit = ч.

ШАГ 1 ИЗ 9 · 4/7

Подготовьте строки для расчёта

Соберите спецификацию на один стол

1. В specification укажите product_id стола, название из B2 спецификации, output_qty = 1 и manufacturer_id = DOC-MAKER. Запишите присвоенный id спецификации.
2. В specification_component добавьте восемь строк. В каждой укажите specification_id и item_id соответствующего ресурса. Для qty возьмите L10:L14 и L19:L21. Для time_norm поставьте 1 у материалов и возьмите J19:J21 для операций.



Спецификация: материалы, количества и нормы технологических операций в Excel. Маршрут: WPF / PostgreSQL.

ШАГ 1 ИЗ 9 · 5/7

Подготовьте строки для расчёта

Соберите спецификацию на один стол

1 Например, у евровинта qty = 0.012 тыс. шт. и time_norm = 1. У сборки модулей qty = 1 и time_norm = 0.75 ч. Эти нормы относятся к одному столу.

ШАГ 1 ИЗ 9 · 6/7

Подготовьте строки для расчёта

Добавьте цены и заказ

1. В price добавьте по строке для каждого из восьми ресурсов. По названию в A2:A9 файла «Цены.xlsx» найдите item_id, цену возьмите из столбца D. Прайс без даты и единиц тарифа. Для всех строк примите valid_from = 2026-04-01, для операций — тариф за час.
2. В customer_order внесите номер 1 и дату 2026-04-22 из B3 заказа покупателя. Укажите customer_id = DOC-CUSTOMER и executor_id = DOC-MAKER. Запишите присвоенный id заказа.

ШАГ 1 ИЗ 9 · 7/7

Подготовьте строки для расчёта

Добавьте цены и заказ

1. В customer_order_line укажите order_id заказа и product_id стола. Из строки 11 возьмите source_code = ФР-00000034 (P11), qty = 2 (S11), sale_price = 14120 (X11) и discount = 1412 (AA11). По совпадению названия оба кода относятся к одному столу. В item достаточно одной записи о нём.

ЧТО ПРОВЕРИТЬ

В заказе № 1 одна строка на два стола. В спецификации этого стола восемь строк состава. Для каждого из восьми ресурсов внесена цена с датой **01.04.2026**.

Откройте: Заказ покупателя.**xlsx** · Заказ на производство.**xlsx** · Спецификация.**xlsx** · Цены.**xlsx**

ШАГ 1 · КОНТРАГЕНТЫ ИЗ ЗАКАЗА

Подготовьте строки для расчёта

```
INSERT INTO counterparty (id, name, inn, address, phone, party_type)
VALUES
  ('DOC-CUSTOMER', 'ИП Томилин Александр Сергеевич', '', '', '', 'Покупатель'),
  ('DOC-MAKER', 'ООО ТД «Вершина»', '', '', '', 'Производитель');
```

ШАГ 2 ИЗ 9

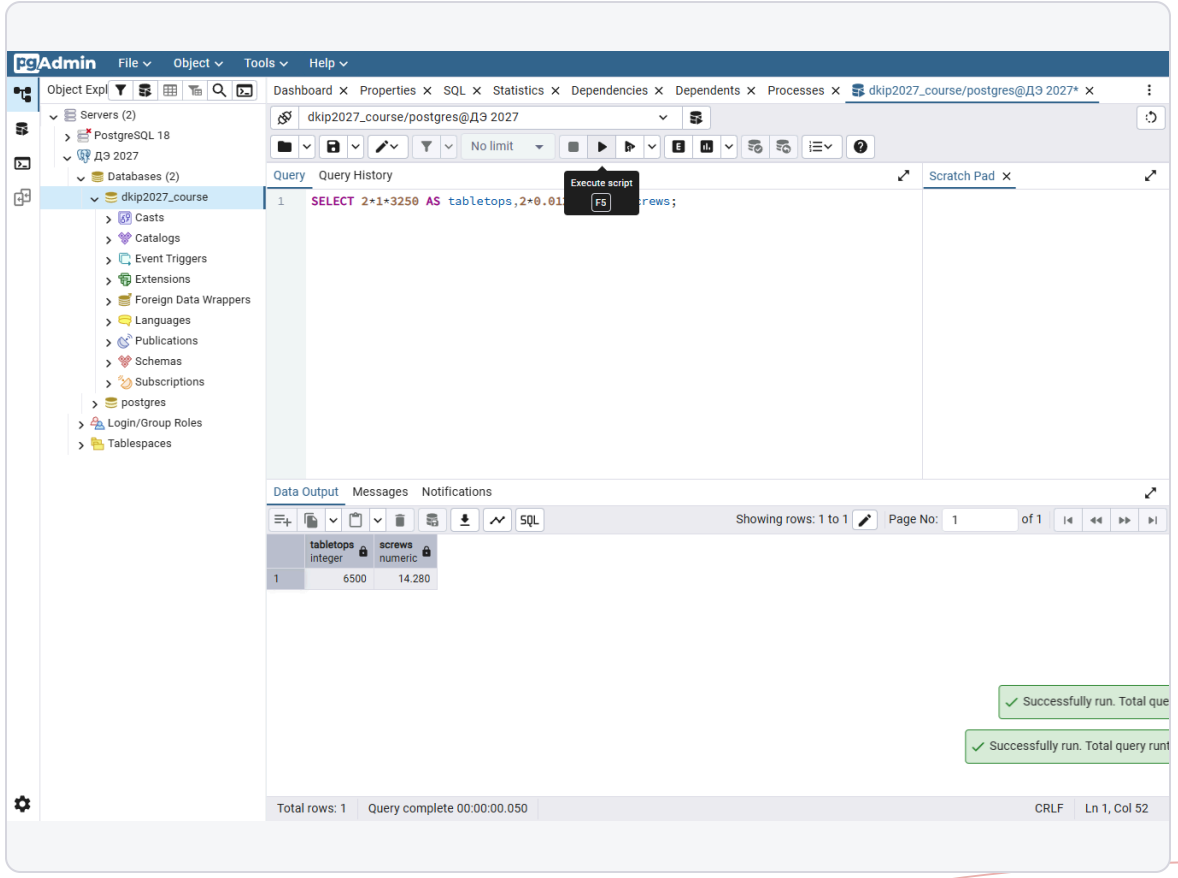
Посчитайте одну строку материала

Стоимость материала = количество изделий × норма × цена.

1. Найдите столешницу: 1 на стол, цена 3250.
2. Для 2 столов вычислите 2 × 1 × 3250.
3. Проверьте евровинт: 2 × 0,012 × 595.

ЧТО ПРОВЕРИТЬ

Столешницы: **6500**; евровинты: **14,28**.



Проверка двух строк материалов через SQL: 6500 и 14,28. Маршрут: WPF / PostgreSQL.

ШАГ 2 · КОД ДЛЯ ЭТОГО ШАГА

Посчитайте одну строку материала

```
SELECT 2*1*3250 AS tabletops, 2*0.012*595 AS screws;
```

ШАГ 3 ИЗ 9

Посчитайте технологические операции

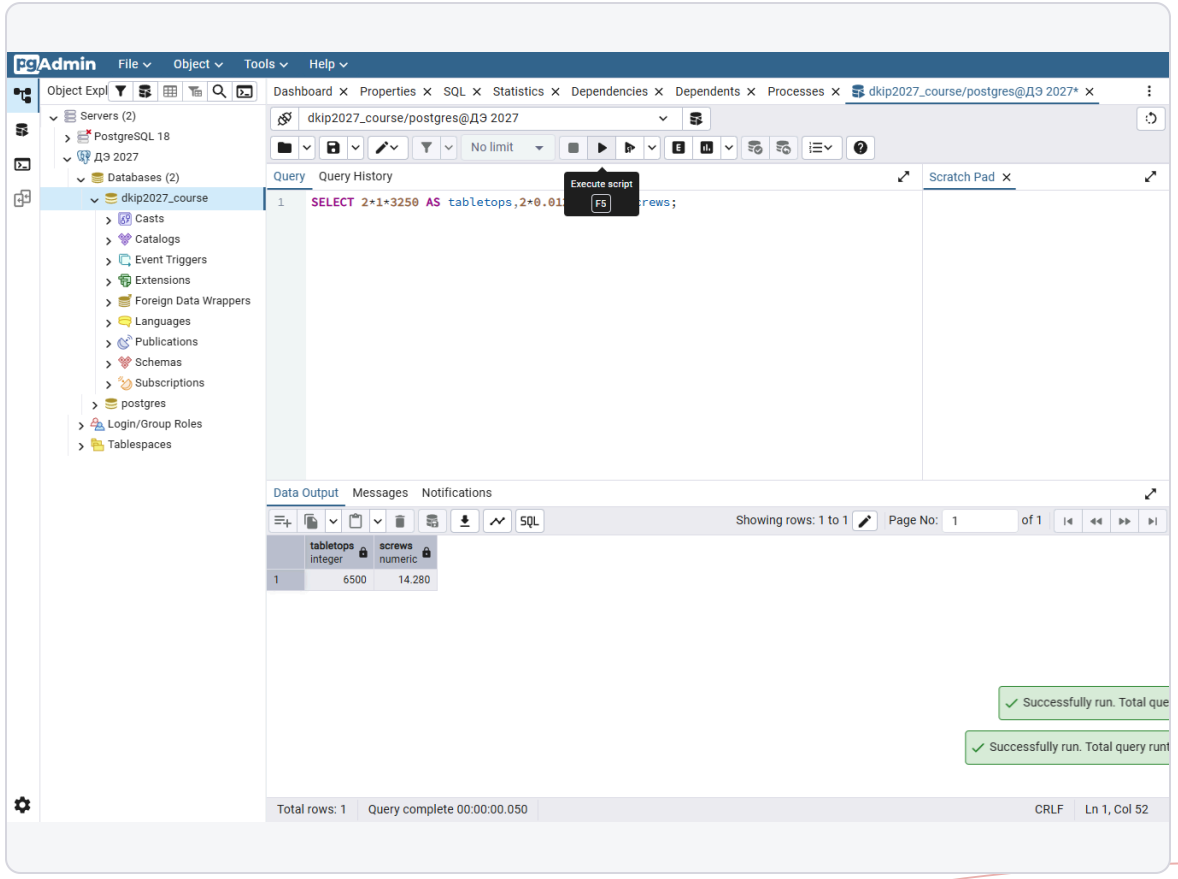
Для операции учитываются количество и норма времени. Тариф за час — принятое допущение.

1. Сборка: 2 × 1 × 0,75 × 1400.
2. Распил: 2 × 1 × 1,5 × 450.
3. Упаковка: 2 × 1 × 0,5 × 950.

ЧТО ПРОВЕРИТЬ

Операции стоят 2100 + 1350 + 950 = 4400.

Спецификация.xlsx, J19:L21; Цены.xlsx, D6:D9



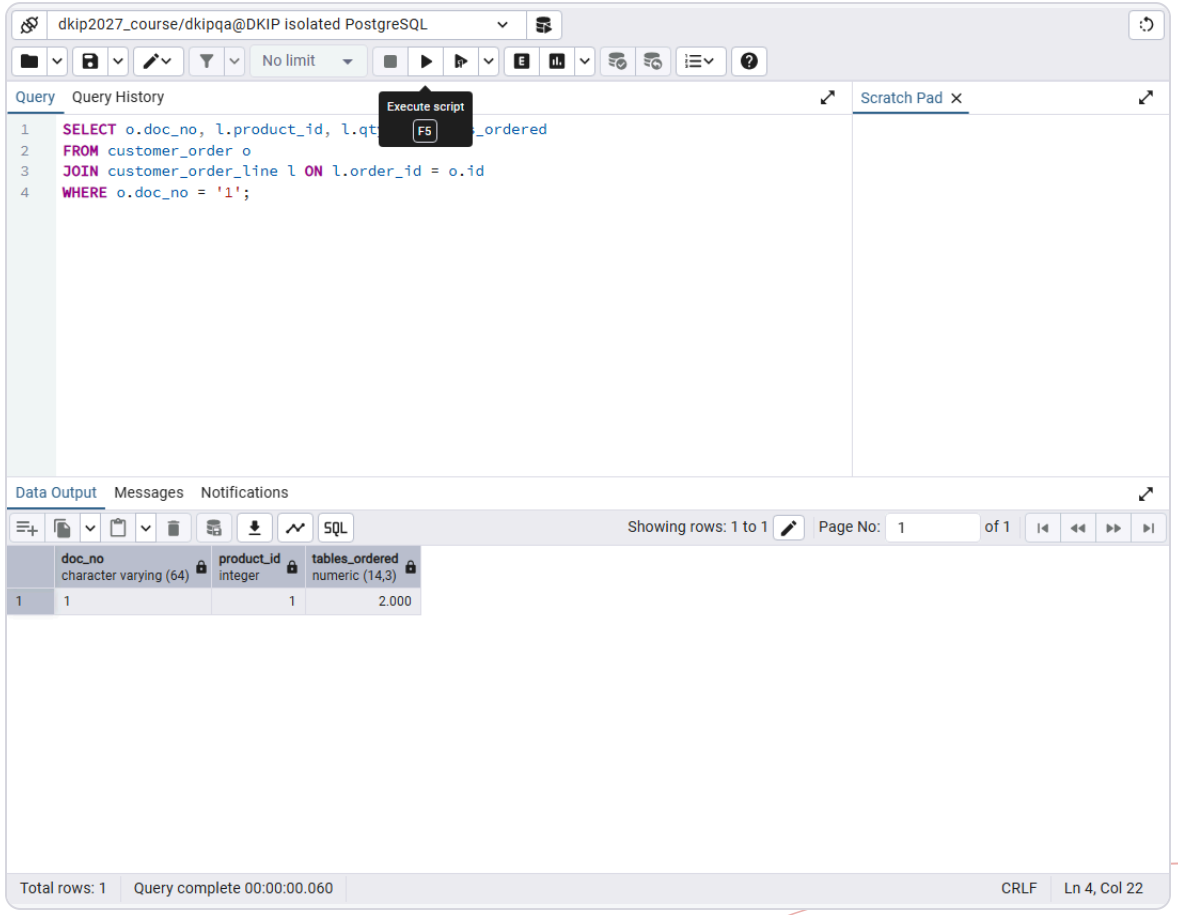
Проверка двух строк материалов через SQL: 6500 и 14,28. Маршрут: WPF / PostgreSQL.

ШАГ 4 ИЗ 9 · 1/2

Найдите заказ и количество столов

Сначала установите, сколько изделий заказали и какой ключ ведёт от позиции заказа к спецификации.

1. В pgAdmin выберите свою базу и откройте Tools → Query Tool. Начните с customer_order и задайте короткое имя o. Условие WHERE o.doc_no = '1' выбирает заказ по номеру из Excel.
2. Присоедините customer_order_line под именем l по l.order_id = o.id. В шапке хранятся номер и дата, в строке — изделие и количество. По ключу заказа строка попадёт именно к своей шапке.



pgAdmin показывает одну строку заказа № 1 с количеством 2.

ШАГ 4 ИЗ 9 · 2/2

Найдите заказ и количество столов

1 Выведите l.product_id и l.qty. Префикс l. указывает на строку заказа, а AS tables_ordered задаёт заголовок столбца. Выполните первый фрагмент перед добавлением следующих JOIN.

ЧТО ПРОВЕРИТЬ

Одна строка заказа № 1. В **tables_ordered** стоит **2**, а **product_id** совпадает с **id** стола в **item**. Если строк нет, сверьте номер заказа и **order_id** позиции.

ШАГ 4 · 1. ЗАКАЗ И СТРОКА ТОВАРА

Найдите заказ и количество столов

```
SELECT o.doc_no, l.product_id, l.qty AS tables_ordered
FROM customer_order o
JOIN customer_order_line l ON l.order_id = o.id
WHERE o.doc_no = '1';
```

ШАГ 5 ИЗ 9 · 1/3

Разверните заказ до строк спецификации

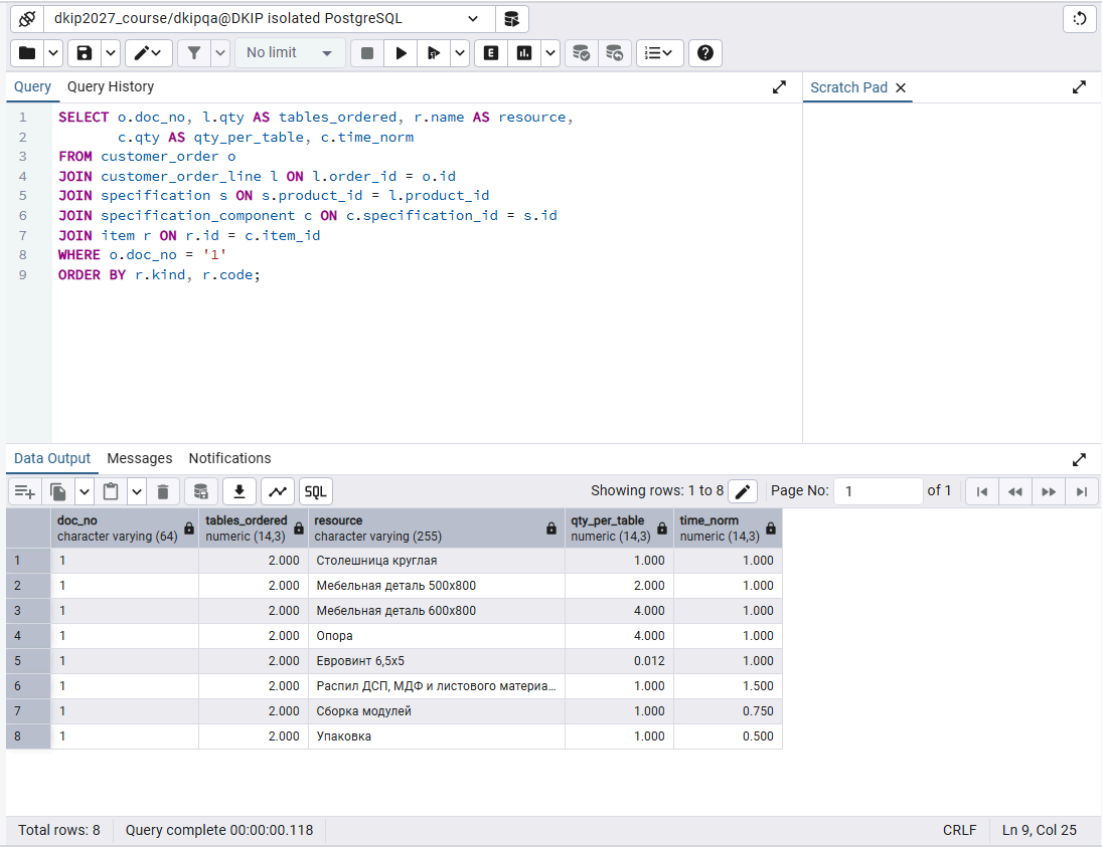
Количество столов берём из заказа, а расход материалов и время операций на один стол — из спецификации.

1. Продолжите запрос с предыдущей карточки. Перед **WHERE** добавьте **JOIN specification s ON s.product_id = l.product_id**. Так позиция заказа получит спецификацию именно своего изделия.
2. Добавьте **JOIN specification_component c ON c.specification_id = s.id**. Теперь одна позиция заказа даёт по строке на каждый ресурс. Материалы и операции уже хранятся в общем составе, поэтому хватает одного соединения с ним.

ШАГ 5 ИЗ 9 · 2/3

Разверните заказ до строк спецификации

Покажите на экране



В pgAdmin у заказа № 1 восемь строк ресурсов с нормами на один стол.

ШАГ 5 ИЗ 9 · 3/3

Разверните заказ до строк спецификации

1 Присоедините item r по r.id = c.item_id, чтобы показать название ресурса. В SELECT оставьте количество столов из l.qty и добавьте c.qty как qty_per_table и c.time_norm. Сортировка по виду и коду упорядочивает вывод.

ЧТО ПРОВЕРИТЬ

Восемь строк, в каждой **tables_ordered = 2**. У евровинта **qty_per_table = 0,012**, у сборки **time_norm = 0,75**. Если строк меньше, сверьте ключи изделия, спецификации и состава.

ШАГ 5 · 2. ЗАКАЗ И ВОСЕМЬ РЕСУРСОВ

Разверните заказ до строк спецификации

```
SELECT o.doc_no, l.qty AS tables_ordered, r.name AS resource,
       c.qty AS qty_per_table, c.time_norm
FROM customer_order o
JOIN customer_order_line l ON l.order_id = o.id
JOIN specification s ON s.product_id = l.product_id
JOIN specification_component c ON c.specification_id = s.id
JOIN item r ON r.id = c.item_id
WHERE o.doc_no = '1'
ORDER BY r.kind, r.code;
```

ШАГ 6 ИЗ 9 · 1/3

Добавьте цены и проверьте пропуски

LEFT JOIN сохраняет все строки состава и показывает NULL у ресурса, для которого на выбранную дату нет цены.

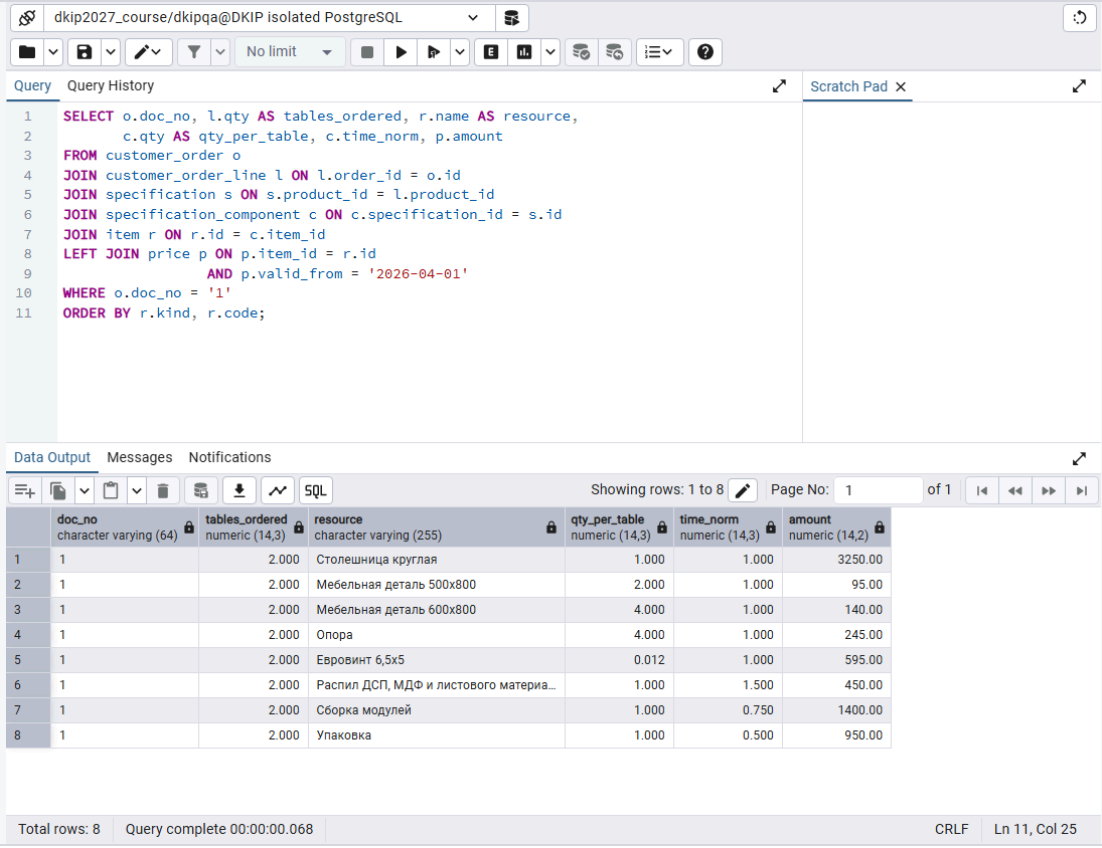
1. Добавьте p.amount в SELECT и LEFT JOIN price p перед WHERE. По p.item_id = r.id найдите цены того же ресурса. Условие по p.valid_from в ON оставляет конкретный прайс от 01.04.2026. Пара «ресурс и дата» даёт максимум одну цену.

2. Оставьте условие даты в ON. Обычный JOIN отбросит ресурс без цены, потому что ему не найдётся пары. Условие даты в WHERE тоже отбросит его, потому что сравнение NULL с датой не проходит фильтр.

ШАГ 6 ИЗ 9 · 2/3

Добавьте цены и проверьте пропуски

Покажите на экране



pgAdmin показывает восемь ресурсов и цены учебного прайса от 01.04.2026.

ШАГ 6 ИЗ 9 · 3/3

Добавьте цены и проверьте пропуски

1 Выполните запрос, затем в редакторе замените дату цены на 2026-04-02 и запустите ещё раз. Снова будет восемь строк, теперь с NULL в **amount**. Верните исходную дату. В следующей карточке дата цены будет определяться по дате заказа.

ЧТО ПРОВЕРИТЬ

Для прайса от **01.04.2026** видны восемь цен. У евровинта **amount** = **595**, у сборки = **1400**. При проверке на **02.04.2026** все восемь строк сохраняются.

ШАГ 6 · 3. ВОСЕМЬ РЕСУРСОВ С ЦЕНАМИ

Добавьте цены и проверьте пропуски

```
SELECT o.doc_no, l.qty AS tables_ordered, r.name AS resource,
       c.qty AS qty_per_table, c.time_norm, p.amount
FROM customer_order o
JOIN customer_order_line l ON l.order_id = o.id
JOIN specification s ON s.product_id = l.product_id
JOIN specification_component c ON c.specification_id = s.id
JOIN item r ON r.id = c.item_id
LEFT JOIN price p ON p.item_id = r.id
                 AND p.valid_from = '2026-04-01'
WHERE o.doc_no = '1'
ORDER BY r.kind, r.code;
```

ШАГ 7 ИЗ 9 · 1/2

Выберите цену на дату заказа

Дата 1 апреля в проверочном запросе относится к конкретному учебному прайсу. Для расчёта берём последнюю действующую цену каждого ресурса на дату заказа.

1. В запросе с предыдущей карточки замените условие с фиксированной датой на подзапрос ниже. Добавьте **o.doc_date** и **p.valid_from** в **SELECT**, чтобы сверить дату заказа и дату найденной цены.
2. В подзапросе **p2.item_id = r.id** ограничивает выбор ценами текущего ресурса. Условие **p2.valid_from <= o.doc_date** исключает будущие цены. **MAX** возвращает самую позднюю из оставшихся дат.
3. Сохраните подзапрос в **ON**. Если допустимой даты нет, **LEFT JOIN** всё равно покажет ресурс, а поля цены будут **NULL**.

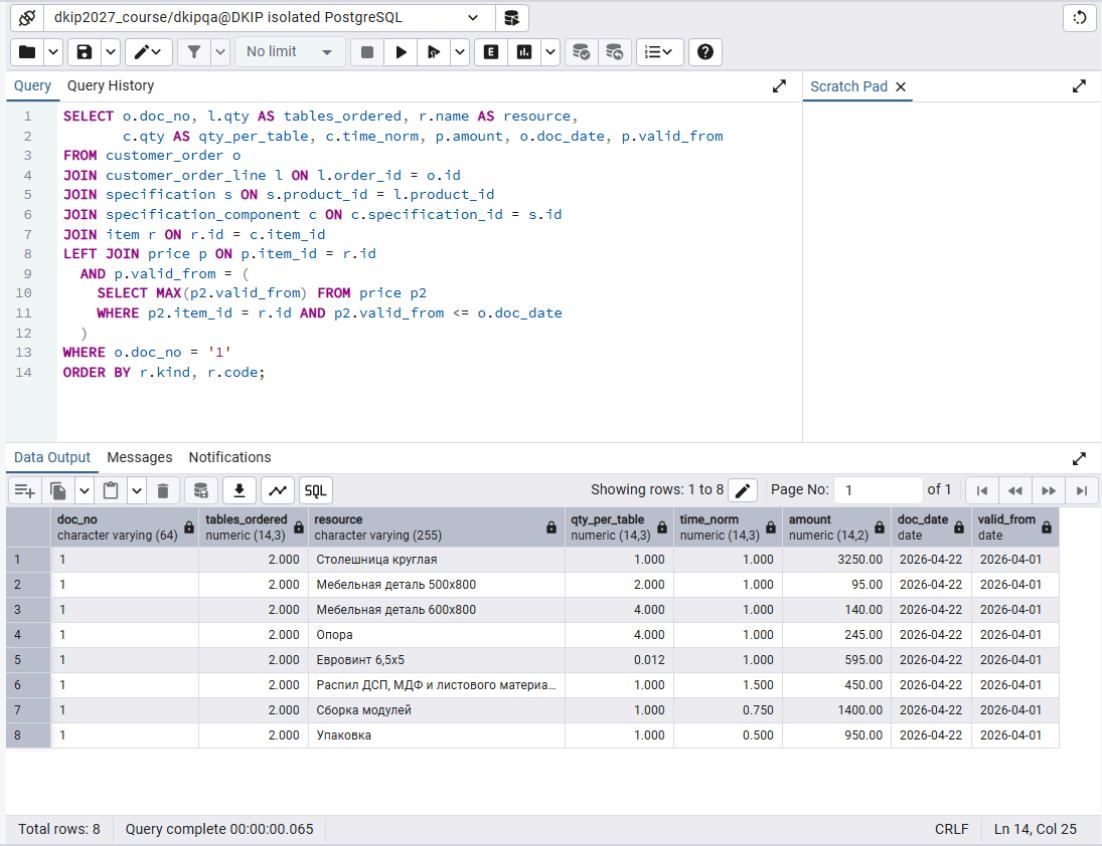
ЧТО ПРОВЕРИТЬ

Для заказа от **22.04.2026** запрос показывает восемь цен с датой **01.04.2026**.

ШАГ 7 ИЗ 9 · 2/2

Выберите цену на дату заказа

Покажите на экране



В pgAdmin видно, какие даты цен выбраны для заказа от 22.04.2026.

ШАГ 7 · УСЛОВИЕ ДЛЯ ЦЕНЫ НА ДАТУ ЗАКАЗА

Выберите цену на дату заказа

```
LEFT JOIN price p ON p.item_id = r.id
  AND p.valid_from = (
    SELECT MAX(p2.valid_from) FROM price p2
    WHERE p2.item_id = r.id AND p2.valid_from <= o.doc_date
  )
```

ШАГ 8 ИЗ 9 · 1/2

Выполните итоговый запрос

Количество заказа делим на выпуск по спецификации. Затем умножаем на норму ресурса, норму времени и цену. SUM складывает вклады ресурсов, ROUND округляет итог.

1. В pgAdmin выполните первый блок. Он создаёт представление order_cost на ваших таблицах. В нём GROUP BY o.id собирает один итог на заказ, а CASE проверяет полноту данных.
2. Выполните второй блок. Он находит ваш заказ по номеру 1 из Excel и показывает присвоенный order_id и рассчитанную стоимость.
3. Сверьте с ручной суммой. Материалы стоят 9 974,28, операции — 4 400.

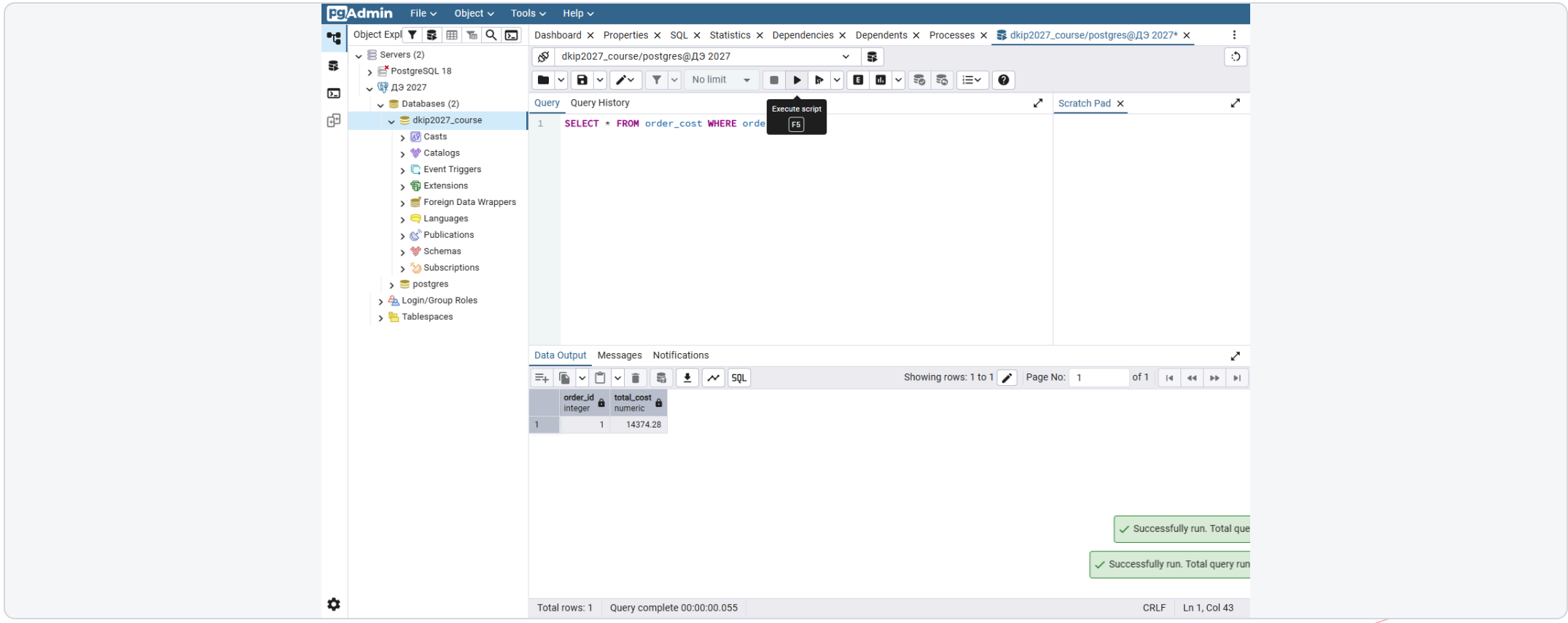
ЧТО ПРОВЕРИТЬ

Нормативная стоимость двух столов — 14 374,28.

ШАГ 8 ИЗ 9 · 2/2

Выполните итоговый запрос

Покажите на экране



Результат расчёта в СУБД: **14 374,28**. Маршрут: **WPF / PostgreSQL**.

ШАГ 8 · ПРЕДСТАВЛЕНИЕ ORDER_COST

Выполните итоговый запрос

```
CREATE OR REPLACE VIEW order_cost AS
SELECT o.id AS order_id,
 CASE WHEN COUNT(p.item_id) = COUNT(*)
      THEN ROUND(SUM(l.qty / s.output_qty * c.qty * c.time_norm * p.amount),2)
 END AS total_cost
FROM customer_order o
JOIN customer_order_line l ON l.order_id=o.id
LEFT JOIN specification s ON s.product_id=l.product_id
LEFT JOIN specification_component c ON c.specification_id=s.id
LEFT JOIN price p ON p.item_id=c.item_id
 AND p.valid_from=(SELECT MAX(p2.valid_from) FROM price p2
                   WHERE p2.item_id=c.item_id AND p2.valid_from<=o.doc_date)
GROUP BY o.id;
-- NULL означает, что для заказа не хватает норм или действующих цен.
```

ШАГ 8 · ИТОГ ПО ЗАКАЗУ № 1

Выполните итоговый запрос

```
SELECT v.order_id, v.total_cost
FROM order_cost v
JOIN customer_order o ON o.id = v.order_id
WHERE o.doc_no = '1';
```

ШАГ 9 ИЗ 9 · 1/2

Обработайте неполные данные

SUM пропускает значения NULL и может вернуть частичную сумму. Поэтому представление order_cost проверяет, для всех ли строк есть действующие цены, прежде чем вернуть итог.

1. LEFT JOIN для спецификации, её состава и действующих цен сохраняет строки с пропусками. COUNT(*) считает все строки, а COUNT(p.item_id) только строки с найденной ценой. При несовпадении счётчиков CASE в представлении оставляет total_cost равным NULL.

2. Выполните запрос ниже. Вместо 1 подставьте order_id, который получили на предыдущей карточке. Здесь CASE читает total_cost и выводит понятный статус в столбце cost_status.

ШАГ 9 ИЗ 9 · 2/2

Обработайте неполные данные

1 В копии базы запишите исходную строку цены евровинта от 01.04.2026 и временно удалите её. Выполните тот же запрос, затем восстановите строку и выполните его ещё раз.

ЧТО ПРОВЕРИТЬ

Без цены **total_cost** равен **NULL**, а статус сообщает о недостающих данных. После восстановления цены итог снова **14 374,28**, статус — «Расчёт выполнен».



ШАГ 9 · ПРОВЕРОЧНЫЙ ЗАПРОС ПО ЗАКАЗУ

Обработайте неполные данные

```
SELECT order_id, total_cost,
       CASE WHEN total_cost IS NULL THEN 'Недостаточно норм или цен на дату заказа'
            ELSE 'Расчёт выполнен'
       END AS cost_status
FROM order_cost WHERE order_id=1;
-- Ожидается 14374.28 при допущениях docs/DECISIONS.md.
```

РЕЗУЛЬТАТ ЭТАПА

Что должно остаться после этапа

03-cost.sql даёт 14 374,28: материалы 9 974,28 и операции 4 400.

ЧТО ПРОВЕРИТЬ

Запрос SQL, результат и записанные допущения.

ЗАДАНИЕ 4 · 60 МИН

Формы и WPF

Сначала собираем форму в конструкторе, затем связываем кнопки с кодом. После входа пользователь и администратор увидят разные возможности.

ЧТО ПРОВЕРИТЬ

Есть проект **MyApp** с исходниками и снимки входа, администрирования и проверки блокировки.

УСЛОВИЕ ИЗ КИМ

Разработка информационной системы

Разработайте модули информационной системы на основании технического задания (Приложение 1)

Приложение: Прил_4_О3_КИМ_09.02.07-5-2027.rar

КИМ · стр. 31; Приложение 1 Исходные файлы · ZIP

ШАГ 1 ИЗ 30

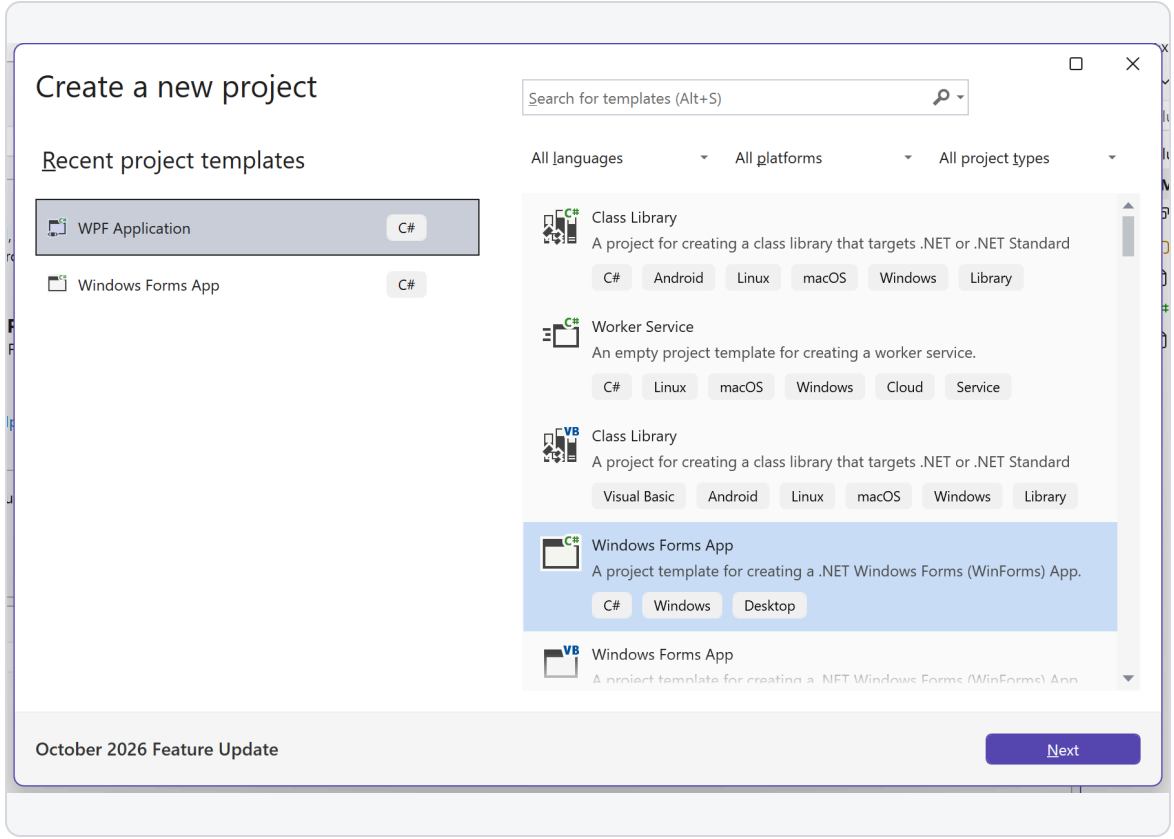
Откройте создание проекта MyApp

Проект объединяет исходные файлы и настройки сборки. Сначала откройте мастер создания нового проекта в Visual Studio.

1. Если видно стартовое окно Visual Studio, нажмите **Create a new project**. Если проект уже открыт, выберите «Файл» → «Создать» → «Проект» (Ctrl+Shift+N). Откроется окно поиска шаблона. Вводите название шаблона в поле поиска, а не в окно кода.

ЧТО ПРОВЕРИТЬ

Открылось окно **Create a new project** с полем поиска шаблонов.



Мастер **Create a new project** в **Visual Studio**: поле **Search for templates** и кнопка **Next**. Введите шаблон из текущего шага; список **Recent** зависит от ранее открытых проектов.

ШАГ 2 ИЗ 30

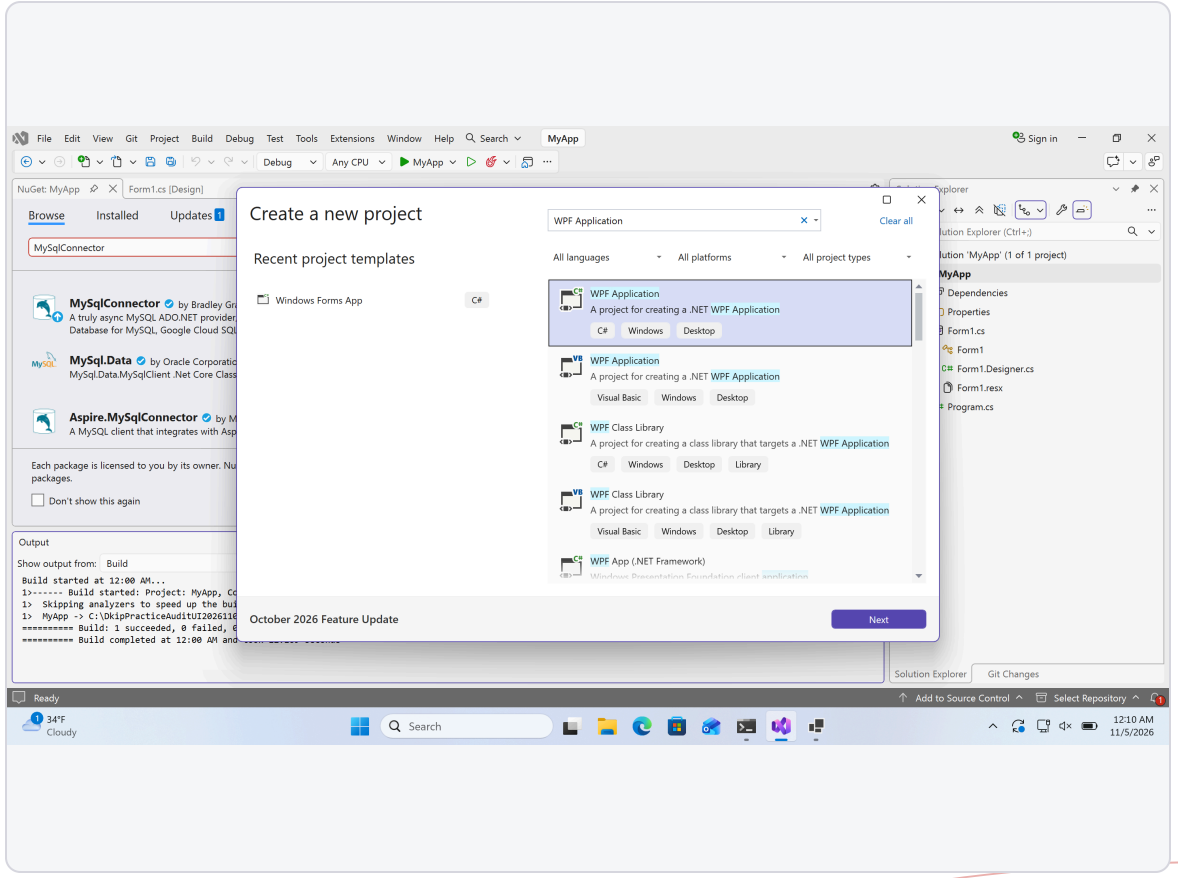
Выберите шаблон MyApp

Шаблон определяет тип приложения и начальные файлы. Названия современного .NET и .NET Framework похожи, но это разные платформы.

1. Найдите «Приложение WPF (WPF Application)». Проверьте язык C#. Для Windows Forms/WPF выбирайте современный .NET, без надписи .NET Framework. Нажмите «Далее». Если шаблона нет, остановитесь и проверьте компоненты Visual Studio с преподавателем.

ЧТО ПРОВЕРИТЬ

Выбран нужный шаблон C# современного .NET; открыта настройка имени проекта.



Выбор современного шаблона **WPF Application** на **C#** в **Visual Studio**. Не выбирайте вариант **.NET Framework**.

ШАГ 3 ИЗ 30

Задайте имя и папку MyApp

Имя отличает проект в Обозревателе решений, расположение определяет папку исходников на диске, платформа — способ сборки.

1. В поле «Имя проекта» введите MyApp. В «Расположение» нажмите кнопку обзора и выберите свою рабочую папку на рабочем столе Windows. Нажмите «Далее». Выберите .NET 10 → «Создать». Дождитесь Обозревателя решений с проектом MyApp.

ЧТО ПРОВЕРИТЬ

В Обозревателе решений появился проект с заданным именем и выбранной платформой .NET 10.

ШАГ 4 ИЗ 30

Проверьте пустое окно

Сначала сделайте кнопку, которая отвечает на нажатие. Потом проверьте доступ к базе, добавьте вход, пазл и рабочее окно.

1 Нажмите «Отладка» → «Запуск без отладки» (Ctrl+F5). Дождитесь окна и закройте его крестиком.

ЧТО ПРОВЕРИТЬ

Своё пустое окно открылось без ошибок и закрылось крестиком.

ШАГ 5 ИЗ 30 · 1/2

Создайте окно «Вход»

Добавьте окно

Окно с именем LoginForm или LoginWindow легко найти в проекте. Назначьте его первым окном при запуске.

1. В Обозревателе решений нажмите правой кнопкой на MyApp → «Добавить» → «Окно WPF». Введите LoginWindow.xaml и нажмите «Добавить».
2. Дважды нажмите LoginWindow.xaml, выберите «Конструктор» и кликните по краю окна. Нажмите F4 и в Title введите «Вход», в Width — 540, в Height — 550.

ШАГ 5 ИЗ 30 · 2/2

Создайте окно «Вход»

Назначьте первое окно

1. Откройте App.xaml. Найдите StartupUri="MainWindow.xaml" и замените свойство значением из блока ниже.
2. Нажмите Ctrl+S, затем Ctrl+F5. Убедитесь, что открылось окно с заголовком «Вход». Закройте его крестиком.

ЧТО ПРОВЕРИТЬ

При запуске открывается пустое окно «Вход». В проекте появился файл этого окна.

ШАГ 5 · СВОЙСТВО ОКНА В APP.XAML

Создайте окно «Вход»

```
StartupUri="LoginWindow.xaml"
```

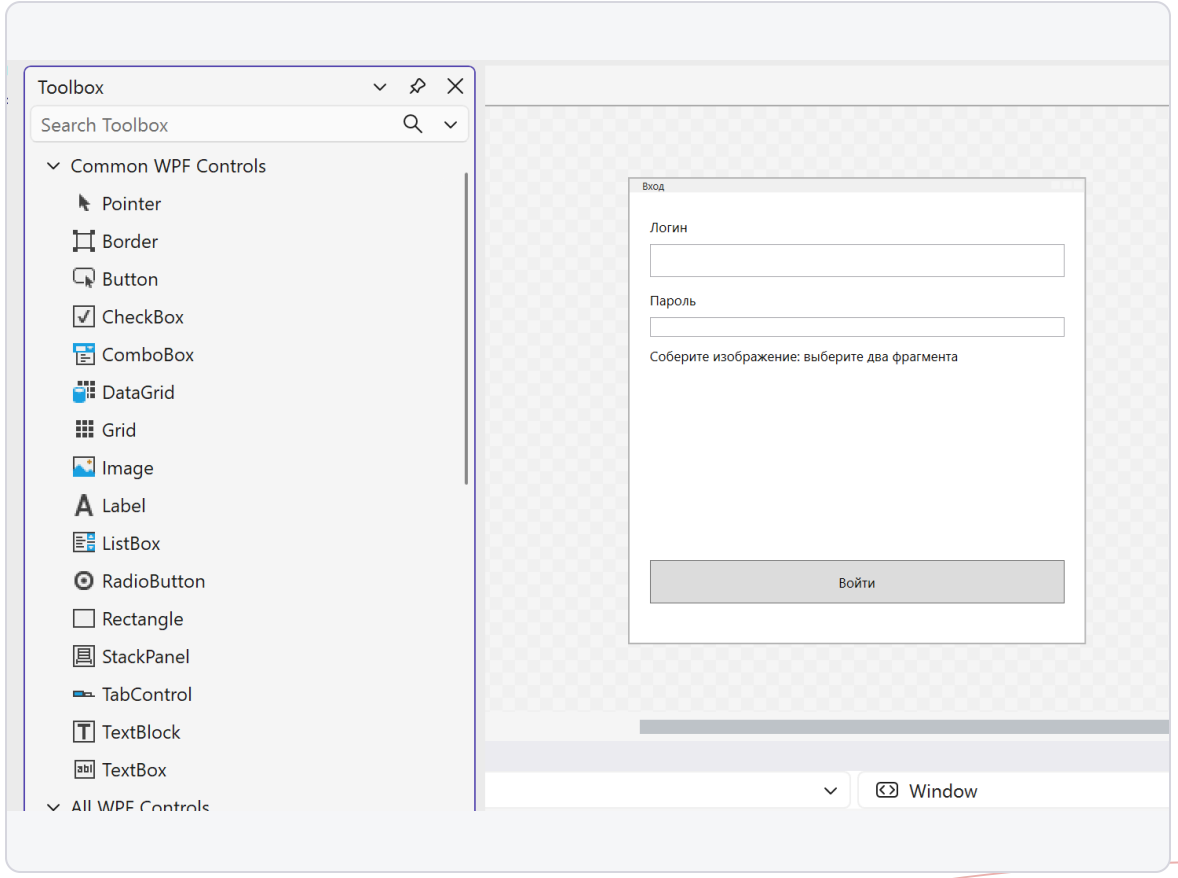
ШАГ 6 ИЗ 30 · 1/3

Добавьте поля логина и пароля

Откройте панель элементов

Элементы получают имена, по которым обработчик позже прочитает логин и пароль.

1. Откройте окно «Вход» в конструкторе. Нажмите Ctrl+Alt+X или выберите «Вид» → «Панель элементов» (Toolbox).
2. В Toolbox откройте Common WPF Controls и найдите StackPanel. Перетащите его на макет. TextBlock и TextBox находятся там же, а PasswordBox можно найти через Search Toolbox. Размещайте элементы внутри StackPanel сверху вниз.



Visual Studio, WPF. В Toolbox слева видны StackPanel, TextBlock, TextBox и Button. Справа — собранный макет LoginWindow.

ШАГ 6 ИЗ 30 · 2/3

Добавьте поля логина и пароля

Добавьте логин

1. Перетащите **TextBlock** в **StackPanel**. В свойствах **Text** задайте «Логин». Ниже добавьте **TextBox**.
2. Выделите **TextBox** → **F4** → в поле **Name** введите **LoginInput** и нажмите **Enter**.

ШАГ 6 ИЗ 30 · 3/3

Добавьте поля логина и пароля

Добавьте пароль

1. Добавьте второй **TextBlock** с текстом «Пароль». Под ним разместите **PasswordBox**. В поле **Name** введите **PasswordInput**.
2. Нажмите **Ctrl+F5**. Впишите несколько символов в оба поля. В логине символы видны, в пароле отображаются точки. Закройте окно.

ЧТО ПРОВЕРИТЬ

В окне «Вход» есть две подписи и два поля. Логин виден, пароль скрывается.

ШАГ 7 ИЗ 30 · 1/3

Добавьте кнопку и свяжите её с кодом

Поставьте кнопку под полями

Нажатие вызывает метод Click. Пока проверяем только этот переход от формы к коду.

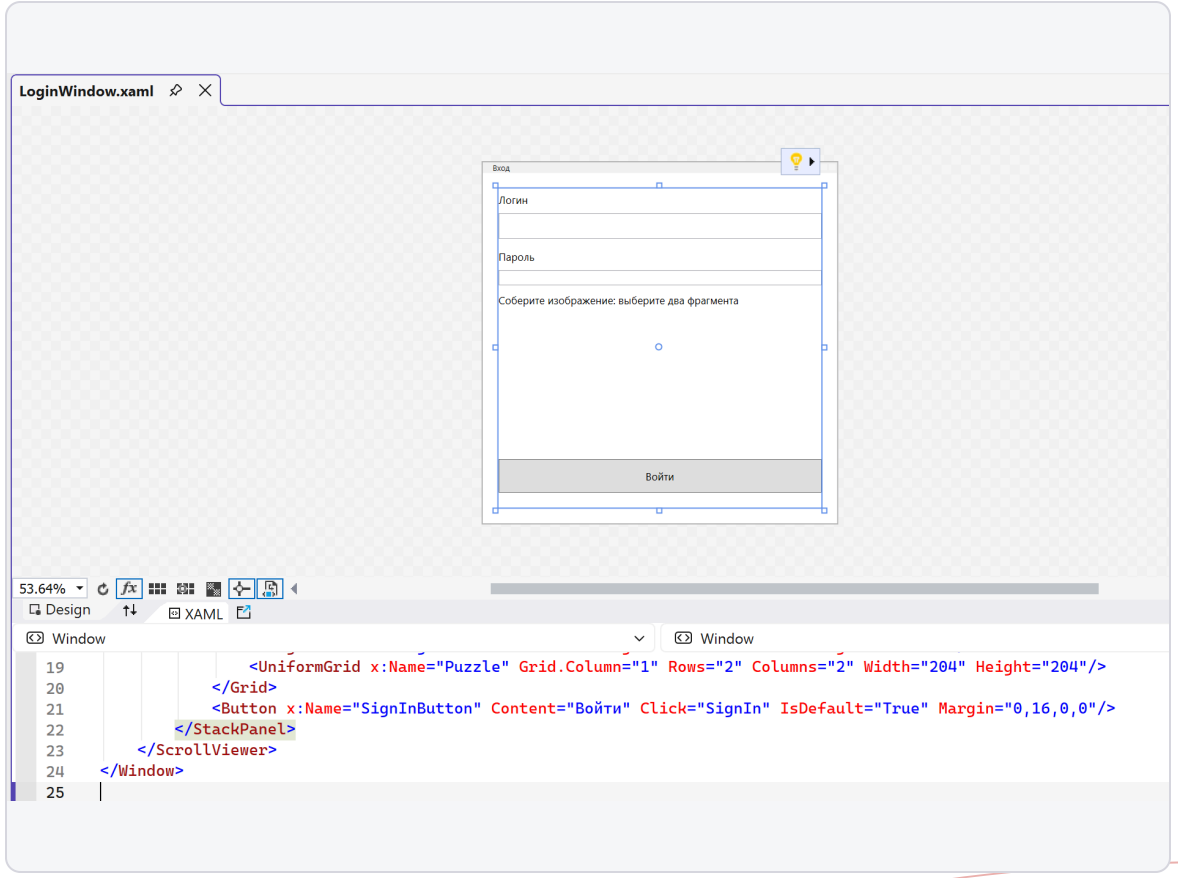
1. В Toolbox найдите Button и перетащите под полем пароля. Выделите кнопку и нажмите F4.
2. В Name введите SignInButton, в Content — «Войти». Нажмите Enter.

ШАГ 7 ИЗ 30 · 2/3

Добавьте кнопку и свяжите её с кодом

Вставьте одну строку в обработчик

1. Дважды щёлкните кнопку на макете. Visual Studio откроет файл C# и создаст метод для события Click. Обычно он называется SignInButton_Click.
2. Поставьте курсор на пустую строку между фигурными скобками метода. Вставьте строку из блока ниже. Скобки и заголовок метода оставьте.
3. Сохраните Ctrl+S, запустите Ctrl+F5 и нажмите «Войти». Если ничего не произошло, вернитесь в конструктор: кнопка → F4 → значок молнии → Click. Там должно стоять имя созданного метода.



Visual Studio, WPF. В нижней панели XAML у кнопки SignInButton видно Click="SignIn". В учебном проекте здесь должно стоять имя вашего обработчика.

ШАГ 7 ИЗ 30 · 3/3

Добавьте кнопку и свяжите её с кодом

Вставьте одну строку в обработчик

1 В WPF та же связь видна в разметке: в LoginWindow.xaml найдите Button с именем SignInButton и сверьте значение Click с именем метода в LoginWindow.xaml.cs.

ЧТО ПРОВЕРИТЬ

Нажатие на «Войти» показывает «Кнопка работает». Дальше код входа добавляется в этот же обработчик.



ШАГ 7 · ОДНА СТРОКА ВНУТРИ CLICK

Добавьте кнопку и свяжите её с кодом

```
MessageBox.Show("Кнопка работает");
```

ШАГ 8 ИЗ 30

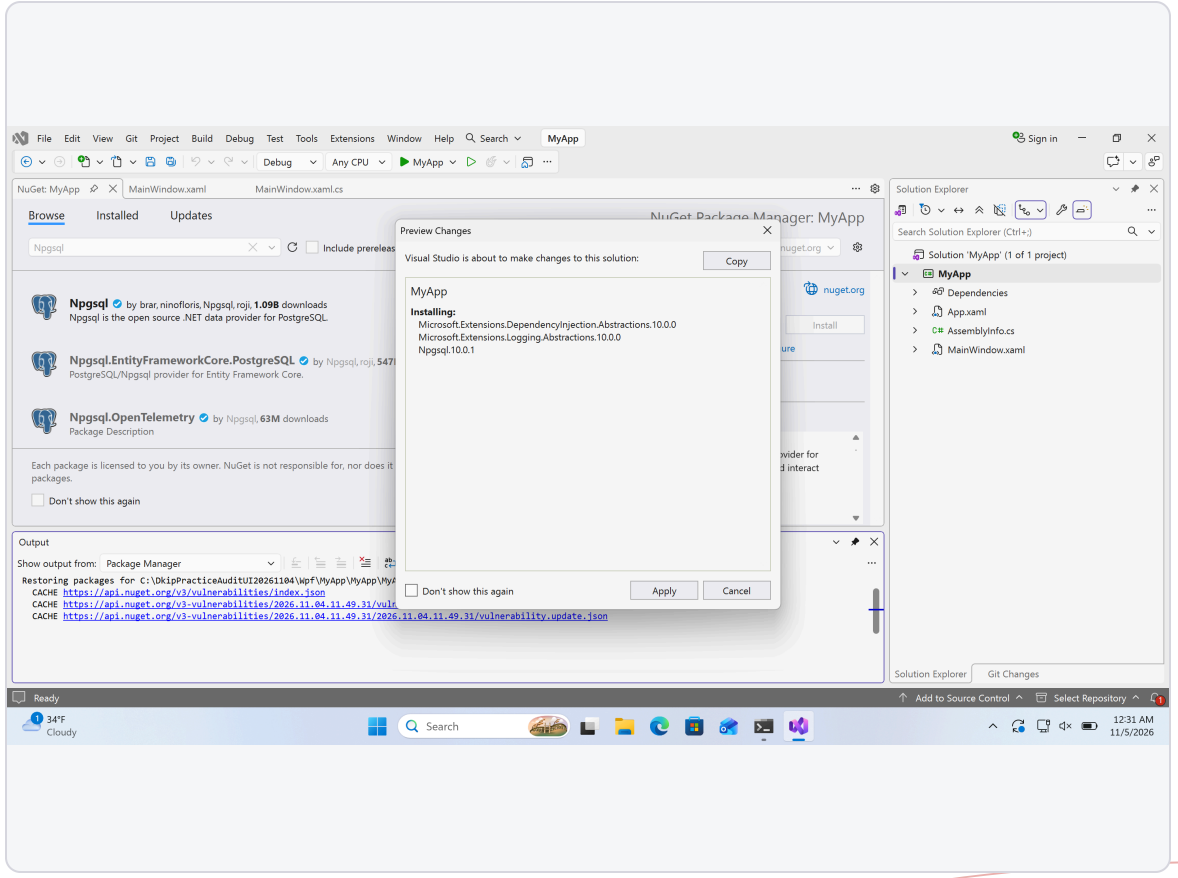
Откройте пакеты MyApp

Провайдер — библиотека, через которую C# подключается к выбранной СУБД. NuGet устанавливает её зависимость в конкретный проект.

1. В Обозревателе решений нажмите правой кнопкой именно по проекту MyApp → «Управление пакетами NuGet». Перейдите на вкладку «Обзор» (Browse). Введите Npgsql в поиск и нажмите Enter. Дождитесь списка результатов. Выберите пакет с точным названием Npgsql.

ЧТО ПРОВЕРИТЬ

Открыт NuGet → Browse; найден пакет провайдера с точным названием.



Окно Preview Changes для Npgsql 10.0.1. Apply подтверждает установку показанного пакета и зависимостей.

ШАГ 9 ИЗ 30

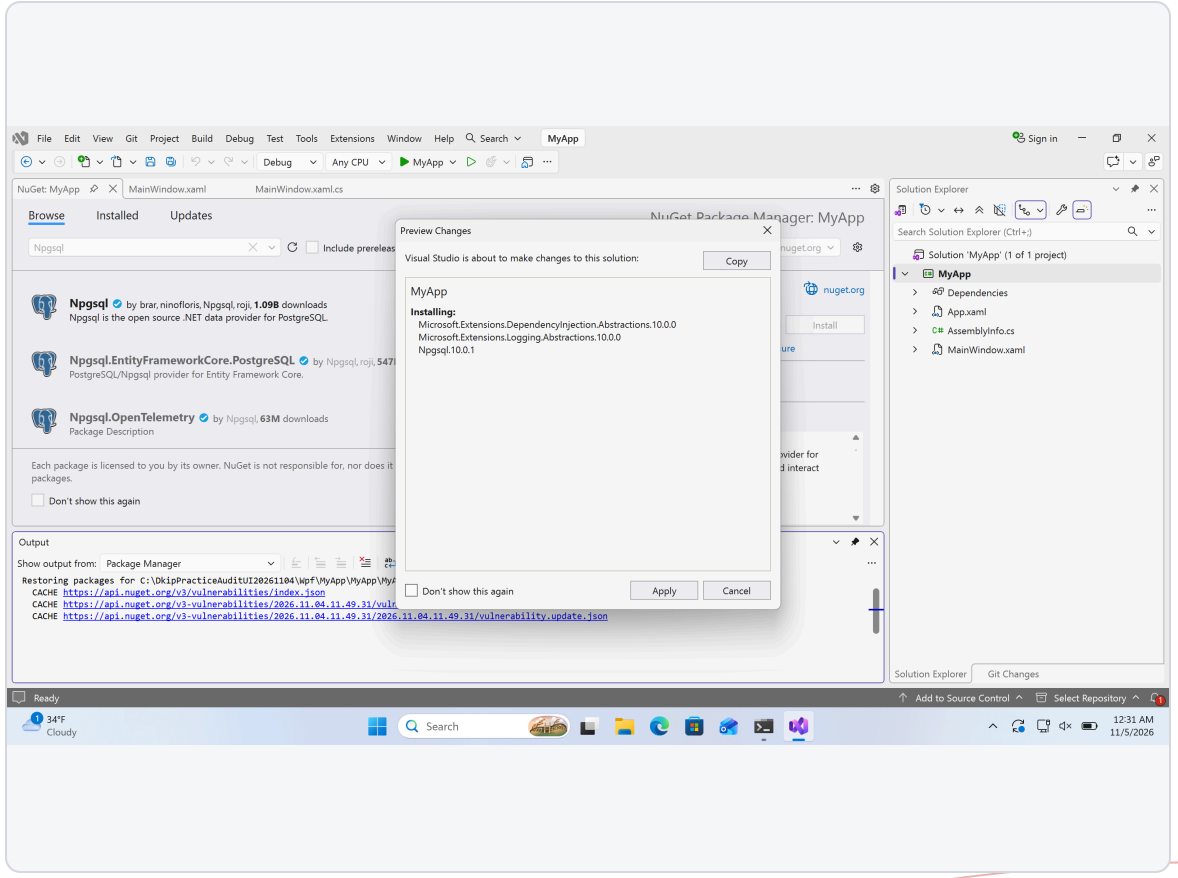
Выберите версию MyApp

Провайдер — библиотека, через которую C# подключается к выбранной СУБД. NuGet устанавливает её зависимость в конкретный проект.

1. В правой панели **NuGet** найдите **Version**. Нажмите стрелку списка и выберите готовый пункт **10.0.1**. Выбор из списка связывает версию с пакетом; простое изменение текста в поле может оставить кнопку **Install** недоступной.

ЧТО ПРОВЕРИТЬ

В **Version** выбрана нужная версия; кнопка **Install** доступна.



Окно **Preview Changes** для **Npgsql 10.0.1**. **Apply** подтверждает установку показанного пакета и зависимостей.

ШАГ 10 ИЗ 30

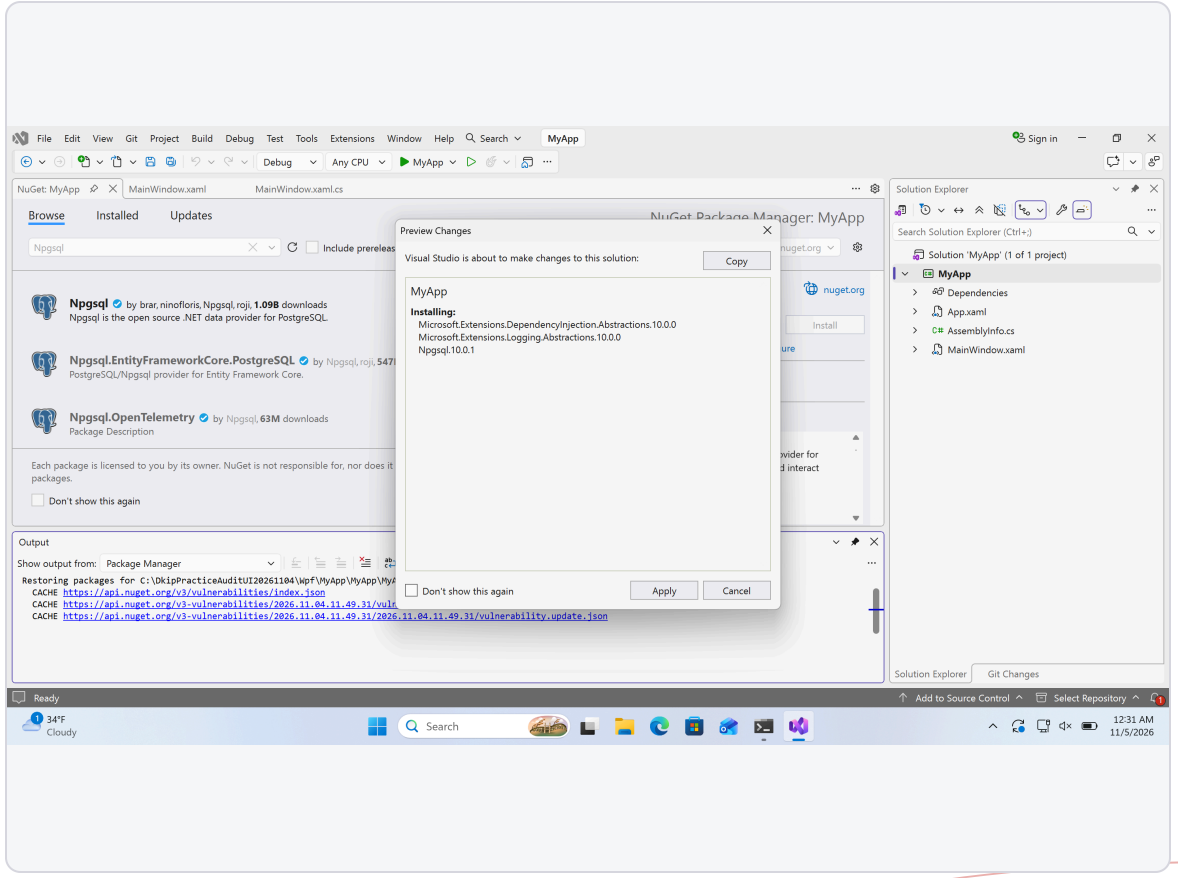
Установите провайдер MyApp

Провайдер — библиотека, через которую C# подключается к выбранной СУБД. NuGet устанавливает её зависимость в конкретный проект.

1. Нажмите **Install** («Установить»). Если появилось окно **Preview Changes** («Просмотр изменений»), сверяйте название пакета и версию, затем нажмите **Apply** («Применить»). Дождитесь окончания установки. Не закрывайте **Visual Studio** во время загрузки.

ЧТО ПРОВЕРИТЬ

Установка завершилась; в карточке пакета показана установленная версия.



Окно **Preview Changes** для **Npgsql 10.0.1**. **Apply** подтверждает установку показанного пакета и зависимостей.

ШАГ 11 ИЗ 30

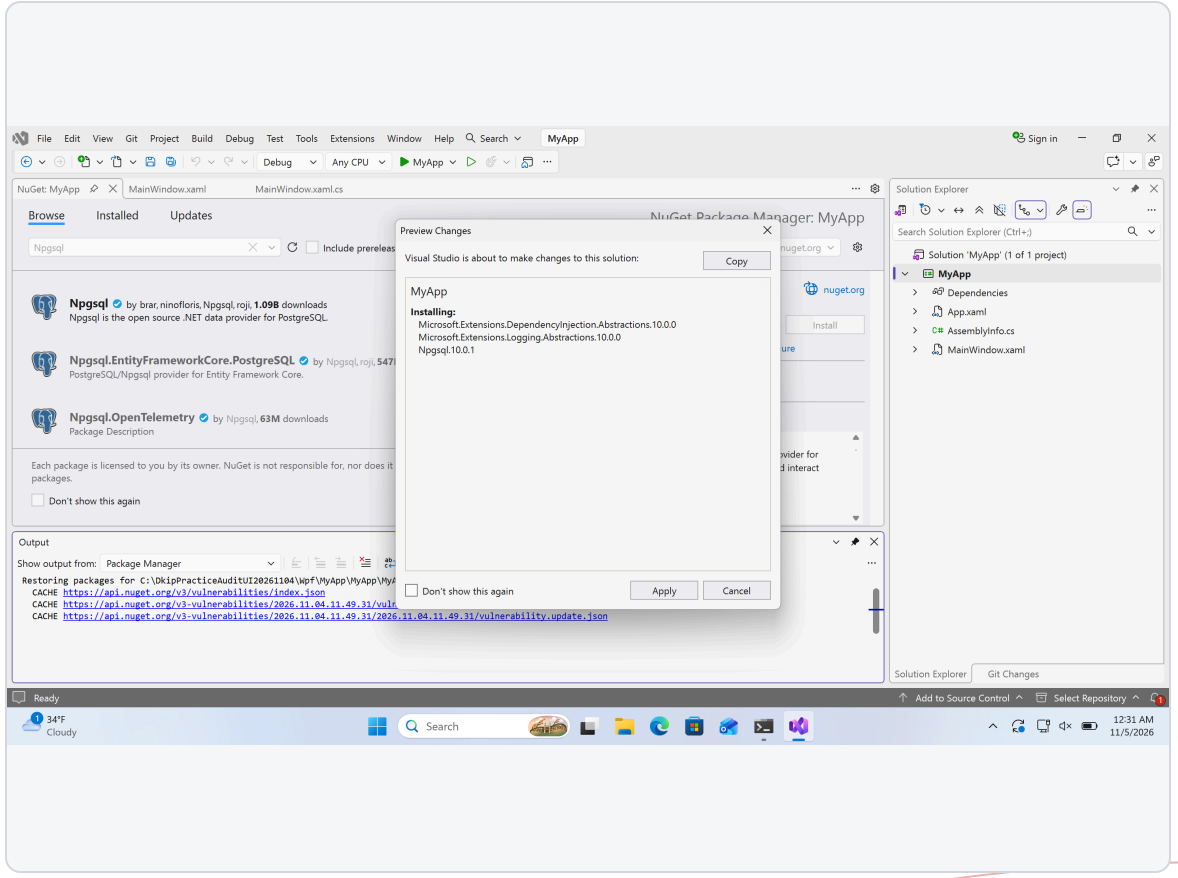
Проверьте провайдер MyApp

Провайдер — библиотека, через которую C# подключается к выбранной СУБД. NuGet устанавливает её зависимость в конкретный проект.

1. В Обозревателе решений раскройте **Dependencies** («Зависимости») → **Packages** («Пакеты»). Должен появиться **Npgsql** выбранной версии. Нажмите **Build** («Сборка») → **Build Solution** («Собрать решение»). В нижнем окне не должно быть ошибок.

ЧТО ПРОВЕРИТЬ

Провайдер появился в **Dependencies** → **Packages**; сборка решения завершилась без ошибок.



Окно **Preview Changes** для **Npgsql 10.0.1**. **Apply** подтверждает установку показанного пакета и зависимостей.

ШАГ 12 ИЗ 30

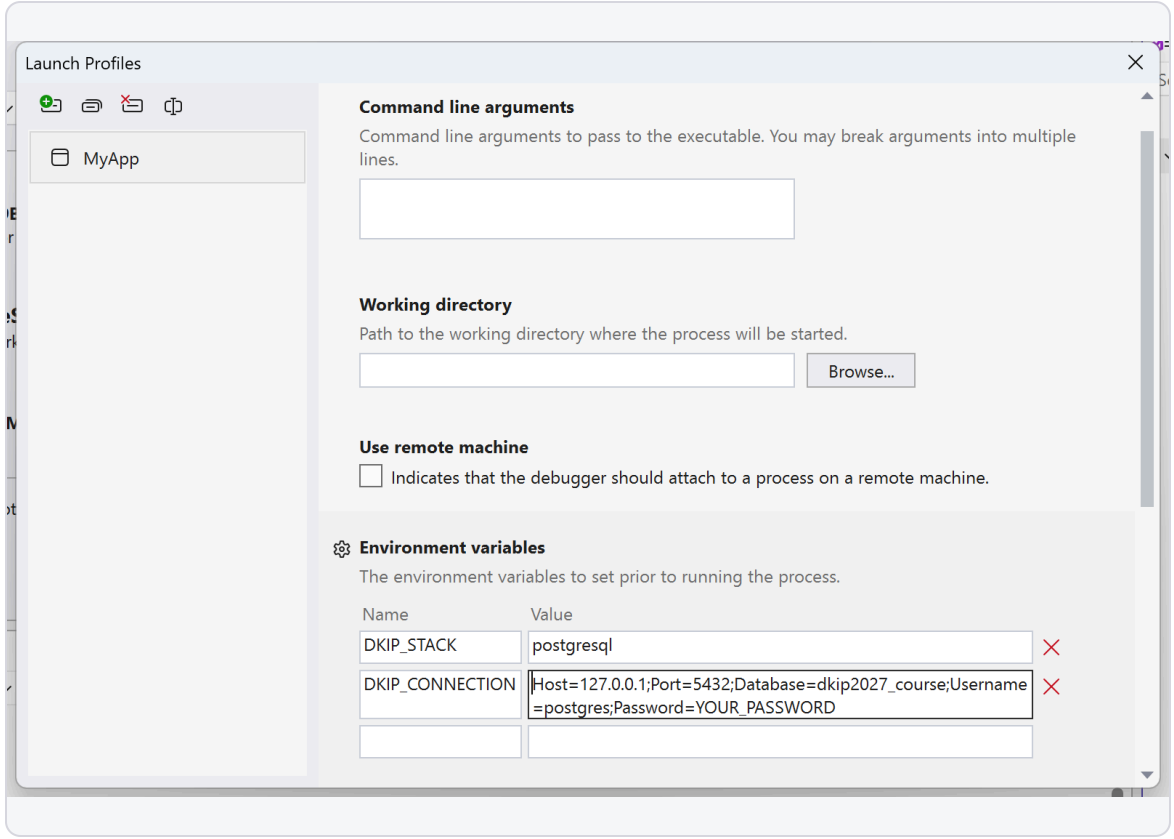
Откройте профиль подключения MyApp

Профиль запуска передаёт настройки программе. Адрес сервера и пароль БД отличаются от логина и пароля пользователя приложения.

1. Выделите проект MyApp в Обозревателе решений. Выберите меню «Отладка» → «Свойства отладки MyApp». Другой путь: свойства проекта → «Отладка» → «Открыть профили запуска отладки». Выберите профиль с запуском самого проекта.

ЧТО ПРОВЕРИТЬ

Открыто окно профилей запуска именно нужного проекта; видна таблица **Environment variables**.



Профиль запуска Visual Studio: DKIP_STACK=postgresql и DKIP_CONNECTION в Environment variables. YOUR_PASSWORD — заглушка, замените её паролем своего сервера.

ШАГ 13 ИЗ 30 · 1/2

Введите параметры подключения MyApp

Профиль запуска передаёт настройки программе. Адрес сервера и пароль БД отличаются от логина и пароля пользователя приложения.

1. Прокрутите окно профиля до **Environment variables**. В первой пустой строке нажмите поле **Name**, введите **DKIP_CONNECTION** и нажмите **Tab**. Скопируйте строку ниже в поле **Value**. Замените имя базы, порт и учётные данные значениями своей учебной установки. Подставьте пароль пользователя **PostgreSQL** вместо ВАШ_ПАРОЛЬ.

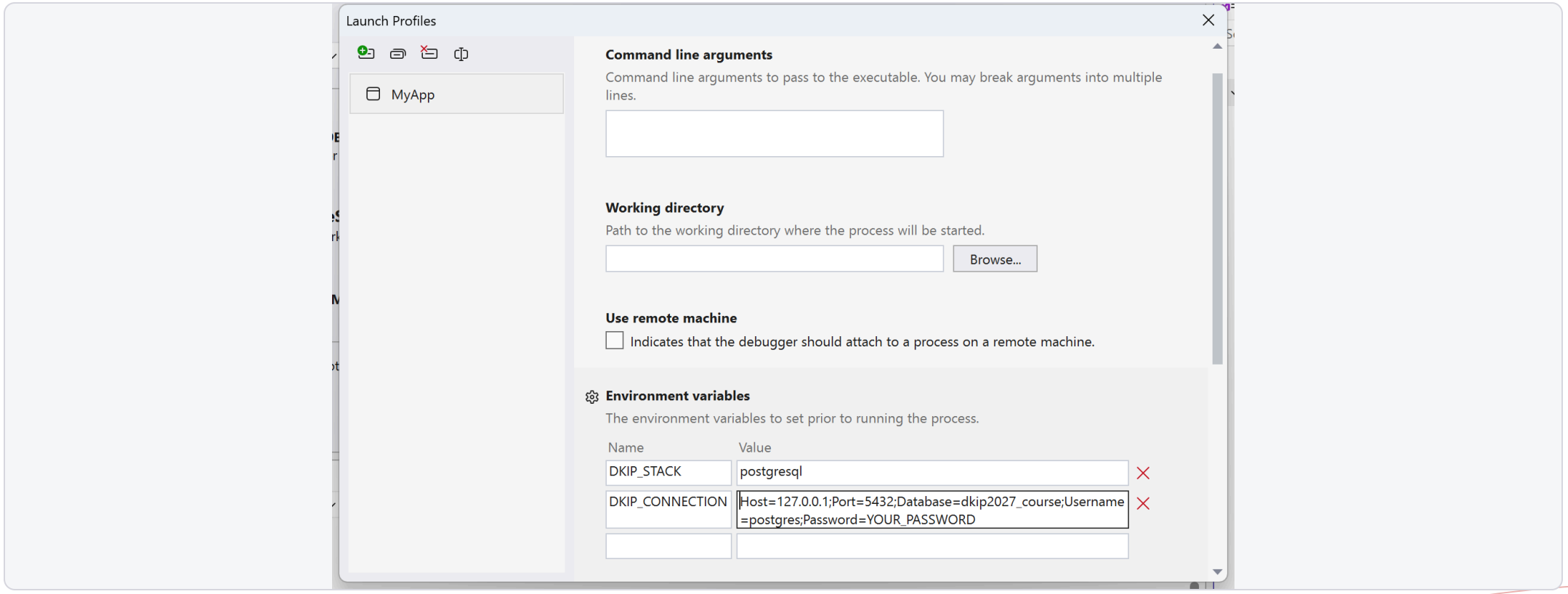
ЧТО ПРОВЕРИТЬ

В **DKIP_CONNECTION** указаны параметры своей учебной базы, без незаменённых заглушек.

ШАГ 13 ИЗ 30 · 2/2

Введите параметры подключения MyApp

Покажите на экране



Профиль запуска **Visual Studio**: **DKIP_STACK**=**postgresql** и **DKIP_CONNECTION** в **Environment variables**. **YOUR_PASSWORD** — заглушка, замените её паролем своего сервера.

ШАГ 13 · КОД ДЛЯ ЭТОГО ШАГА

Введите параметры подключения MyApp

```
Host=127.0.0.1;Port=5432;Database=dkip2027_course;Username=postgres;Password=ВАШ_ПАРОЛЬ
```

ШАГ 14 ИЗ 30

Подключите проект к своей базе

Перед проверкой входа убедитесь, что приложение видит таблицу **users** из задания 2.

1. Правой кнопкой по **MyApp** → «Добавить» → «Класс». Введите **Database.cs**. Вставьте полный код из кнопки «Открыть Database.cs», заменив содержимое созданного файла. Проверьте **namespace MyApp** в начале файла.
2. Дважды щёлкните «Войти» в конструкторе. Внутри **Click** замените старую строку кодом ниже. Нажмите **Ctrl+S** и **Ctrl+F5**. Сравните число с количеством строк **users** в редакторе базы.

ЧТО ПРОВЕРИТЬ

По нажатию кнопки выводится число строк таблицы **users**, включая **0** для пустой таблицы.

ШАГ 14 · ВРЕМЕННАЯ ПРОВЕРКА ПОДКЛЮЧЕНИЯ

Подключите проект к своей базе

```
MessageBox.Show(Database.Scalar("SELECT COUNT(*) FROM users")?.ToString());
```

ШАГ 14 · DATABASE.CS · 1/4

Подключите проект к своей базе

```
using System.Data;
using System.Data.Common;
using Npgsql;
namespace MyApp;

public static class Database
{
    public static DbConnection Open()
    {
        // Профиль запуска передаёт строку подключения при запуске приложения.
        var connectionString = Environment.GetEnvironmentVariable("DKIP_CONNECTION");
        if (string.IsNullOrWhiteSpace(connectionString))
            throw new InvalidOperationException("Укажите DKIP_CONNECTION в профиле запуска.");

        var connection = new NpgsqlConnection(connectionString);
        try
```



ШАГ 14 · DATABASE.CS · 2/4

Подключите проект к своей базе

```
        {
            connection.Open();
            return connection;
        }
        catch { connection.Dispose(); throw; }
    }
    public static DbCommand Command(DbConnection connection, string sql, params object?[] values)
    {
        var cmd = connection.CreateCommand();
        cmd.CommandText = sql;
        for (int i = 0; i < values.Length; i++)
        {
            var p = cmd.CreateParameter();
            p.ParameterName = "p" + i;
            p.Value = values[i] ?? DBNull.Value;
```



ШАГ 14 · DATABASE.CS · 3/4

Подключите проект к своей базе

```
            cmd.Parameters.Add(p);
        }
        return cmd;
    }
    public static int Execute(string sql, params object?[] values)
    {
        using var c = Open();
        using var cmd = Command(c, sql, values);
        return cmd.ExecuteNonQuery();
    }
    public static DataTable Query(string sql, params object?[] values)
    {
        using var c = Open();
        using var cmd = Command(c, sql, values);
        using var reader = cmd.ExecuteReader();
        var result = new DataTable();
```



ШАГ 14 · DATABASE.CS · 4/4

Подключите проект к своей базе

```
        result.Load(reader);
        return result;
    }
    public static object? Scalar(string sql, params object?[] values)
    {
        using var c = Open();
        using var cmd = Command(c, sql, values);
        return cmd.ExecuteScalar();
    }
}
```

ШАГ 15 ИЗ 30

Заполните пустую таблицу двумя учебными записями

Для проверки нужны обе роли. Две строки можно добавить в редакторе базы и сразу приступить к окнам.

1. В **pgAdmin** выберите свою учебную базу, затем «Инструменты» → **Query Tool**.
2. Если **users** уже содержит подходящие тестовые записи с обеими ролями, используйте их. Для пустой **users** вставьте **SQL** из блока и выполните его один раз.
3. Проверьте строки в **users**. Логины для окна «Вход» — **admin** и **student**. Для обеих записей учебный пароль **Demo2027!**.

ЧТО ПРОВЕРИТЬ

В **users** есть **admin** с ролью **admin** и **student** с ролью **user**. Счётчики равны нулю, обе записи разблокированы.

ШАГ 15 · SQL ДЛЯ ПУСТОЙ УЧЕБНОЙ ТАБЛИЦЫ USERS

Заполните пустую таблицу двумя учебными записями

```
INSERT INTO users (login, password_hash, role, failed_attempts, is_locked)
VALUES
    ('admin', 'ZGtpcC1kZW1vLWFkbWluIQ==:razCURk4WRm93+4n6JRjHafFWVGLT9mE/zK6yM6vFq8=', 'admin', 0, FALSE),
    ('student', 'ZGtpcC1kZW1vLXN0dWQuIQ==:oGrwAbgIg57bmM8lZOkiV+Um0GPIyCSS4+sb9YweW+A=', 'user', 0, FALSE);
```

ШАГ 16 ИЗ 30 · 1/3

Выберите операции для запроса

Сначала решите, что должен получить пользователь. От этого зависят поля, таблицы и операции запроса. Дальше вы проверите эти решения в редакторе базы и примените их в приложении.

1. Одна строка результата. Сначала определите, что она представляет: заказ, ресурс, учётную запись или итог по группе. Выберите исходную таблицу в **FROM**, поля в **SELECT** и условие отбора в **WHERE**.
2. Данные из нескольких таблиц. **JOIN** возвращает строки с совпадением по **ON**. **LEFT JOIN** сохраняет все строки слева. Связь берите из ER-диаграммы, а после соединения сверяйте число строк.

ШАГ 16 ИЗ 30 · 2/3

Выберите операции для запроса

1. Число строк или итог по группе. COUNT считает, SUM складывает, MAX выбирает наибольшее значение. GROUP BY задаёт, по какому ключу получить отдельный итог. В задании 3 MAX(valid_from) выбирает последнюю допустимую дату цены после отбора по дате заказа.
2. Запасное значение или правило. COALESCE выбирает первое значение, отличное от NULL. CASE выбирает результат по условию. Перед подстановкой проверьте, что означает NULL: в расчёте заказа отсутствие цены должно быть видно.

ШАГ 16 ИЗ 30 · 3/3

Выберите операции для запроса

1 Округление и порядок. ROUND округляет числовой результат, например итог расчёта до копеек. ORDER BY сортирует готовые строки. Для проверки сначала выведите исходные поля, затем добавляйте по одной операции.

ЧТО ПРОВЕРИТЬ

Для задачи «показать заказ с именем покупателя» вы можете назвать обе таблицы и связующий ключ. Для задачи «узнать, занят ли логин» достаточно отобрать и посчитать строки **users**.

ШАГ 17 ИЗ 30 · 1/2

Прочитайте учётную запись

Для входа требуется одна строка из **users**. Отдельный **SELECT** позволяет проверить выбор записи перед тем, как тот же поиск выполнит обработчик кнопки.

1. В **pgAdmin** выделите учебную базу и откройте **Tools → Query Tool**. Результат появится в **Data Output**. Выполняйте запросы по одному. Для примеров используйте записи, подготовленные в заданиях 3 и 4.
2. Определите, что должна означать одна строка результата. Для входа нужна одна учётная запись, поэтому начните с **FROM users**. В **SELECT** перечислите нужные поля, а в **WHERE** укажите логин.

ШАГ 17 ИЗ 30 · 2/2

Прочитайте учётную запись

1 В Accounts.SignIn C# передаёт логин в параметр @p0. Запрос также читает хеш. FOR UPDATE блокирует выбранную строку до конца транзакции, чтобы одновременные попытки входа не перезаписали счётчик друг друга.

ЧТО ПРОВЕРИТЬ

Для созданной записи **student** вернётся одна строка. Сверьте её с таблицей **users**. Если строк нет, проверьте подключение и подставьте логин своей записи.

ШАГ 17 · НАЙТИ УЧЕБНУЮ ЗАПИСЬ

Прочитайте учётную запись

```
SELECT id, login, role, failed_attempts, is_locked
FROM users
WHERE login = 'student';
```

ШАГ 18 ИЗ 30

Посчитайте записи с таким логином

При сохранении формы надо проверить, занят ли логин другой записью. Условие WHERE определяет, какие строки попадут в счётчик.

1. При сохранении важно узнать, занят ли логин. COUNT(*) считает строки, которые прошли WHERE, и возвращает одно число.
2. В Accounts.Save условие id <> @p1 исключает редактируемую запись. В редакторе добавьте к WHERE условие AND id <> и подставьте после него id из результата предыдущего шага. Ограничение UNIQUE на login также защищает таблицу от повторов.

ЧТО ПРОВЕРИТЬ

Если **student** существует, **matches** = 1. При исключении **id** этой записи получится **0**. Другой пользователь с этим логином сохраниться не должен.

ШАГ 18 · ПОСЧИТАТЬ СОВПАДЕНИЯ

Посчитайте записи с таким логином

```
SELECT COUNT(*) AS matches
FROM users
WHERE login = 'student';
```

ШАГ 19 ИЗ 30

Сохраните старое значение или выберите новое

Одна команда UPDATE может сохранить прежний хеш и изменить состояние блокировки по правилам формы. Отдельный SELECT показывает, какие значения дадут эти выражения.

1. При редактировании поле нового пароля можно оставить пустым. Тогда C# передаёт NULL, а COALESCE(@p2, password_hash) берёт первый аргумент, отличный от NULL, и сохраняет старый хеш.
2. Для флажка «Снять блокировку» нужно выбрать действие по условию. CASE WHEN @p3 THEN 0 ELSE failed_attempts END сбрасывает счётчик при включённом флажке. Второе выражение CASE так же выбирает значение is_locked.

ЧТО ПРОВЕРИТЬ

В столбцах по порядку появятся «старое», «новое», 0 и 2. Меняйте аргументы в редакторе и сверяйте результат.

ШАГ 19 · ПРОВЕРИТЬ ВЫРАЖЕНИЯ БЕЗ ЗАПИСИ В ТАБЛИЦУ

Сохраните старое значение или выберите новое

```
SELECT COALESCE(NULL, 'старое') AS kept,
       COALESCE('новое', 'старое') AS changed,
       CASE WHEN TRUE THEN 0 ELSE 2 END AS reset_count,
       CASE WHEN FALSE THEN 0 ELSE 2 END AS kept_count;
```

ШАГ 20 ИЗ 30

Соедините таблицы по ключу

Номер заказа и имя покупателя хранятся в разных таблицах. JOIN собирает их в одну строку по связи между внешним и первичным ключами.

1. Для списка учётных записей задания 4 достаточно **users**. Для заказа с именем покупателя нужны две таблицы: номер берётся из **customer_order**, а имя из **counterparty**. Сначала выведите номер и **customer_id**, затем добавьте JOIN и имя.

2. В запросе **o** обозначает **customer_order**, а **c** — **counterparty**. Условие **ON c.id = o.customer_id** повторяет связь FK → PK из ER-диаграммы. В задании 5 таким же способом находится логин автора по **notes.id_user** → **users.id**.

ЧТО ПРОВЕРИТЬ

Номер и код покупателя совпадают в обоих результатах. Имя во втором запросе совпадает с **name** той строки **counterparty**, чей **id** равен **customer_id**.



ШАГ 20 · 1. НАЙТИ КОД ПОКУПАТЕЛЯ В ЗАКАЗЕ

Соедините таблицы по ключу

```
SELECT doc_no, customer_id
FROM customer_order
WHERE doc_no = '1';
```

ШАГ 20 · 2. ПОЛУЧИТЬ ИМЯ ПОКУПАТЕЛЯ ПО КЛЮЧУ

Соедините таблицы по ключу

```
SELECT o.doc_no, o.customer_id, c.name AS customer_name
FROM customer_order o
JOIN counterparty c ON c.id = o.customer_id
WHERE o.doc_no = '1';
```

ШАГ 21 ИЗ 30

Сохраните строки без совпадения

У пользователя может ещё не быть заметок. Чтобы он попал в отчёт с числом 0, запрос должен сохранить его строку при соединении.

1. Если в отчёте нужны все пользователи, включая пользователей без заметок, начните с **users** и используйте **LEFT JOIN**. Для пользователя без заметок поля **n.*** будут **NULL**.
2. **GROUP BY** оставляет одну строку на пользователя. **COUNT(n.id)** считает только существующие заметки. **COUNT(*)** посчитал бы и строку с **NULL**, которую сохраняет **LEFT JOIN**.

ЧТО ПРОВЕРИТЬ

Строк столько же, сколько пользователей. Пока **notes** пуста, у каждого **notes_count = 0**. После добавления заметок в задании 5 сверьте счётчики по **id_user**.

ШАГ 21 · ПОСЧИТАТЬ ЗАМЕТКИ КАЖДОГО ПОЛЬЗОВАТЕЛЯ

Сохраните строки без совпадения

```
SELECT u.id, u.login, COUNT(n.id) AS notes_count
FROM users u
LEFT JOIN notes n ON n.id_user = u.id
GROUP BY u.id, u.login
ORDER BY u.id;
```

ШАГ 22 ИЗ 30 · 1/2

Проверьте логин и пароль

Отработайте поиск записи и проверку пароля отдельно. Тогда ошибку сборки пазла будет проще отличить от ошибки входа.

1. Правой кнопкой по MyApp → «Добавить» → «Класс» → Accounts.cs. Замените всё содержимое кодом из «Открыть Accounts.cs». В строке **namespace Dkip** замените **Dkip** на **MyApp**.
2. Проследите начало **SignIn**. Пустые поля проверяются до запроса. По логину метод находит строку **users**, считывает сохранённый хеш, роль и состояние блокировки. **Verify** проверяет введённый пароль по хешу из этой строки.
3. Замените временный вывод числа строк блоком ниже. Временно передайте **true** вместо результата пазла. На следующем этапе это значение заменится настоящей проверкой.

ШАГ 22 ИЗ 30 · 2/2

Проверьте логин и пароль

1 Проверьте пустые поля. Затем для разблокированного **student** с нулевым счётчиком введите один неверный пароль и обновите **users** в редакторе БД. В **failed_attempts** появится **1**. Введите верный пароль и снова обновите строку. Счётчик вернётся к **0**.

ЧТО ПРОВЕРИТЬ

Вход с пустыми полями останавливается до поиска. Неверный пароль увеличивает счётчик до **1**, а правильный вход сбрасывает его до **0**.



ШАГ 22 · ВНУТРИ ОБРАБОТЧИКА CLICK

Проверьте логин и пароль

```
try
{
    var user = Accounts.SignIn(LoginInput.Text, PasswordInput.Password, true);
    MessageBox.Show(user.Login);
}
catch (InvalidOperationException ex)
{
    MessageBox.Show(ex.Message);
}
catch (System.Data.Common.DbException)
{
    MessageBox.Show("Проверьте запуск базы и DKIP_CONNECTION.");
}
```

ШАГ 22 · ACCOUNTS.CS · 1/9

Проверьте логин и пароль

```
using System.Security.Cryptography;
namespace Dkip;

public record Account(int Id, string Login, string Role);
public static class Accounts
{
    public const string Wrong = "Вы ввели неверный логин или пароль. Пожалуйста проверьте ещё раз введенные данные";
    public const string Locked = "Вы заблокированы. Обратитесь к администратору";
    const int MaxAttempts = 3;

    // Соль хранится рядом с хешем. Сам пароль в БД не записываем.
    public static string Hash(string password)
    {
        var salt = RandomNumberGenerator.GetBytes(16);
        var hash = DeriveHash(password, salt);
```



ШАГ 22 · ACCOUNTS.CS · 2/9

Проверьте логин и пароль

```
        return Convert.ToBase64String(salt) + ":" + Convert.ToBase64String(hash);
    }
    public static bool Verify(string password, string hash)
    {
        try
        {
            var parts = hash.Split(':');
            if (parts.Length != 2)
                return false;

            var salt = Convert.FromBase64String(parts[0]);
            var savedHash = Convert.FromBase64String(parts[1]);
            return CryptographicOperations.FixedTimeEquals(savedHash, DeriveHash(password, salt));
        }
        catch (FormatException)
```



ШАГ 22 · ACCOUNTS.CS · 3/9

Проверьте логин и пароль

```
        {
            return false;
        }
    }

    static byte[] DeriveHash(string password, byte[] salt) =>
        Rfc2898DeriveBytes.Pbkdf2(password, salt, 100000, HashAlgorithmName.SHA256, 32);

    public static Account SignIn(string login, string password, bool puzzleSolved)
    {
        login = login.Trim().ToLowerInvariant();
        if (login.Length == 0 || password.Length == 0)
            throw new InvalidOperationException("Заполните логин и пароль.");
        // Пока проверяем вход, другая попытка не может изменить эту строку.
        using var connection = Database.Open();
        using var transaction = connection.BeginTransaction();
```



ШАГ 22 · ACCOUNTS.CS · 4/9

Проверьте логин и пароль

```
        using var findUser = Database.Command(connection,
            "SELECT id, password_hash, role, failed_attempts, is_locked FROM users WHERE login=@p0 FOR UPDATE", login);
        findUser.Transaction = transaction;

        int id;
        int attempts;
        string savedHash;
        string role;
        using (var reader = findUser.ExecuteReader())
        {
            if (!reader.Read())
                throw new InvalidOperationException(Wrong);
            if (Convert.ToBoolean(reader["is_locked"]))
                throw new InvalidOperationException(Locked);
```

ШАГ 22 · ACCOUNTS.CS · 5/9

Проверьте логин и пароль

```
            id = Convert.ToInt32(reader["id"]);
            savedHash = (string)reader["password_hash"];
            role = (string)reader["role"];
            attempts = Convert.ToInt32(reader["failed_attempts"]);
        }

        bool passwordCorrect = Verify(password, savedHash);
        bool accepted = passwordCorrect && puzzleSolved;

        // Пароль и пазл делят один счётчик. Успешный вход его обнуляет.
        if (accepted)
            attempts = 0;
        else
            attempts++;

        bool blocked = attempts >= MaxAttempts;
```

ШАГ 22 · ACCOUNTS.CS · 6/9

Проверьте логин и пароль

```
        using var update = Database.Command(connection,
            "UPDATE users SET failed_attempts=@p0, is_locked=@p1 WHERE id=@p2", attempts, blocked, id);
        update.Transaction = transaction;
        update.ExecuteNonQuery();
        // Сохраняем результат даже при ошибке, иначе попытка потеряется.
        transaction.Commit();

        if (blocked)
            throw new InvalidOperationException(Locked);
        if (!passwordCorrect)
            throw new InvalidOperationException(Wrong);
        if (!puzzleSolved)
            throw new InvalidOperationException("Пазл собран неверно. Осталось попыток: " + (MaxAttempts - attempts));
```



ШАГ 22 · ACCOUNTS.CS · 7/9

Проверьте логин и пароль

```
        return new Account(id, login, role);
    }

    public static void RequireAdmin(Account actor)
    {
        if (!Equals(Database.Scalar("SELECT role FROM users WHERE id=@p0 AND is_locked=FALSE", actor.Id), "admin"))
            throw new InvalidOperationException("Действие доступно только администратору.");
    }
    public static void Save(Account actor, int? id, string login, string password, string role, bool unlock)
    {
        RequireAdmin(actor);
        login = login.Trim().ToLowerInvariant();
        if (login.Length is < 1 or > 64 || role is not ("admin" or "user"))
```



ШАГ 22 · ACCOUNTS.CS · 8/9

Проверьте логин и пароль

```
            throw new InvalidOperationException("Введите логин до 64 символов и выберите роль.");
        if (id is null && password.Length == 0)
            throw new InvalidOperationException("Введите пароль нового пользователя.");
        if (id == actor.Id && role != "admin")
            throw new InvalidOperationException("Нельзя снять собственную роль администратора.");
        if (Convert.ToInt32(Database.Scalar("SELECT COUNT(*) FROM users WHERE login=@p0 AND id<>@p1", login, id ?? -1))
> 0)
            throw new InvalidOperationException("Пользователь с указанным логином уже существует.");
        if (id is null)
        {
            Database.Execute(
                "INSERT INTO users(login, password_hash, role, failed_attempts, is_locked)
VALUES(@p0,@p1,@p2,0,FALSE)",
```

ШАГ 22 · ACCOUNTS.CS · 9/9

Проверьте логин и пароль

```
                login, Hash(password), role);
            return;
        }

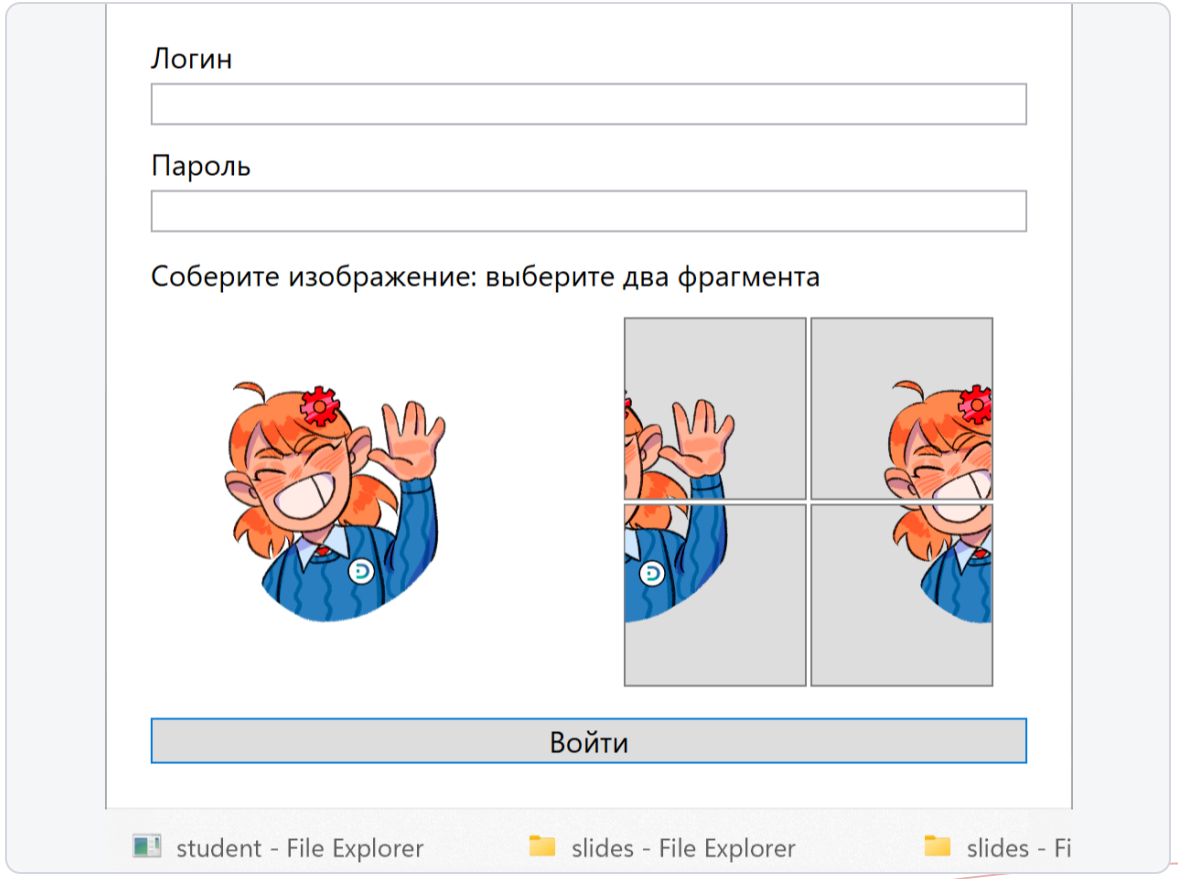
        // Пустой новый пароль сохраняет старый хеш. При разблокировке сбрасываем оба поля.
        string? newHash = password.Length > 0 ? Hash(password) : null;
        Database.Execute("""
            UPDATE users SET login=@p0, role=@p1, password_hash=COALESCE(@p2, password_hash),
                failed_attempts=CASE WHEN @p3 THEN 0 ELSE failed_attempts END,
                is_locked=CASE WHEN @p3 THEN FALSE ELSE is_locked END
            WHERE id=@p4
            """, login, role, newHash, unlock, id);
    }
}
```

ШАГ 23 ИЗ 30 · 1/3

Покажите фрагменты пазла

Четыре исходных PNG уже являются частями одной картинки. Правильный порядок нужен и для образца, и для проверки.

1. В Обозревателе решений нажмите правой кнопкой по MyApp → «Добавить» → «Новая папка», назовите Assets. Правой кнопкой по Assets → «Добавить» → «Существующий элемент». Выберите 1.png–4.png из исходных файлов задания 4.
2. Выделите добавленные PNG и нажмите F4. В «Копировать в выходной каталог» выберите «Копировать, если новее». Так программа найдёт PNG рядом с EXE.



WPF. Окно «Вход»: поля логина и пароля, образец пазла слева и четыре фрагмента справа.

ШАГ 23 ИЗ 30 · 2/3

Покажите фрагменты пазла

1. Откройте **LoginWindow.xaml**. Во вкладке XAML найдите тег **Button**. Вставьте перед ним разметку первого блока, внутри **StackPanel**. У кнопки укажите **Click="SignInButton_Click"**.
2. В Обозревателе решений раскройте **LoginWindow.xaml** и откройте **LoginWindow.xaml.cs**. Замените всё содержимое вторым блоком. В нём уже есть прежняя проверка пароля и готовые методы пазла.
3. В коде **order** хранит номера фрагментов, а индекс массива обозначает ячейку. Индексы начинаются с **0**. При отрисовке ячейка **i** получает фрагмент с номером **order[i]**.

ШАГ 23 ИЗ 30 · 3/3

Покажите фрагменты пазла

1 Запустите приложение. Слева сравните образец с файлами. Справа проверьте, что все четыре части видны и расположены в другом порядке.

ЧТО ПРОВЕРИТЬ

Окно показывает исходное изображение и перемешанную сетку 2 × 2.

ШАГ 23 · 1. РАЗМЕТКА ПАЗЛА В LOGINWINDOW.XAML

Покажите фрагменты пазла

```
<Grid Height="204">
    <Grid.ColumnDefinitions>
        <ColumnDefinition/>
        <ColumnDefinition/>
    </Grid.ColumnDefinitions>
    <Image x:Name="Original" Width="200" Height="200"/>
    <UniformGrid x:Name="Puzzle" Grid.Column="1" Rows="2" Columns="2"/>
</Grid>
```



ШАГ 23 · 2. ПОЛНЫЙ LOGINWINDOW.XAML.CS НА ЭТОМ ШАГЕ · 1/8

Покажите фрагменты пазла

```
using System.Data.Common;
using System.IO;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Media.Imaging;
namespace MyApp;

public partial class LoginWindow : Window
{
    readonly int[] order = [1, 0, 3, 2];
    readonly BitmapSource[] pieces = new BitmapSource[4];
    readonly Button[] tiles = new Button[4];
    int selected = -1;
    public LoginWindow()
    {
```

ШАГ 23 · 2. ПОЛНЫЙ LOGINWINDOW.XAML.CS НА ЭТОМ ШАГЕ · 2/8

Покажите фрагменты пазла

```
        InitializeComponent();
        try
        {
            LoadPuzzle();
        }
        catch (Exception ex) when (ex is IOException or NotSupportedException)
        {
            SignInButton.IsEnabled = false;
            MessageBox.Show("Проверьте четыре PNG в папке Assets рядом с приложением.",
                "Не удалось открыть пазл", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }

    void LoadPuzzle()
    {
        // Четыре PNG уже идут в правильном порядке. Сначала показываем образец.
```



ШАГ 23 · 2. ПОЛНЫЙ LOGINWINDOW.XAML.CS НА ЭТОМ ШАГЕ · 3/8

Покажите фрагменты пазла

```
        var visual = new DrawingVisual();
        using (var drawing = visual.RenderOpen())
        {
            for (int i = 0; i < 4; i++)
            {
                var image = new BitmapImage();
                image.BeginInit();
                image.CacheOption = BitmapCacheOption.OnLoad;
                image.UriSource = new Uri(Path.Combine(AppContext.BaseDirectory, "Assets", (i + 1) + ".png"));
                image.EndInit();
                image.Freeze();
                pieces[i] = image;
                drawing.DrawImage(image, new Rect(i % 2 * 200, i / 2 * 200, 200, 200));
            }
        }
```

ШАГ 23 · 2. ПОЛНЫЙ LOGINWINDOW.XAML.CS НА ЭТОМ ШАГЕ · 4/8

Покажите фрагменты пазла

```
        var preview = new RenderTargetBitmap(400, 400, 96, 96, PixelFormats.Pbgra32);
        preview.Render(visual);
        preview.Freeze();
        Original.Source = preview;
        for (int i = 0; i < 4; i++)
        {
            tiles[i] = new Button { Padding = new Thickness(0), Margin = new Thickness(1) };
            System.Windows.Automation.AutomationProperties.SetName(tiles[i], "Фрагмент " + (i + 1));
            tiles[i].Click += SelectTile;
            Puzzle.Children.Add(tiles[i]);
        }
        RenderPuzzle();
    }
    void SelectTile(object sender, RoutedEventArgs e)
    {
```



ШАГ 23 · 2. ПОЛНЫЙ LOGINWINDOW.XAML.CS НА ЭТОМ ШАГЕ · 5/8

Покажите фрагменты пазла

```
        int position = Array.IndexOf(tiles, (Button)sender);
        if (selected == -1)
        {
            selected = position;
        }
        else
        {
            // Второе нажатие меняет местами два номера в массиве.
            int fragment = order[selected];
            order[selected] = order[position];
            order[position] = fragment;
            selected = -1;
        }
        RenderPuzzle();
    }
```

ШАГ 23 · 2. ПОЛНЫЙ LOGINWINDOW.XAML.CS НА ЭТОМ ШАГЕ · 6/8

Покажите фрагменты пазла

```
bool IsSolved() => order.SequenceEqual(new[] { 0, 1, 2, 3 });

void RenderPuzzle()
{
    for (int i = 0; i < 4; i++)
    {
        tiles[i].Content = new Image { Source = pieces[order[i]], Stretch = Stretch.Fill };
        tiles[i].BorderBrush = selected == i ? Brushes.Red : Brushes.Gray;
        tiles[i].BorderThickness = new Thickness(selected == i ? 3 : 1);
    }
}
void Shuffle()
{
    do
    {
        Random.Shared.Shuffle(order);
```

ШАГ 23 · 2. ПОЛНЫЙ LOGINWINDOW.XAML.CS НА ЭТОМ ШАГЕ · 7/8

Покажите фрагменты пазла

```
        } while (IsSolved());
        selected = -1;
        RenderPuzzle();
    }
    void SignInButton_Click(object sender, RoutedEventArgs e)
    {
        try
        {
            var user = Accounts.SignIn(LoginInput.Text, PasswordInput.Password, true);
            MessageBox.Show(user.Login);
        }
        catch (InvalidOperationException ex)
        {
            MessageBox.Show(ex.Message);
        }
        catch (System.Data.Common.DbException)
```

ШАГ 23 · 2. ПОЛНЫЙ LOGINWINDOW.XAML.CS НА ЭТОМ ШАГЕ · 8/8

Покажите фрагменты пазла

```
            {
                MessageBox.Show("Проверьте запуск базы и DKIP_CONNECTION.");
            }
        }
    }
```

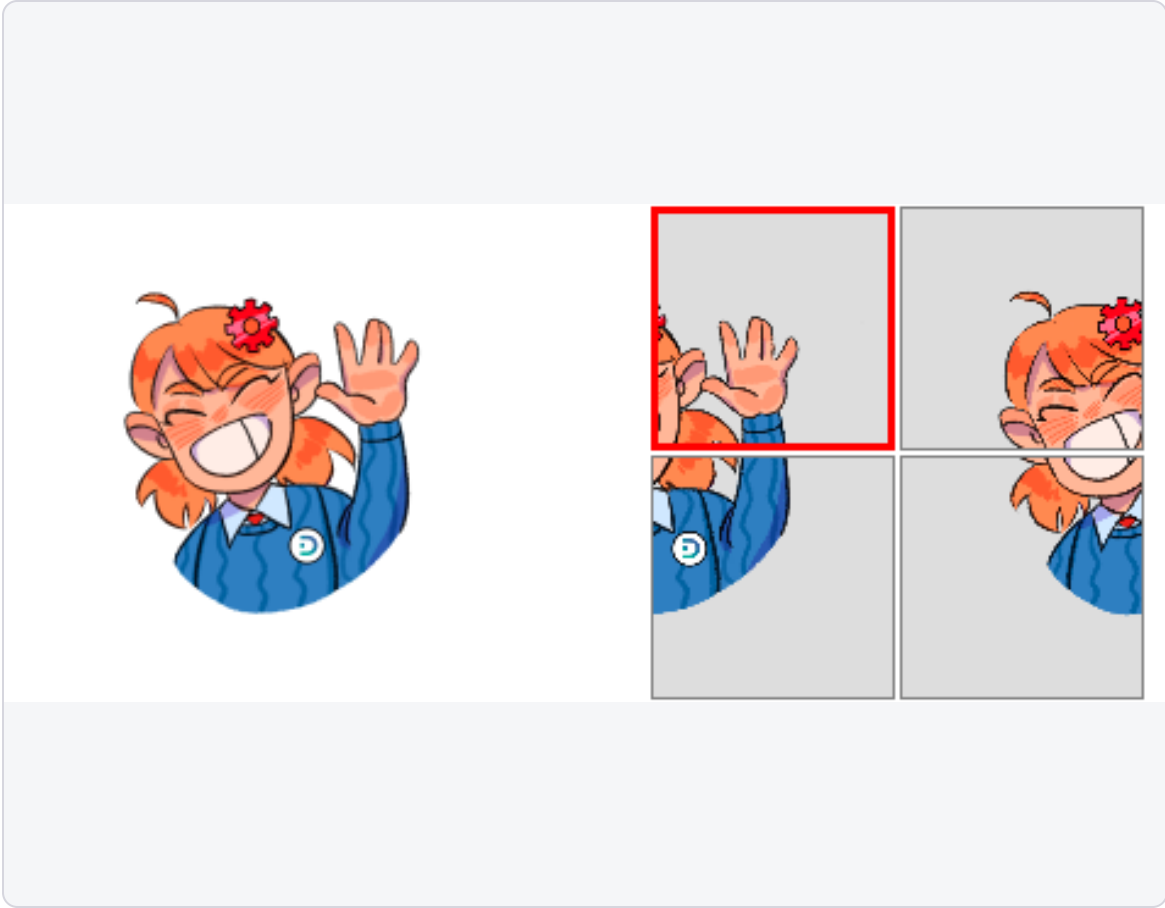
ШАГ 24 ИЗ 30 · 1/3

Поменяйте местами две части

Первый щелчок: запомните ячейку

Первое нажатие запоминает ячейку. Второе меняет местами две части и обновляет картинки.

1. При запуске order = [1, 0, 3, 2], selected = -1. Щёлкните по верхней левой ячейке (0). Теперь selected = 0, вокруг ячейки появилась рамка, порядок частей не изменился.



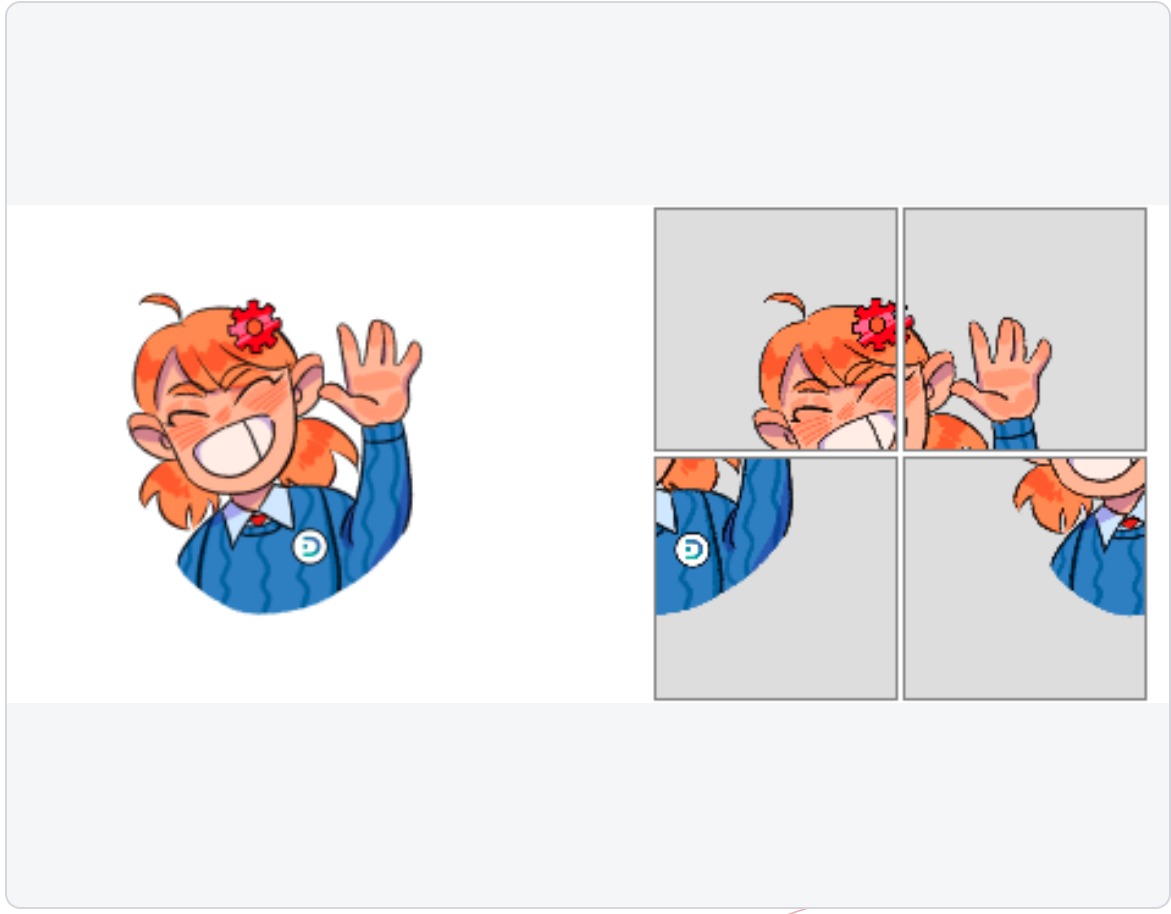
WPF: верхняя левая часть выделена рамкой, фрагменты остались на местах.

ШАГ 24 ИЗ 30 · 2/3

Поменяйте местами две части

Второй щелчок: обменяйте части

1. Щёлкните по верхней правой ячейке (1). В SelectTile переменная fragment хранит старое значение первой ячейки. После обмена order = [0, 1, 3, 2], selected = -1 и рамка исчезает.



WPF: верхние фрагменты поменялись местами, рамки больше нет.

ШАГ 24 ИЗ 30 · 3/3

Поменяйте местами две части

Проверьте собранный рисунок

1. Щёлкните по нижней левой и нижней правой ячейкам (2 и 3). Получится **order** = [0, 1, 2, 3], и **IsSolved** вернёт **true**.

ЧТО ПРОВЕРИТЬ

После первого щелчка есть только выделение. После первой пары выстроен верхний ряд, после второй пары весь рисунок совпадает с образцом.

ШАГ 24 · ОБМЕН ДВУХ ЭЛЕМЕНТОВ В SELECTTILE

Поменяйте местами две части

```
int fragment = order[selected];
order[selected] = order[position];
order[position] = fragment;
selected = -1;
```

ШАГ 25 ИЗ 30 · 1/2

Свяжите пазл с входом и проверьте блокировку

В методе SignIn неправильный пароль и неверный пазл учитываются одинаково. Состояние хранится в users.

1. Дважды щёлкните «Войти» в конструкторе. Замените содержимое обработчика блоком ниже. В третьем аргументе теперь проверяется пазл. После неудачного входа он перемешивается.

2. В SignIn условие passwordCorrect && puzzleSolved требует успеха обеих проверок. При любом отказе увеличивается один failed_attempts. При достижении 3 выставляется is_locked. Метод сохраняет оба поля до сообщения об ошибке, поэтому перезапуск окна сохраняет блокировку.

ШАГ 25 ИЗ 30 · 2/2

Свяжите пазл с входом и проверьте блокировку

1. На тестовой записи **student** с нулевым счётчиком дважды введите неверный пароль. В третьей попытке введите правильный пароль и оставьте пазл несобранным. После каждой попытки обновите **users** и проследите значения (1, FALSE), (2, FALSE), (3, TRUE).
2. Закройте и снова запустите приложение. Повторите вход с верными данными и собранным пазлом. В **users** проверьте **failed_attempts** и **is_locked**.

ЧТО ПРОВЕРИТЬ

На третьей ошибке и при повторном входе появляется сообщение о блокировке. В **users** осталось **3** и признак блокировки.

ШАГ 25 · ГОТОВЫЙ КОД ВНУТРИ CLICK С ПРОВЕРКОЙ ПАЗЛА

Свяжите пазл с входом и проверьте блокировку

```
try
{
    var user = Accounts.SignIn(LoginInput.Text, PasswordInput.Password, IsSolved());
    MessageBox.Show(user.Login);
}
catch (InvalidOperationException ex)
{
    MessageBox.Show(ex.Message);
    Shuffle();
}
catch (System.Data.Common.DbException)
{
    MessageBox.Show("Проверьте запуск базы и DKIP_CONNECTION.");
}
```

ШАГ 26 ИЗ 30 · 1/4

Соберите рабочее окно

Откройте конструктор рабочего окна

Рабочее окно показывает логин и роль. Таблица и поля управления стоят на отдельной панели администратора.

1. Откройте **MainWindow.xaml**, созданный вместе с проектом. Из **Toolbox** добавьте **StackPanel**, затем поместите в него **TextBlock**. В **Name** укажите **RoleLabel**.
2. Ниже подписи поместите второй **StackPanel** внутрь первого. В **Name** введите **AdminPanel**. Все следующие элементы добавляйте внутрь него.

ШАГ 26 ИЗ 30 · 2/4

Соберите рабочее окно

Поставьте таблицу и поля

1. Добавьте в AdminPanel DataGrid. В свойствах укажите имя UsersGrid и включите IsReadOnly.
2. Под таблицей добавьте подписи «Логин», «Новый пароль» и «Роль». Для логина перетащите TextBox с именем UserLogin. Для пароля добавьте PasswordBox с именем NewPassword.
3. Добавьте ComboBox с именем UserRole. Откройте Items в свойствах. Добавьте сначала «Пользователь», потом «Администратор». Установите SelectedIndex=0.

ШАГ 26 ИЗ 30 · 3/4

Соберите рабочее окно

Поставьте таблицу и поля

1. Если редактор Items недоступен, во вкладке XAML добавьте внутрь ComboBox два тега ComboBoxItem. Укажите Content="Пользователь", затем Content="Администратор".

ШАГ 26 ИЗ 30 · 4/4

Соберите рабочее окно

Поставьте флажок и кнопки

1. Из **Toolbox** перетащите **CheckBox**. Задайте имя **Unlock** и подпись «Снять блокировку» в **Content**.
2. Добавьте два **Button**. Для первого задайте имя **NewButton** и подпись «Новый». Для второго — **SaveButton** и подпись «Сохранить». Нажмите **Ctrl+S**.

ЧТО ПРОВЕРИТЬ

В конструкторе видны подпись, таблица, три поля, флажок и две кнопки. Их имена совпадают с инструкцией.

ШАГ 27 ИЗ 30 · 1/4

Подключите таблицу и кнопки администратора

Вставьте готовый код рабочего окна

SelectUser читает выбранную строку, NewUser очищает поля, SaveUser вызывает запись в базу.

1. Раскройте MainWindow.xaml в Обозревателе решений. Откройте MainWindow.xaml.cs и замените его кодом из кнопки ниже. Разметку MainWindow.xaml оставьте своей.
2. Замените namespace Dkip на MyApp. Сохраните. Код использует имена, которые вы задали в свойствах.

ШАГ 27 ИЗ 30 · 2/4

Подключите таблицу и кнопки администратора

Назначьте события в свойствах

1. Выделите UsersGrid. Нажмите F4, затем значок молнии. В строке SelectionChanged выберите SelectUser.
2. Выделите NewButton. В событии Click выберите NewUser. Для SaveButton в событии Click выберите SaveUser.
3. Нажмите «Сборка» → «Собрать решение». Если имя не найдено, сверьте Name элемента и название обработчика.

ШАГ 27 ИЗ 30 · 3/4

Подключите таблицу и кнопки администратора

Проследите выбор между INSERT и UPDATE

1. Нажатие «Новый» вызывает NewUser и сбрасывает editingId в null. Нажатие на строку вызывает SelectUser и запоминает id именно этой записи.
2. SaveUser передаёт editingId в Accounts.Save. При null метод выполняет INSERT. При выбранном id он выполняет UPDATE с условием WHERE id = … . Сам логин можно изменить, поэтому запись для обновления выбирают по id.
3. Найдите в Accounts.Save знакомые выражения из блока «SQL-запросы». COUNT(*) проверяет повтор логина, COALESCE сохраняет старый хеш при пустом новом пароле, CASE меняет счётчик и блокировку по флажку.

ШАГ 27 ИЗ 30 · 4/4

Подключите таблицу и кнопки администратора

Проследите выбор между INSERT и UPDATE

1 Панель показывается для роли admin. Перед записью Accounts.Save дополнительно вызывает RequireAdmin, чтобы право на изменение проверялось в коде, который обращается к базе.

ЧТО ПРОВЕРИТЬ

Решение собирается. Список и обе кнопки связаны со своими обработчиками.

ШАГ 27 · MAINWINDOW.XAML.CS · 1/7

Подключите таблицу и кнопки администратора

```
using System.Data;
using System.Data.Common;
using System.Windows;
using System.Windows.Controls;

namespace Dkip;

public partial class MainWindow : Window
{
    private readonly Account actor = null!;
    private int? editingId;

    public MainWindow()
    {
        InitializeComponent();
    }
```

ШАГ 27 · MAINWINDOW.XAML.CS · 2/7

Подключите таблицу и кнопки администратора

```
public MainWindow(Account user) : this()
{
    actor = user;
    RoleLabel.Text = user.Login + " · " + (user.Role == "admin" ? "Администратор" : "Пользователь");
    AdminPanel.Visibility = user.Role == "admin" ? Visibility.Visible : Visibility.Collapsed;
    if (user.Role == "admin")
        LoadUsers();
}

private void LoadUsers()
{
    try
    {
```

ШАГ 27 · MAINWINDOW.XAML.CS · 3/7

Подключите таблицу и кнопки администратора

```
            Accounts.RequireAdmin(actor);
            UsersGrid.ItemsSource = Database.Query(
                "SELECT id, login, role, failed_attempts, is_locked FROM users ORDER BY id").DefaultView;
        }
        catch (InvalidOperationException ex)
        {
            ShowWarning(ex.Message);
        }
        catch (DbException)
        {
            ShowDatabaseError();
        }
    }

    private void SelectUser(object sender, SelectionChangedEventArgs e)
```

ШАГ 27 · MAINWINDOW.XAML.CS · 4/7

Подключите таблицу и кнопки администратора

```
    {
        if (UsersGrid.SelectedItem is not DataRowView row)
            return;

        editingId = Convert.ToInt32(row["id"]);
        UserLogin.Text = (string)row["login"];
        UserRole.SelectedIndex = (string)row["role"] == "admin" ? 1 : 0;
        NewPassword.Clear();
        Unlock.IsChecked = false;
    }

    private void NewUser(object sender, RoutedEventArgs e)
    {
        // Без выбранного id кнопка «Сохранить» добавит новую запись.
        editingId = null;
        UsersGrid.UnselectAll();
```

ШАГ 27 · MAINWINDOW.XAML.CS · 5/7

Подключите таблицу и кнопки администратора

```
        UserLogin.Clear();
        NewPassword.Clear();
        UserRole.SelectedIndex = 0;
        Unlock.IsChecked = false;
    }

    private void SaveUser(object sender, RoutedEventArgs e)
    {
        try
        {
            string role = UserRole.SelectedIndex == 1 ? "admin" : "user";
            Accounts.Save(actor, editingId, UserLogin.Text, NewPassword.Password, role, Unlock.IsChecked == true);
            NewUser(sender, e);
            LoadUsers();
```

ШАГ 27 · MAINWINDOW.XAML.CS · 6/7

Подключите таблицу и кнопки администратора

```
            MessageBox.Show("Данные сохранены", "Информация", MessageBoxButton.OK, MessageBoxImage.Information);
        }
        catch (InvalidOperationException ex)
        {
            ShowWarning(ex.Message);
        }
        catch (DbException)
        {
            ShowDatabaseError();
        }
    }

    private void ShowWarning(string message) =>
        MessageBox.Show(message, "Проверьте данные", MessageBoxButton.OK, MessageBoxImage.Warning);
```

ШАГ 27 · MAINWINDOW.XAML.CS · 7/7

Подключите таблицу и кнопки администратора

```
    private void ShowDatabaseError() =>
        MessageBox.Show("Проверьте подключение к базе данных и повторите действие.",
            "Ошибка базы данных", MessageBoxButton.OK, MessageBoxImage.Error);
}
```

ШАГ 28 ИЗ 30 · 1/2

Откройте рабочее окно после входа

Обработчик Click считывает поля, вызывает Accounts.SignIn и открывает окно при успехе.

1. Откройте конструктор окна «Вход». Дважды щёлкните «Войти» и выделите весь код между фигурными скобками метода Click.
2. Вставьте код из блока ниже. Название метода и скобки оставьте. В начало файла добавьте строку using System.Data.Common; если её ещё нет.
3. Нажмите Ctrl+S и Ctrl+F5. Войдите под admin и соберите пазл. После сообщения об успехе проверьте рабочее окно.

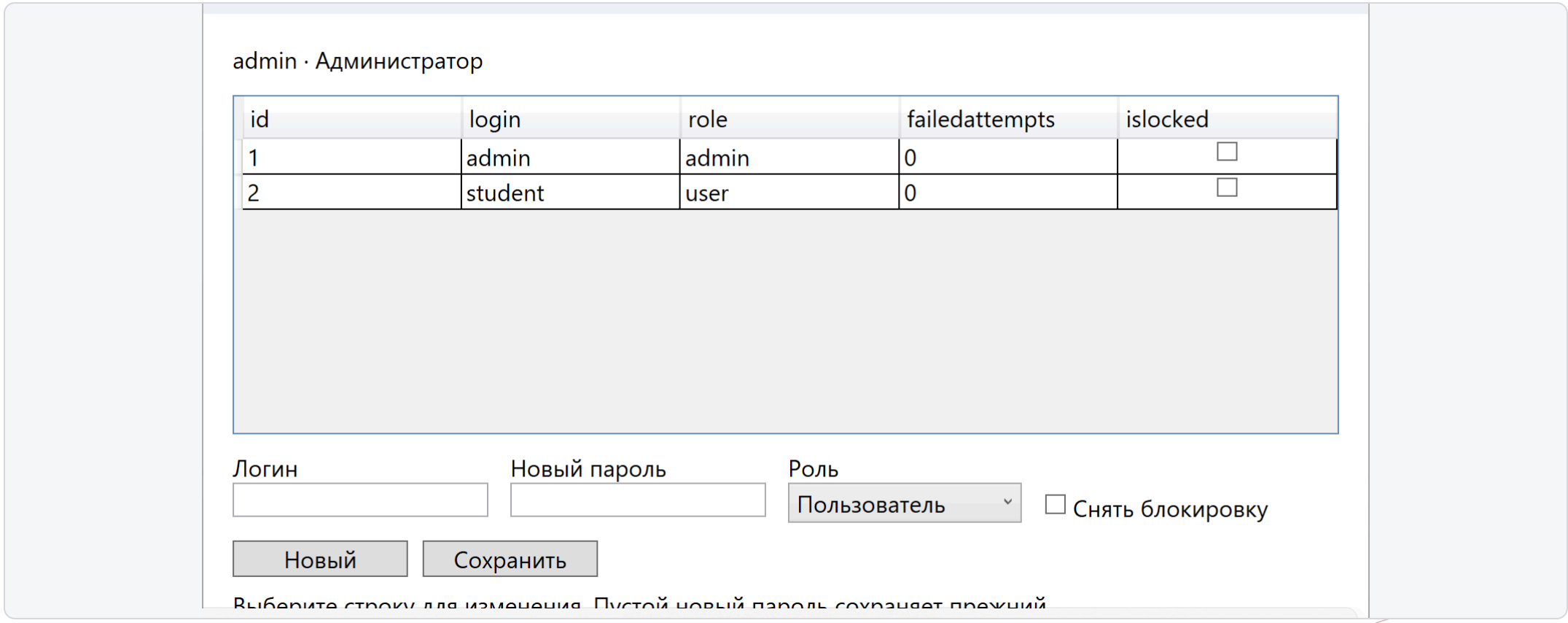
ЧТО ПРОВЕРИТЬ

Под **admin** открывается таблица пользователей. Для обычной записи видно только имя и роль.

ШАГ 28 ИЗ 30 · 2/2

Откройте рабочее окно после входа

Покажите на экране



WPF. Рабочее окно **admin**: таблица учётных записей, поля редактирования и кнопки «Новый» и «Сохранить».

ШАГ 28 · ГОТОВЫЙ КОД ВНУТРИ CLICK КНОПКИ «ВОЙТИ» · 1/2

Откройте рабочее окно после входа

```
try
{
    // Проверяем всё в одном месте. При ошибке рабочее окно не откроется.
    var user = Accounts.SignIn(LoginInput.Text, PasswordInput.Password, IsSolved());
    MessageBox.Show("Вы успешно авторизовались", "Информация", MessageBoxButton.OK, MessageBoxImage.Information);

    var main = new MainWindow(user) { Owner = this };
    main.ShowDialog();
    PasswordInput.Clear();
    Shuffle();
}
catch (InvalidOperationException ex)
{
    MessageBox.Show(ex.Message, "Ошибка входа", MessageBoxButton.OK, MessageBoxImage.Warning);
    Shuffle();
```

ШАГ 28 · ГОТОВЫЙ КОД ВНУТРИ CLICK КНОПКИ «ВОЙТИ» · 2/2

Откройте рабочее окно после входа

```
}
catch (DbException)
{
    MessageBox.Show("Проверьте запуск PostgreSQL и настройки DKIP_CONNECTION.",
        "Ошибка подключения", MessageBoxButton.OK, MessageBoxImage.Error);
}
```

ШАГ 28 · LOGINWINDOW.XAML.CS · 1/8

Откройте рабочее окно после входа

```
using System.Data.Common;
using System.IO;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Media.Imaging;
namespace Dkip;

public partial class LoginWindow : Window
{
    readonly int[] order = [1, 0, 3, 2];
    readonly BitmapSource[] pieces = new BitmapSource[4];
    readonly Button[] tiles = new Button[4];
    int selected = -1;
    public LoginWindow()
    {
```



ШАГ 28 · LOGINWINDOW.XAML.CS · 2/8

Откройте рабочее окно после входа

```
        InitializeComponent();
        try
        {
            LoadPuzzle();
        }
        catch (Exception ex) when (ex is IOException or NotSupportedException)
        {
            SignInButton.IsEnabled = false;
            MessageBox.Show("Проверьте четыре PNG в папке Assets рядом с приложением.",
                "Не удалось открыть пазл", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }

    void LoadPuzzle()
    {
        // Четыре PNG уже идут в правильном порядке. Сначала показываем образец.
```

ШАГ 28 · LOGINWINDOW.XAML.CS · 3/8

Откройте рабочее окно после входа

```
var visual = new DrawingVisual();
using (var drawing = visual.RenderOpen())
{
    for (int i = 0; i < 4; i++)
    {
        var image = new BitmapImage();
        image.BeginInit();
        image.CacheOption = BitmapCacheOption.OnLoad;
        image.UriSource = new Uri(Path.Combine(AppContext.BaseDirectory, "Assets", (i + 1) + ".png"));
        image.EndInit();
        image.Freeze();
        pieces[i] = image;
        drawing.DrawImage(image, new Rect(i % 2 * 200, i / 2 * 200, 200, 200));
    }
}
```

ШАГ 28 · LOGINWINDOW.XAML.CS · 4/8

Откройте рабочее окно после входа

```
        var preview = new RenderTargetBitmap(400, 400, 96, 96, PixelFormats.Pbgra32);
        preview.Render(visual);
        preview.Freeze();
        Original.Source = preview;
        for (int i = 0; i < 4; i++)
        {
            tiles[i] = new Button { Padding = new Thickness(0), Margin = new Thickness(1) };
            System.Windows.Automation.AutomationProperties.SetName(tiles[i], "Фрагмент " + (i + 1));
            tiles[i].Click += SelectTile;
            Puzzle.Children.Add(tiles[i]);
        }
        RenderPuzzle();
    }
    void SelectTile(object sender, RoutedEventArgs e)
    {
```

ШАГ 28 · LOGINWINDOW.XAML.CS · 5/8

Откройте рабочее окно после входа

```
        int position = Array.IndexOf(tiles, (Button)sender);
        if (selected == -1)
        {
            selected = position;
        }
        else
        {
            // Второе нажатие меняет местами два номера в массиве.
            int fragment = order[selected];
            order[selected] = order[position];
            order[position] = fragment;
            selected = -1;
        }
        RenderPuzzle();
    }
```



ШАГ 28 · LOGINWINDOW.XAML.CS · 6/8

Откройте рабочее окно после входа

```
    bool IsSolved() => order.SequenceEqual(new[] { 0, 1, 2, 3 });

    void RenderPuzzle()
    {
        for (int i = 0; i < 4; i++)
        {
            tiles[i].Content = new Image { Source = pieces[order[i]], Stretch = Stretch.Fill };
            tiles[i].BorderBrush = selected == i ? Brushes.Red : Brushes.Gray;
            tiles[i].BorderThickness = new Thickness(selected == i ? 3 : 1);
        }
    }
    void Shuffle()
    {
        do
        {
            Random.Shared.Shuffle(order);
```

ШАГ 28 · LOGINWINDOW.XAML.CS · 7/8

Откройте рабочее окно после входа

```
        } while (IsSolved());
        selected = -1;
        RenderPuzzle();
    }
    void SignIn(object sender, RoutedEventArgs e)
    {
        try
        {
            // Проверяем всё в одном месте. При ошибке рабочее окно не откроется.
            var user = Accounts.SignIn(LoginInput.Text, PasswordInput.Password, IsSolved());
            MessageBox.Show("Вы успешно авторизовались", "Информация", MessageBoxButton.OK,
MessageBoxImage.Information);

            var main = new MainWindow(user) { Owner = this };
            main.ShowDialog();
            PasswordInput.Clear();
```

ШАГ 28 · LOGINWINDOW.XAML.CS · 8/8

Откройте рабочее окно после входа

```
            Shuffle();
        }
        catch (InvalidOperationException ex)
        {
            MessageBox.Show(ex.Message, "Ошибка входа", MessageBoxButton.OK, MessageBoxImage.Warning);
            Shuffle();
        }
        catch (DbException)
        {
            MessageBox.Show("Проверьте запуск PostgreSQL и настройки DKIP_CONNECTION.",
                "Ошибка подключения", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
}
```

ШАГ 29 ИЗ 30 · 1/2

Проверьте добавление, изменение и разблокировку

Одна кнопка сохраняет оба вида изменений. Если запись не выбрана, добавляем новую. Если выбрана, меняем запись по id.

1. Под **admin** нажмите «Новый». Введите **test_student**, учебный пароль и роль «Пользователь». Нажмите «Сохранить» и найдите строку.
2. Снова нажмите «Новый» и повторите логин. Проверьте сообщение о дубликате.
3. Выберите уже созданного **test_student**, запишите его **id** и измените логин на **test_student_edit**. Сохраните с пустым полем нового пароля. Сверьте прежний **id** и убедитесь, что в списке по-прежнему одна запись для этого пользователя.

ШАГ 29 ИЗ 30 · 2/2

Проверьте добавление, изменение и разблокировку

1 Выберите строку **student**, включите «Снять блокировку» и сохраните. Оставьте новый пароль пустым, чтобы сохранить старый. Закройте рабочее окно и войдите под **student**.

ЧТО ПРОВЕРИТЬ

Добавление создало новый **id**. Редактирование сохранило этот **id**. Дубликат отклонён, а **student** после разблокировки входит со старым паролем.

ШАГ 30 ИЗ 30 · 1/2

Пройдите короткую проверку перед сдачей

Сценарий проверяет все развилки задания и помогает воспроизвести их по памяти.

1. Войдите как **student** с правильным паролем и пазлом. Убедитесь, что в рабочем окне нет управления пользователями.
2. Проверьте три ошибки подряд на тестовой записи, затем войдите администратором и снимите блокировку. Попробуйте создать дубликат логина.
3. Запустите тестовый профиль с неверным портом БД. Проверьте сообщение об ошибке и верните исходную настройку.

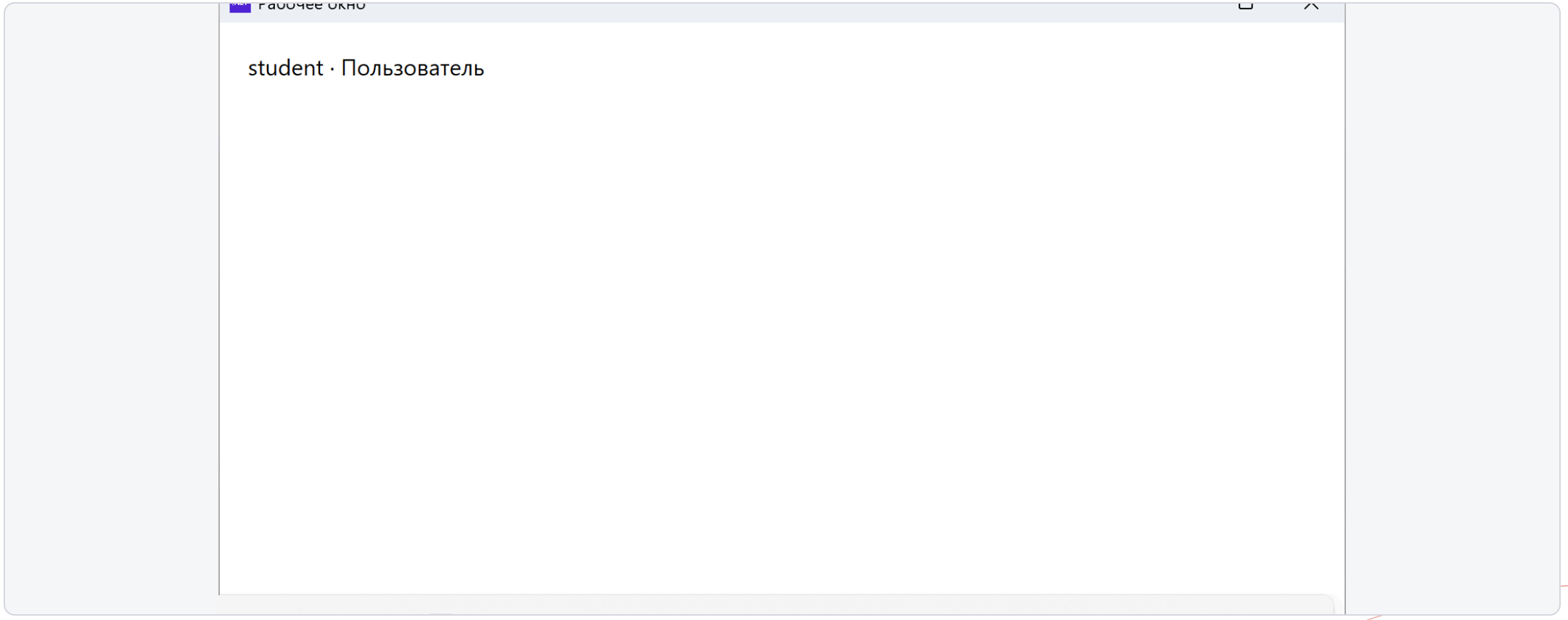
ЧТО ПРОВЕРИТЬ

Приложение выполняет сценарии входа, блокировки, прав и сохранения. При недоступной базе остаётся открытым.

ШАГ 30 ИЗ 30 · 2/2

Пройдите короткую проверку перед сдачей

Покажите на экране



WPF. Рабочее окно **student**: видны логин и роль «Пользователь».

ШАГ 30 · LOGINWINDOW.XAML.CS · 1/8

Пройдите короткую проверку перед сдачей

```
using System.Data.Common;
using System.IO;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Media;
using System.Windows.Media.Imaging;
namespace Dkip;

public partial class LoginWindow : Window
{
    readonly int[] order = [1, 0, 3, 2];
    readonly BitmapSource[] pieces = new BitmapSource[4];
    readonly Button[] tiles = new Button[4];
    int selected = -1;
    public LoginWindow()
    {
```

ШАГ 30 · LOGINWINDOW.XAML.CS · 2/8

Пройдите короткую проверку перед сдачей

```
        InitializeComponent();
        try
        {
            LoadPuzzle();
        }
        catch (Exception ex) when (ex is IOException or NotSupportedException)
        {
            SignInButton.IsEnabled = false;
            MessageBox.Show("Проверьте четыре PNG в папке Assets рядом с приложением.",
                "Не удалось открыть пазл", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }

    void LoadPuzzle()
    {
        // Четыре PNG уже идут в правильном порядке. Сначала показываем образец.
```

ШАГ 30 · LOGINWINDOW.XAML.CS · 3/8

Пройдите короткую проверку перед сдачей

```
        var visual = new DrawingVisual();
        using (var drawing = visual.RenderOpen())
        {
            for (int i = 0; i < 4; i++)
            {
                var image = new BitmapImage();
                image.BeginInit();
                image.CacheOption = BitmapCacheOption.OnLoad;
                image.UriSource = new Uri(Path.Combine(AppContext.BaseDirectory, "Assets", (i + 1) + ".png"));
                image.EndInit();
                image.Freeze();
                pieces[i] = image;
                drawing.DrawImage(image, new Rect(i % 2 * 200, i / 2 * 200, 200, 200));
            }
        }
```

ШАГ 30 · LOGINWINDOW.XAML.CS · 4/8

Пройдите короткую проверку перед сдачей

```
        var preview = new RenderTargetBitmap(400, 400, 96, 96, PixelFormats.Pbgra32);
        preview.Render(visual);
        preview.Freeze();
        Original.Source = preview;
        for (int i = 0; i < 4; i++)
        {
            tiles[i] = new Button { Padding = new Thickness(0), Margin = new Thickness(1) };
            System.Windows.Automation.AutomationProperties.SetName(tiles[i], "Фрагмент " + (i + 1));
            tiles[i].Click += SelectTile;
            Puzzle.Children.Add(tiles[i]);
        }
        RenderPuzzle();
    }
    void SelectTile(object sender, RoutedEventArgs e)
    {
```

ШАГ 30 · LOGINWINDOW.XAML.CS · 5/8

Пройдите короткую проверку перед сдачей

```
        int position = Array.IndexOf(tiles, (Button)sender);
        if (selected == -1)
        {
            selected = position;
        }
        else
        {
            // Второе нажатие меняет местами два номера в массиве.
            int fragment = order[selected];
            order[selected] = order[position];
            order[position] = fragment;
            selected = -1;
        }
        RenderPuzzle();
    }
```



ШАГ 30 · LOGINWINDOW.XAML.CS · 6/8

Пройдите короткую проверку перед сдачей

```
bool IsSolved() => order.SequenceEqual(new[] { 0, 1, 2, 3 });

void RenderPuzzle()
{
    for (int i = 0; i < 4; i++)
    {
        tiles[i].Content = new Image { Source = pieces[order[i]], Stretch = Stretch.Fill };
        tiles[i].BorderBrush = selected == i ? Brushes.Red : Brushes.Gray;
        tiles[i].BorderThickness = new Thickness(selected == i ? 3 : 1);
    }
}
void Shuffle()
{
    do
    {
        Random.Shared.Shuffle(order);
```

ШАГ 30 · LOGINWINDOW.XAML.CS · 7/8

Пройдите короткую проверку перед сдачей

```
        } while (IsSolved());
        selected = -1;
        RenderPuzzle();
    }
    void SignIn(object sender, RoutedEventArgs e)
    {
        try
        {
            // Проверяем всё в одном месте. При ошибке рабочее окно не откроется.
            var user = Accounts.SignIn(LoginInput.Text, PasswordInput.Password, IsSolved());
            MessageBox.Show("Вы успешно авторизовались", "Информация", MessageBoxButton.OK,
MessageBoxImage.Information);

            var main = new MainWindow(user) { Owner = this };
            main.ShowDialog();
            PasswordInput.Clear();
```

ШАГ 30 · LOGINWINDOW.XAML.CS · 8/8

Пройдите короткую проверку перед сдачей

```
            Shuffle();
        }
        catch (InvalidOperationException ex)
        {
            MessageBox.Show(ex.Message, "Ошибка входа", MessageBoxButton.OK, MessageBoxImage.Warning);
            Shuffle();
        }
        catch (DbException)
        {
            MessageBox.Show("Проверьте запуск PostgreSQL и настройки DKIP_CONNECTION.",
                "Ошибка подключения", MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
}
```

РЕЗУЛЬТАТ ЭТАПА

Что должно остаться после этапа

Есть проект MyApp с исходниками и снимки входа, администрирования и проверки блокировки.

ЧТО ПРОВЕРИТЬ

Проект **WPF**, работающий с **PostgreSQL**.

ЗАДАНИЕ 5 · 60 МИН

API и проверки

Создадим отдельный HTTP-сервис на ASP.NET Core Minimal APIs. Он читает ту же базу, что и настольное приложение.

ЧТО ПРОВЕРИТЬ

Сохранены проект **Api**, коллекция **Postman** и результаты тестов. Сервис возвращает поля и формат из приложения к КИМ.

УСЛОВИЕ ИЗ КИМ

Интеграция программных модулей

Разработайте API, которое при обращении возвращает данные из базы данных. Данные не должны выдаваться в исходном виде — перед отправкой клиенту они обязательно проходят обработку в соответствии с требованиями, указанными в Приложении 2.

С использованием предоставленного средства автоматизации тестирования API разработайте коллекцию запросов. Внутри коллекции должны быть реализованы тесты, подтверждающие работоспособность решения.

Приложение: Прил_5_О3_КИМ_09.02.07-5-2027.pdf

КИМ · стр. 31–32; Приложение 2 Исходные файлы · ZIP

ШАГ 1 ИЗ 24 · 1/2

Проверьте таблицу заметок из задания 2

notes хранит исходные данные. Связь с **users** даёт логин автора без копирования его в каждую заметку.

1. Откройте свою учебную базу из заданий 2–4. Найдите таблицу **notes** и откройте её структуру. Если таблицы нет, вернитесь к карточке «Создайте остальные таблицы и связи» задания 2 и добавьте недостающую таблицу по разбору.
2. Проверьте **id**, **title**, **content**, **id_user** и **created_at**. Убедитесь, что **id_user** ссылается на **users.id**. Заголовок с логином и дату для ответа сформирует приложение на следующих карточках.

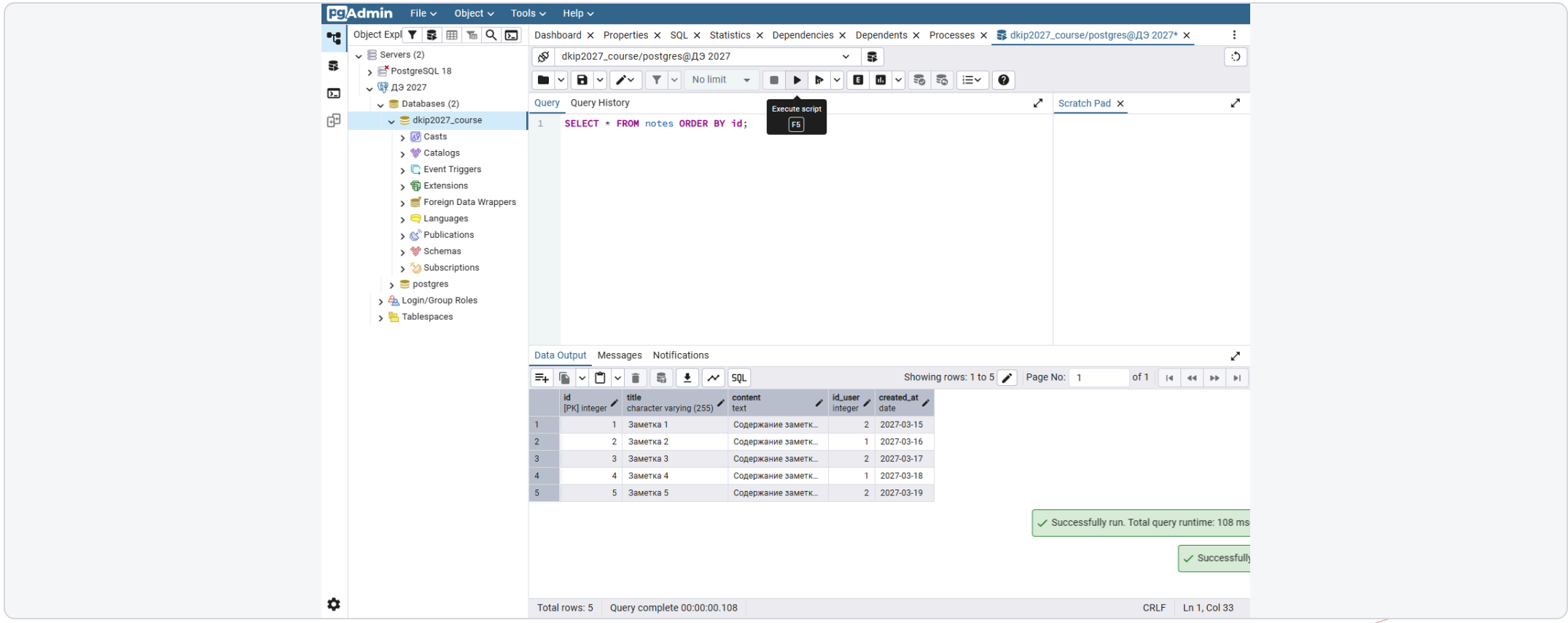
ЧТО ПРОВЕРИТЬ

В **notes** есть пять полей, **created_at** имеет тип **DATE**, а **id_user** ссылается на **users.id**.

ШАГ 1 ИЗ 24 · 2/2

Проверьте таблицу заметок из задания 2

Покажите на экране



Пять исходных записей **notes** в редакторе СУБД. Маршрут: **WPF / PostgreSQL**.

ШАГ 2 ИЗ 24 · 1/3

Добавьте тестовые заметки

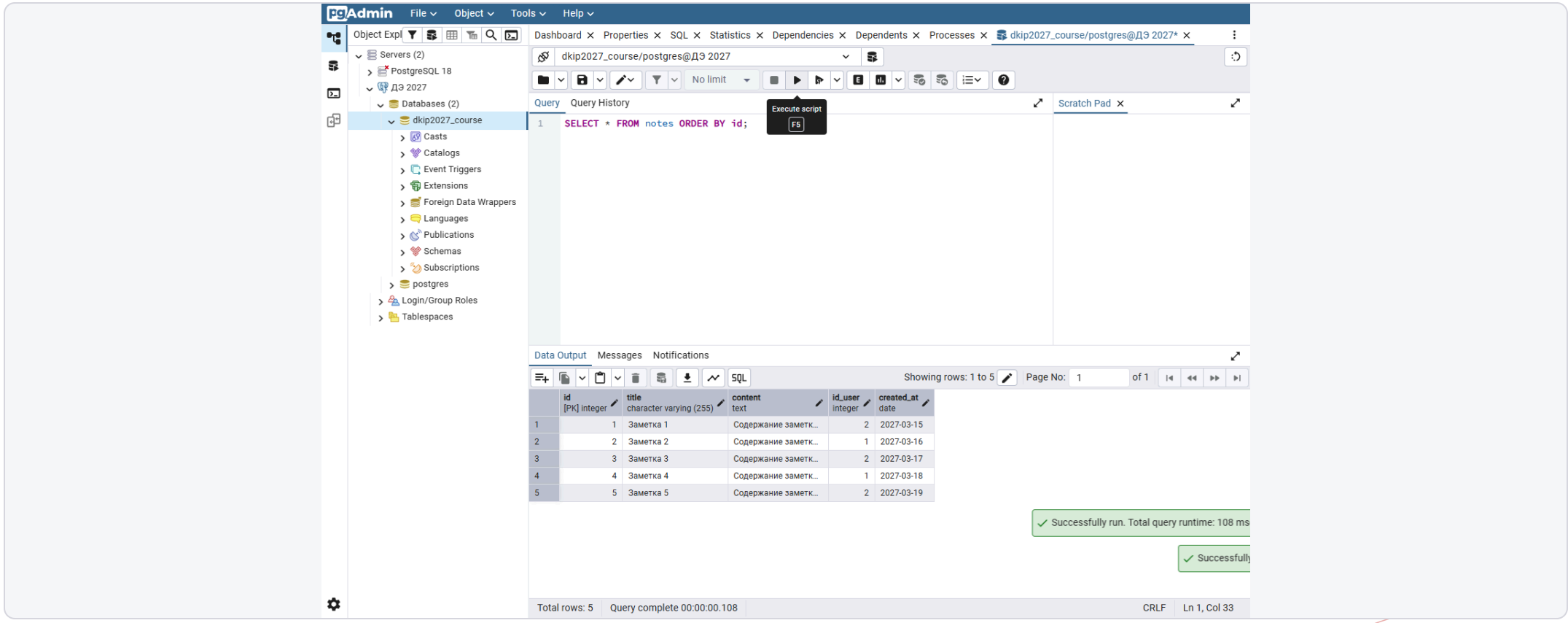
Контрольную строку из базы можно проследить до ответа API и затем проверить в Postman.

1. В pgAdmin найдите пользователей **student** и **admin** из задания 4. Если в **notes** уже есть 3–5 подходящих записей, выберите контрольную строку и используйте её данные дальше.
2. Для пустой **notes** выполните первый блок один раз. **INSERT** берёт **id** автора по логину. Так учебные заметки привязываются к реальному пользователю, даже если ему присвоен другой **id**. Если часть заметок уже внесли, добавьте только недостающие.

ШАГ 2 ИЗ 24 · 2/3

Добавьте тестовые заметки

Покажите на экране



Пять исходных записей **notes** в редакторе СУБД. Маршрут: **WPF / PostgreSQL**.

ШАГ 2 ИЗ 24 · 3/3

Добавьте тестовые заметки

1 Выполните второй блок. Для строки «Заметка 1» запишите присвоенный id, id_user, title, content и created_at. Для примера ниже дата исходной строки — 2027-03-15.

ЧТО ПРОВЕРИТЬ

В **notes** есть **3–5** записей. Контрольная заметка ссылается на существующего автора, её **id** и исходные значения записаны.

ШАГ 2 · 1. ТЕСТОВЫЕ ЗАМЕТКИ ДЛЯ ПУСТОЙ ТАБЛИЦЫ

Добавьте тестовые заметки

```
INSERT INTO notes (title, content, id_user, created_at)
SELECT 'Заметка 1', 'Содержание заметки 1', id, CAST('2027-03-15' AS DATE)
FROM users WHERE login = 'student';
INSERT INTO notes (title, content, id_user, created_at)
SELECT 'Заметка 2', 'Содержание заметки 2', id, CAST('2027-03-16' AS DATE)
FROM users WHERE login = 'student';
INSERT INTO notes (title, content, id_user, created_at)
SELECT 'Заметка 3', 'Содержание заметки 3', id, CAST('2027-03-17' AS DATE)
FROM users WHERE login = 'admin';
```

ШАГ 2 · 2. СВЕРКА ИСХОДНЫХ СТРОК

Добавьте тестовые заметки

```
SELECT id, title, id_user, content, created_at FROM notes ORDER BY id;
```

ШАГ 3 ИЗ 24

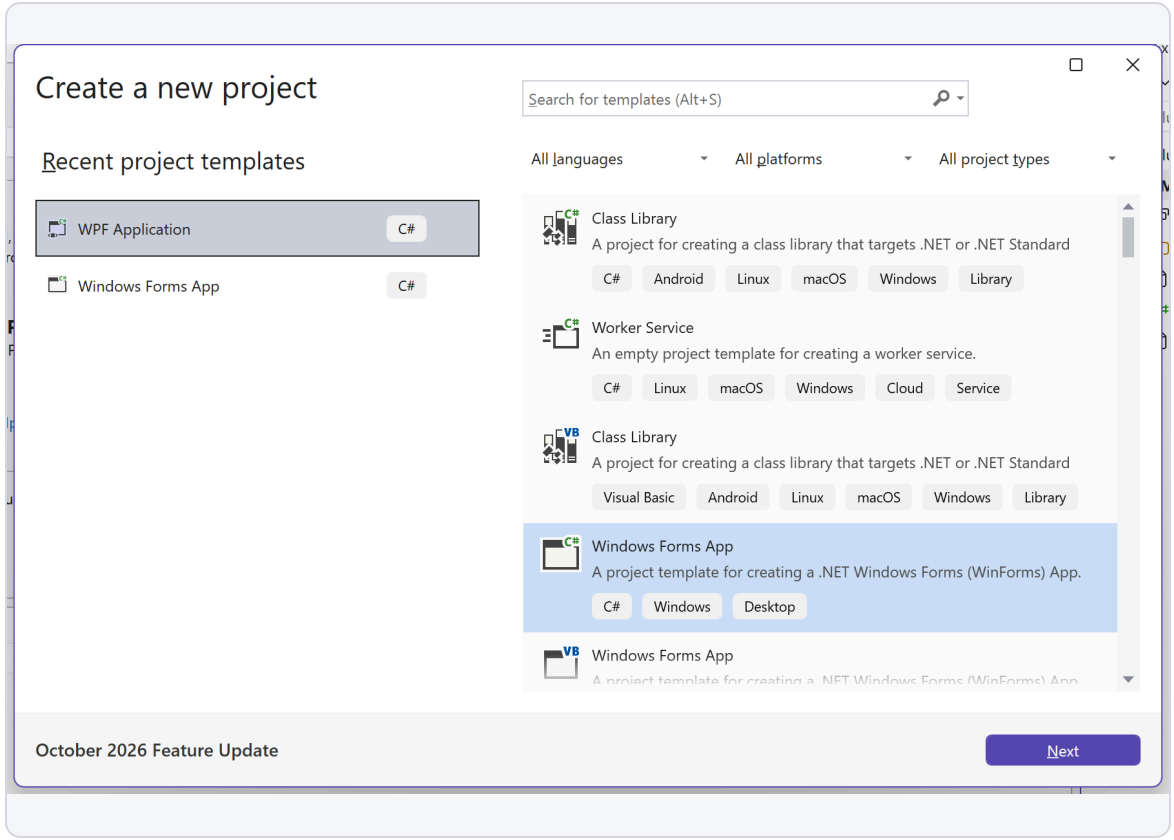
Откройте создание проекта Api

Проект объединяет исходные файлы и настройки сборки. Сначала откройте мастер создания нового проекта в Visual Studio.

1. Если видно стартовое окно Visual Studio, нажмите **Create a new project**. Если проект уже открыт, выберите «Файл» → «Создать» → «Проект» (Ctrl+Shift+N). Откроется окно поиска шаблона. Вводите название шаблона в поле поиска, а не в окно кода.

ЧТО ПРОВЕРИТЬ

Открылось окно **Create a new project** с полем поиска шаблонов.



Мастер **Create a new project** в **Visual Studio**: поле **Search for templates** и кнопка **Next**. Введите шаблон из текущего шага; список **Recent** зависит от ранее открытых проектов.

ШАГ 4 ИЗ 24

Выберите шаблон Api

Шаблон определяет тип приложения и начальные файлы. Названия современного .NET и .NET Framework похожи, но это разные платформы.

1. Найдите «Пустой проект ASP.NET Core (ASP.NET Core Empty)». Проверьте язык C#. Для Windows Forms/WPF выбирайте современный .NET, без надписи .NET Framework. Нажмите «Далее». Если шаблона нет, остановитесь и проверьте компоненты Visual Studio с преподавателем.

ЧТО ПРОВЕРИТЬ

Выбран нужный шаблон C# современного .NET; открыта настройка имени проекта.

ШАГ 5 ИЗ 24

Задайте имя и папку Api

Имя отличает проект в Обозревателе решений, расположение определяет папку исходников на диске, платформа — способ сборки.

1. В поле «Имя проекта» введите Api. В «Расположение» нажмите кнопку обзора и выберите свою рабочую папку на рабочем столе Windows. Нажмите «Далее». Выберите .NET 10 → «Создать». Дождитесь Обозревателя решений с проектом Api.

ЧТО ПРОВЕРИТЬ

В Обозревателе решений появился проект с заданным именем и выбранной платформой .NET 10.

ШАГ 6 ИЗ 24

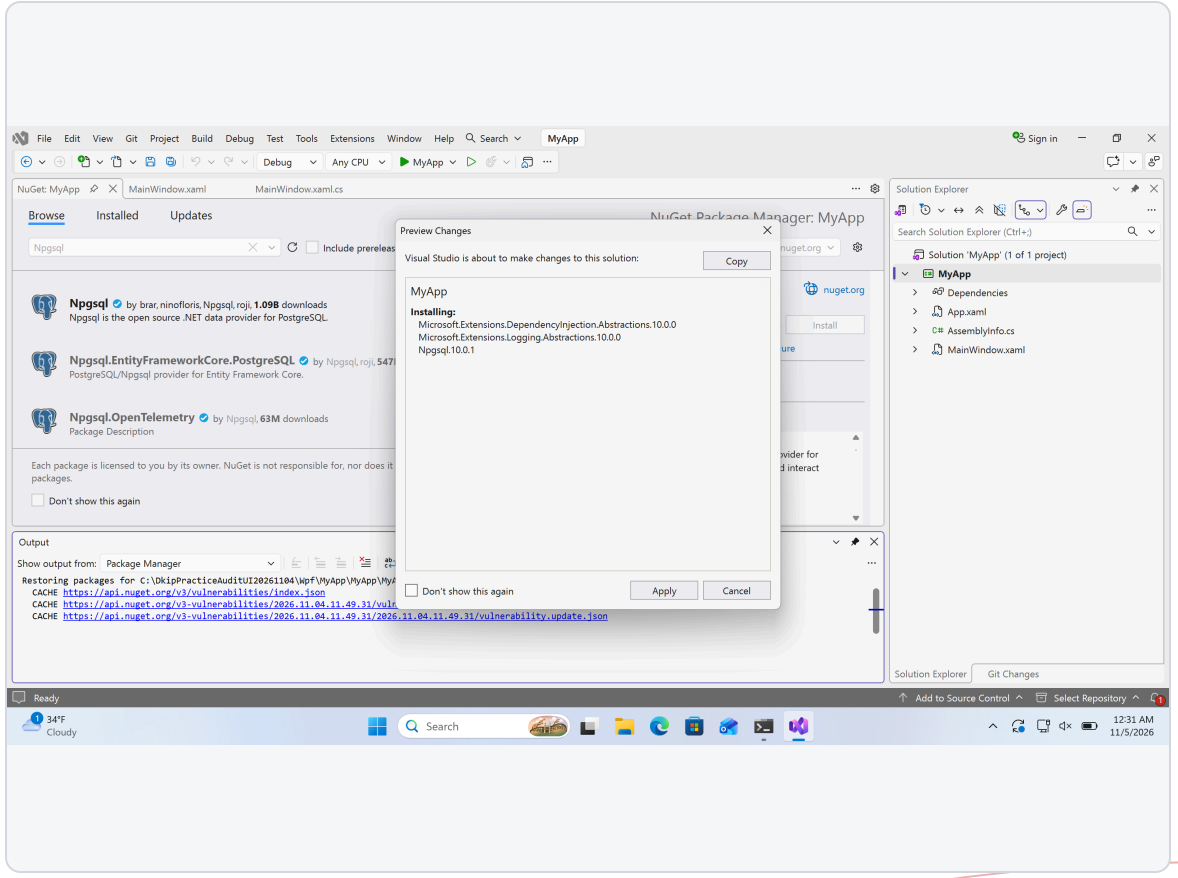
Откройте пакеты Api

Провайдер — библиотека, через которую C# подключается к выбранной СУБД. NuGet устанавливает её зависимость в конкретный проект.

1. В Обозревателе решений нажмите правой кнопкой именно по проекту Api → «Управление пакетами NuGet». Перейдите на вкладку «Обзор» (Browse). Введите Npgsql в поиск и нажмите Enter. Дождитесь списка результатов. Выберите пакет с точным названием Npgsql.

ЧТО ПРОВЕРИТЬ

Открыт NuGet → Browse; найден пакет провайдера с точным названием.



Окно Preview Changes для Npgsql 10.0.1. Apply подтверждает установку показанного пакета и зависимостей.

ШАГ 7 ИЗ 24

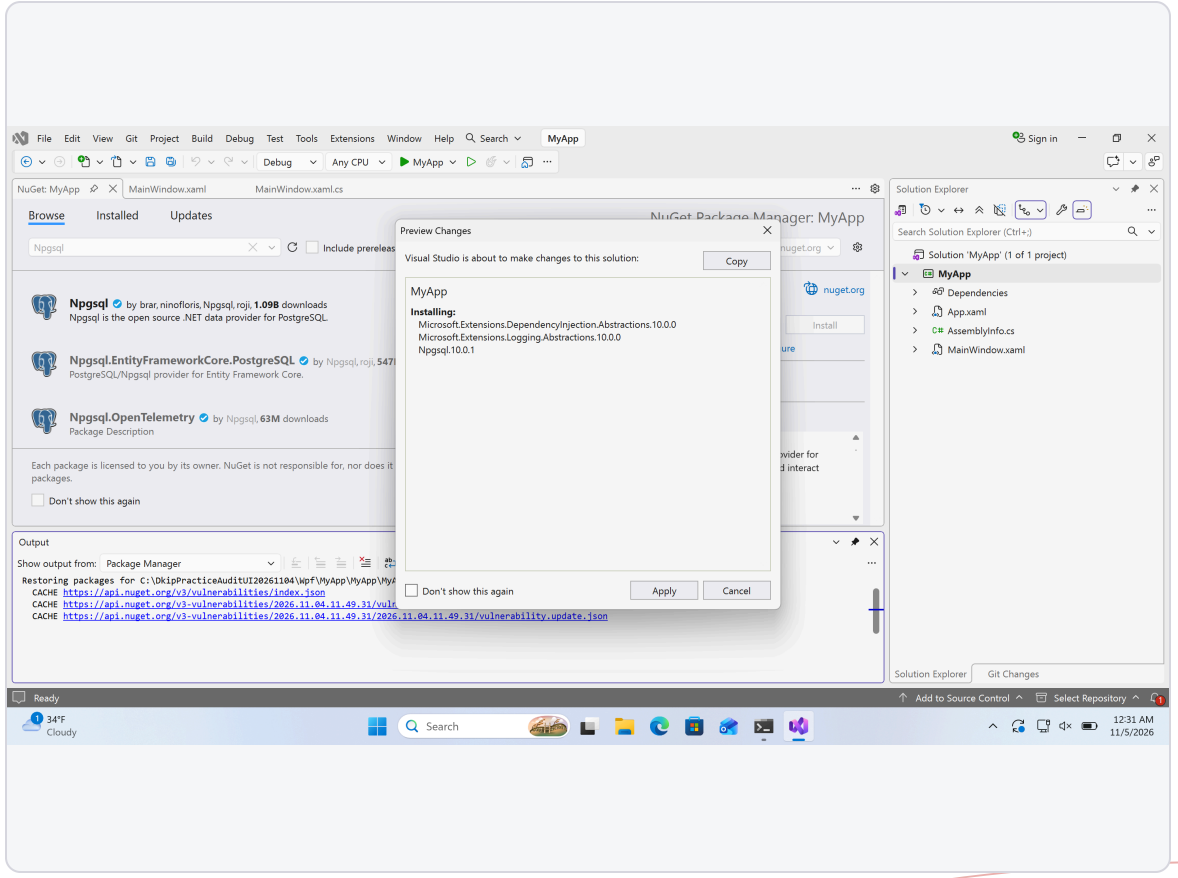
Выберите версию Api

Провайдер — библиотека, через которую C# подключается к выбранной СУБД. NuGet устанавливает её зависимость в конкретный проект.

1. В правой панели NuGet найдите Version. Нажмите стрелку списка и выберите готовый пункт 10.0.1. Выбор из списка связывает версию с пакетом; простое изменение текста в поле может оставить кнопку Install недоступной.

ЧТО ПРОВЕРИТЬ

В Version выбрана нужная версия; кнопка Install доступна.



Окно Preview Changes для Npgsql 10.0.1. Apply подтверждает установку показанного пакета и зависимостей.

ШАГ 8 ИЗ 24

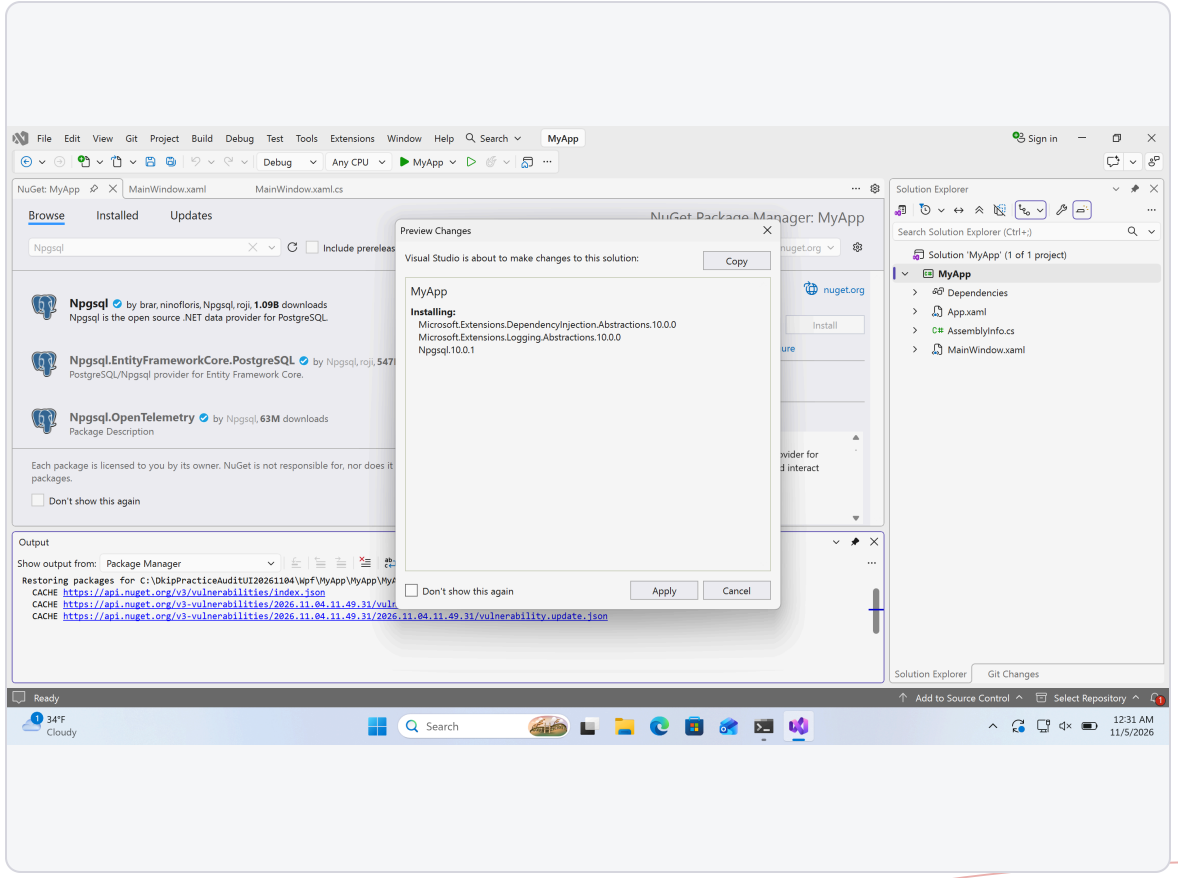
Установите провайдер Api

Провайдер — библиотека, через которую C# подключается к выбранной СУБД. NuGet устанавливает её зависимость в конкретный проект.

1. Нажмите Install («Установить»). Если появилось окно Preview Changes («Просмотр изменений»), сверяйте название пакета и версию, затем нажмите Apply («Применить»). Дождитесь окончания установки. Не закрывайте Visual Studio во время загрузки.

ЧТО ПРОВЕРИТЬ

Установка завершилась; в карточке пакета показана установленная версия.



Окно **Preview Changes** для **Npgsql 10.0.1**. **Apply** подтверждает установку показанного пакета и зависимостей.

ШАГ 9 ИЗ 24

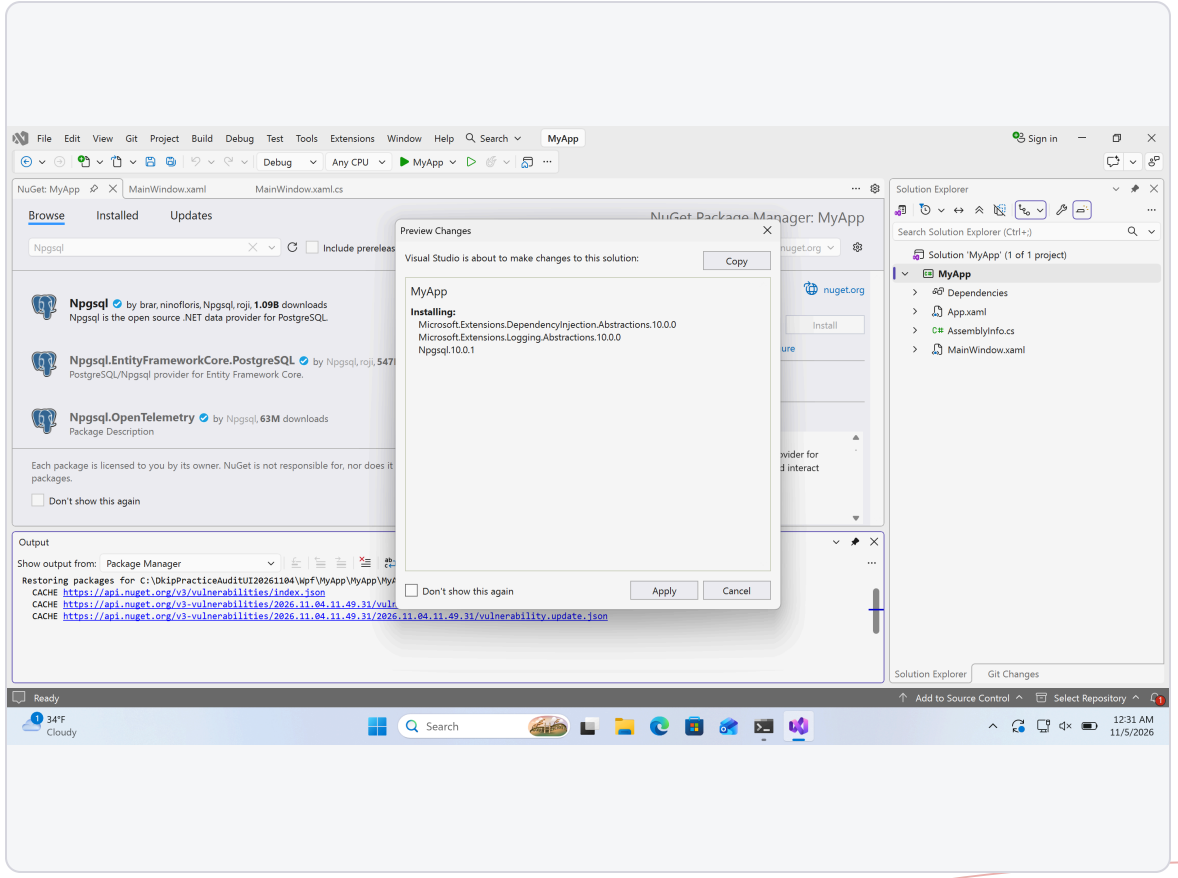
Проверьте провайдер Api

Провайдер — библиотека, через которую C# подключается к выбранной СУБД. NuGet устанавливает её зависимость в конкретный проект.

1. В Обозревателе решений раскройте **Dependencies** («Зависимости») → **Packages** («Пакеты»). Должен появиться **Npgsql** выбранной версии. Нажмите **Build** («Сборка») → **Build Solution** («Собрать решение»). В нижнем окне не должно быть ошибок.

ЧТО ПРОВЕРИТЬ

Провайдер появился в **Dependencies** → **Packages**; сборка решения завершилась без ошибок.



Окно **Preview Changes** для **Npgsql 10.0.1**. **Apply** подтверждает установку показанного пакета и зависимостей.

ШАГ 10 ИЗ 24

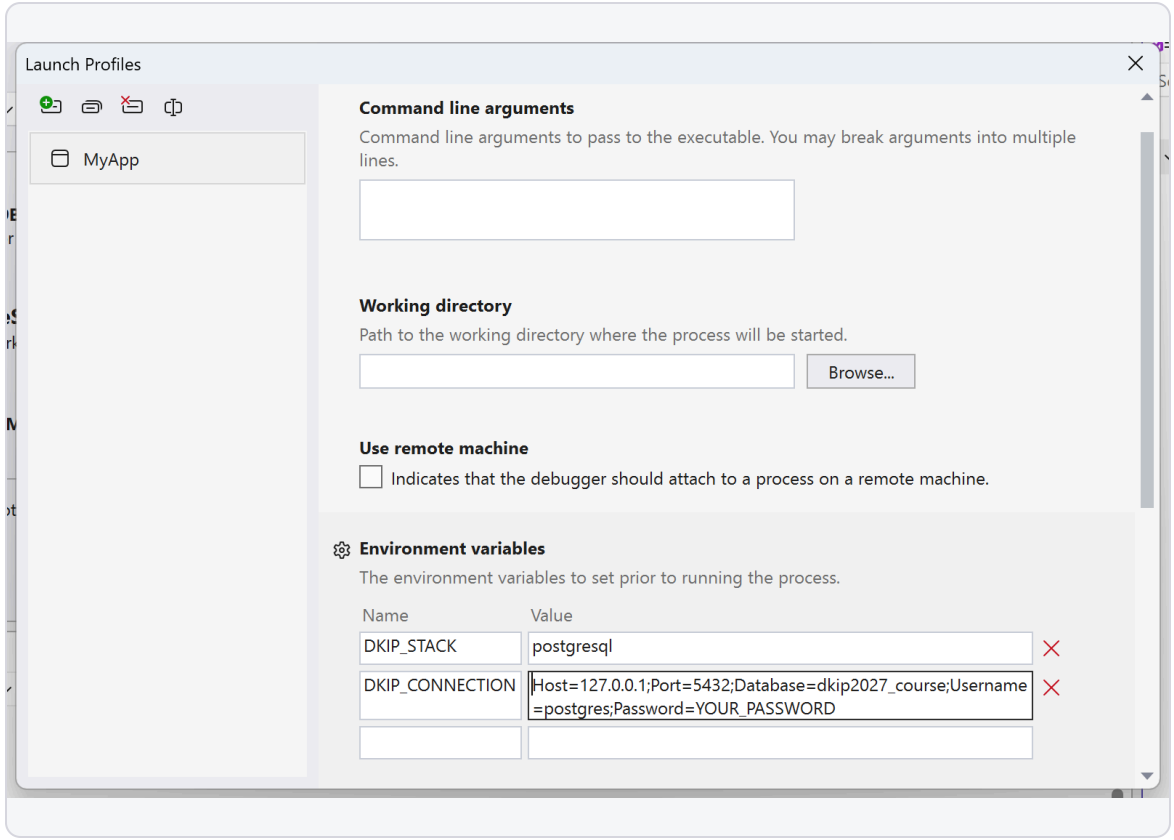
Откройте профиль подключения Api

Профиль запуска передаёт настройки программе. Адрес сервера и пароль БД отличаются от логина и пароля пользователя приложения.

1. Выделите проект Api в Обозревателе решений. Выберите меню «Отладка» → «Свойства отладки Api». Другой путь: свойства проекта → «Отладка» → «Открыть профили запуска отладки». Выберите профиль с запуском самого проекта.

ЧТО ПРОВЕРИТЬ

Открыто окно профилей запуска именно нужного проекта; видна таблица **Environment variables**.



Профиль запуска Visual Studio: DKIP_STACK=postgresql и DKIP_CONNECTION в Environment variables. YOUR_PASSWORD — заглушка, замените её паролем своего сервера.

ШАГ 11 ИЗ 24 · 1/2

Введите параметры подключения Api

Профиль запуска передаёт настройки программе. Адрес сервера и пароль БД отличаются от логина и пароля пользователя приложения.

1. Прокрутите окно профиля до **Environment variables**. В первой пустой строке нажмите поле **Name**, введите **DKIP_CONNECTION** и нажмите **Tab**. Скопируйте строку ниже в поле **Value**. Замените имя базы, порт и учётные данные значениями своей учебной установки. Подставьте пароль пользователя **PostgreSQL** вместо ВАШ_ПАРОЛЬ.

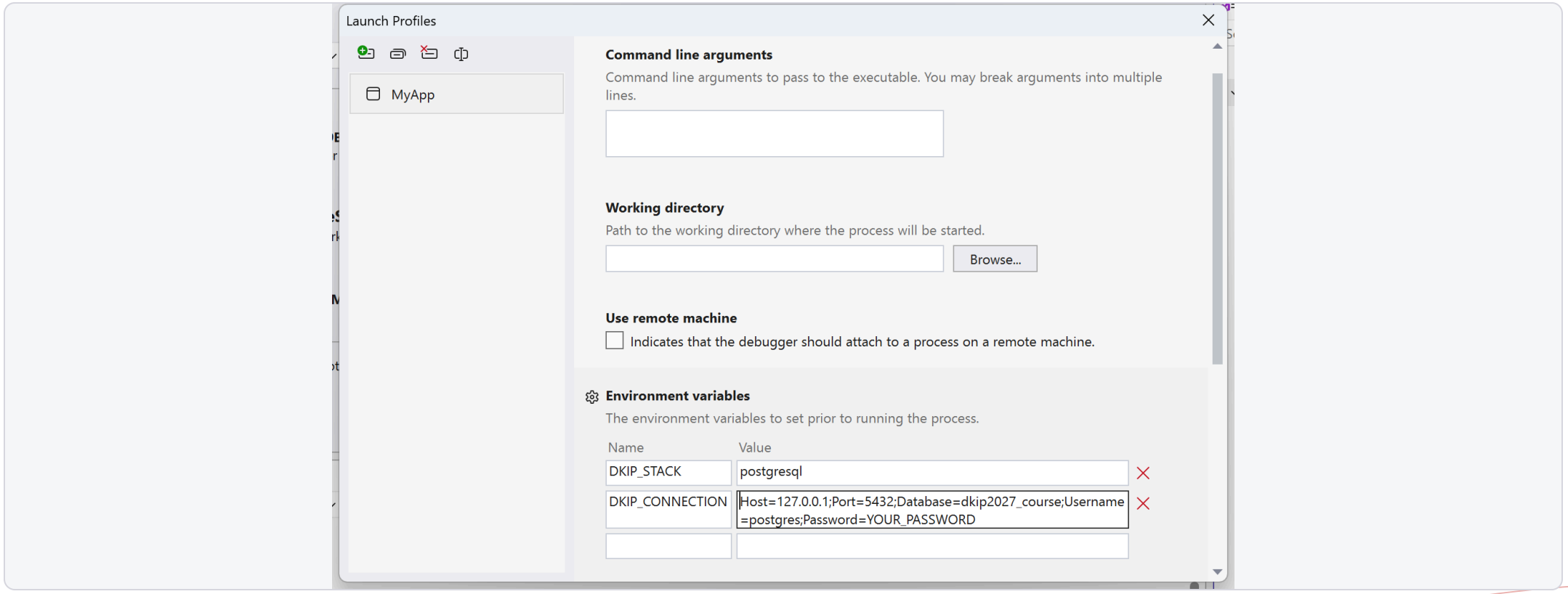
ЧТО ПРОВЕРИТЬ

В **DKIP_CONNECTION** указаны параметры своей учебной базы, без незаменённых заглушек.

ШАГ 11 ИЗ 24 · 2/2

Введите параметры подключения Api

Покажите на экране



Профиль запуска **Visual Studio**: DKIP_STACK=postgresql и DKIP_CONNECTION в **Environment variables**. YOUR_PASSWORD — заглушка, замените её паролем своего сервера.

ШАГ 11 · КОД ДЛЯ ЭТОГО ШАГА

Введите параметры подключения Api

```
Host=127.0.0.1;Port=5432;Database=dkip2027_course;Username=postgres;Password=ВАШ_ПАРОЛЬ
```

ШАГ 12 ИЗ 24 · 1/2

Задайте маршрут и сохраните Api

Профиль запуска передаёт настройки программе. Адрес сервера и пароль БД отличаются от логина и пароля пользователя приложения.

1. После ввода **Value** нажмите **Tab**. В появившейся пустой строке **Name** введите **DKIP_STACK**; в **Value** — **postgresql**. Нажмите **Tab** и закройте окно **Launch Profiles** крестиком. Значения сохраняются автоматически, отдельной кнопки **Save** нет. Профиль действует только для этого проекта. Если запускаете другой проект, настройте его отдельно. Файл **Properties/launchSettings.json** с настоящим паролем не отправляйте в **Git**; добавьте его в **.gitignore**.

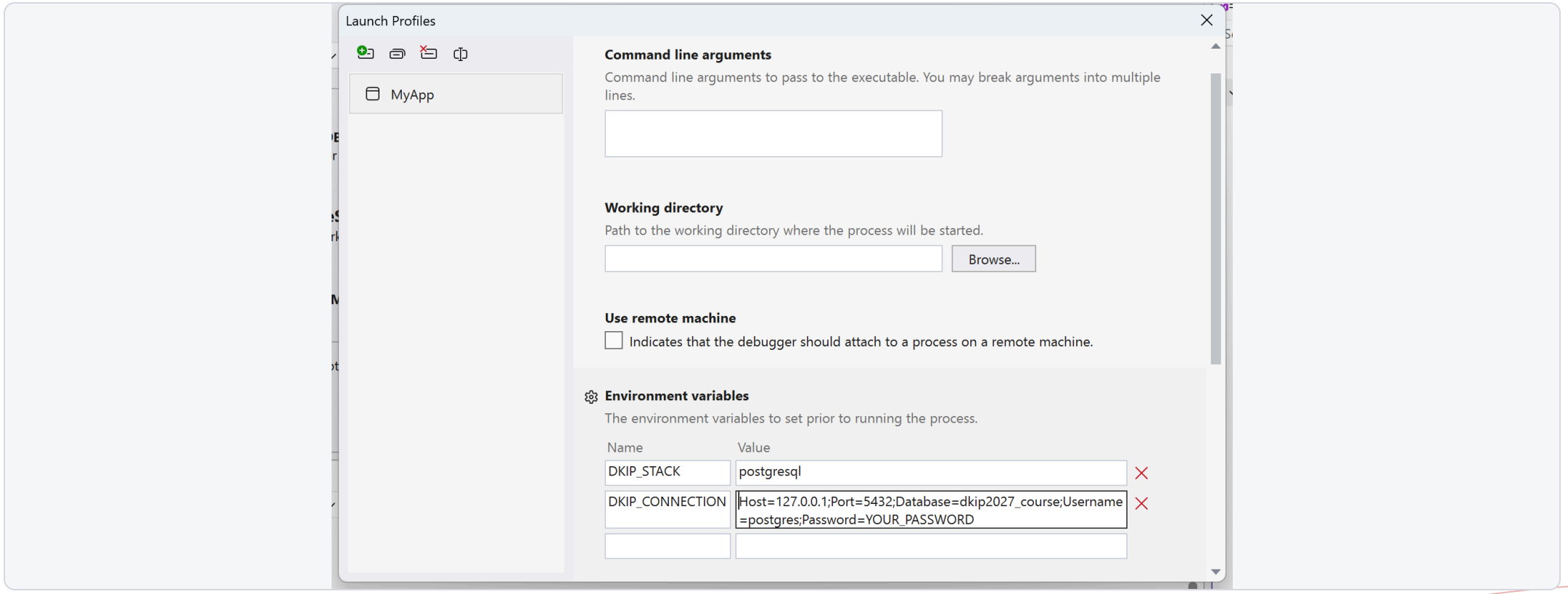
ЧТО ПРОВЕРИТЬ

В профиле сохранён **DKIP_STACK** выбранной СУБД; файл с настоящим паролем исключён из **Git**.

ШАГ 12 ИЗ 24 · 2/2

Задайте маршрут и сохраните Api

Покажите на экране



Профиль запуска **Visual Studio**: **DKIP_STACK**=**postgresql** и **DKIP_CONNECTION** в **Environment variables**. **YOUR_PASSWORD** — заглушка, замените её паролем своего сервера.

ШАГ 13 ИЗ 24

Настройте HTTP-адрес Api

HTTP-запрос идёт в отдельный работающий процесс API. Приложение с окнами, сервер БД и API — три разных процесса, которые могут останавливаться независимо.

1. Откройте «Отладка» → «Свойства отладки Api». Выберите профиль проекта **http**, а не **IIS Express**. В поле «URL-адрес приложения» (App URL) введите **http://127.0.0.1:5272**. Для своего локального учебного запуска используйте этот HTTP-адрес; параметры чужих приложений не меняйте.

ЧТО ПРОВЕРИТЬ

Выбран профиль проекта **http**. **App URL** содержит адрес текущего учебного маршрута.

ШАГ 14 ИЗ 24

Запустите первый HTTP-проект

HTTP-запрос идёт в отдельный работающий процесс API. Приложение с окнами, сервер БД и API — три разных процесса, которые могут останавливаться независимо.

1. Нажмите **Ctrl+F5**. В открывшемся браузере должен появиться **Hello World!** из **app.MapGet("/", ...)**. Это подтверждает запуск HTTP-приложения, но ещё не подключение к БД. Вернитесь в **Visual Studio**. Нажмите «Отладка» → «Остановить отладку»; если запускали без отладки, закройте окно работающего приложения.

ЧТО ПРОВЕРИТЬ

Браузер показывает **Hello World!** от своего API; после проверки процесс остановлен.

ШАГ 15 ИЗ 24

Соберите исходные данные через JOIN

В **notes** хранится **id_user**, а в ответе нужен логин. По внешнему ключу можно найти именно автора выбранной заметки.

1. В **pgAdmin** выполните первый блок. Найдите контрольную заметку по её **id** и запишите **id_user**. В результате пока есть только ключ автора.
2. Добавьте **JOIN users u ON u.id = n.id_user**. Для каждой заметки он находит пользователя по ключу. Выведите **u.login** рядом с **n.title** и выполните второй блок.
3. Сверьте число строк до и после **JOIN**. Внешний ключ гарантирует существование автора, а первичный ключ **users.id** даёт одну запись автора на каждую заметку.

ЧТО ПРОВЕРИТЬ

Число заметок сохранилось. Для «Заметка 1» видны **login = student** и исходная дата **2027-03-15**.

ШАГ 15 · 1. ИСХОДНЫЕ ПОЛЯ NOTES

Соберите исходные данные через JOIN

```
SELECT id, title, id_user, content, created_at FROM notes ORDER BY id;
```



ШАГ 15 · 2. ТЕ ЖЕ ЗАМЕТКИ С ЛОГИНАМИ АВТОРОВ

Соберите исходные данные через JOIN

```
SELECT n.id, n.title, u.login, n.content, n.created_at
FROM notes n
JOIN users u ON u.id = n.id_user
ORDER BY n.id;
```

ШАГ 16 ИЗ 24 · 1/3

Преобразуйте поля ответа

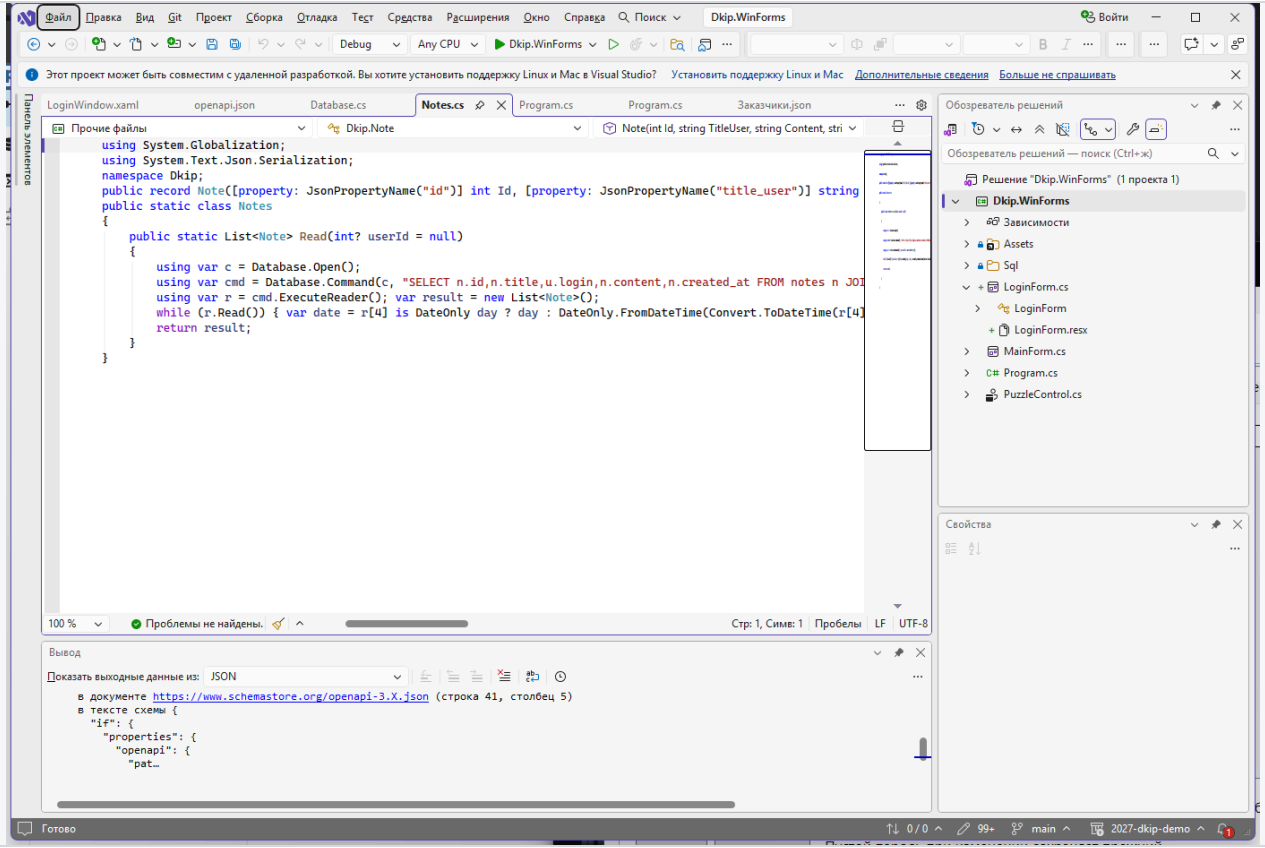
SQL возвращает пять исходных значений, а контракт требует четыре поля JSON. Класс **Note** задаёт имена, а **Notes.Read** преобразует каждую строку.

1. В проект Api добавьте **Database.cs** из заготовки своего стека и задайте **namespace** Api. Создайте **Notes.cs**. В первый блок входят **using**, **namespace** и **record Note**. Атрибуты **JsonPropertyName** задают имена полей в ответе.
2. В том же файле ниже добавьте второй блок. SQL берёт те же поля, которые вы проверили в редакторе. Цикл читает их слева направо с индекса **0**. В новом **Note id** и **content** сохраняются, **title** и **login** склеиваются через « - », а дата форматируется как **dd.MM.yyyy**.

ШАГ 16 ИЗ 24 · 2/3

Преобразуйте поля ответа

Покажите на экране



Код **JOIN** и преобразования полей ответа в **Notes.cs**. Маршрут: **WPF / PostgreSQL**.

ШАГ 16 ИЗ 24 · 3/3

Преобразуйте поля ответа

1 Соберите решение. По контрольной строке заранее составьте ожидаемый JSON. Для «Заметка 1» получатся title_user «Заметка 1 - student» и formatted_date «15.03.2027». Подставьте id из своей БД.

ЧТО ПРОВЕРИТЬ

Проект компилируется, а четыре ожидаемых поля контрольной заметки можно вывести из SQL-строки.

Скачать заготовку Database.cs

ШАГ 16 · 1. ОБЪЯВЛЕНИЕ ОТВЕТА В NOTES.CS

Преобразуйте поля ответа

```
using System.Globalization;
using System.Text.Json.Serialization;
namespace Api;

public record Note(
    [property: JsonPropertyName("id")] int Id,
    [property: JsonPropertyName("title_user")] string TitleUser,
    [property: JsonPropertyName("content")] string Content,
    [property: JsonPropertyName("formatted_date")] string FormattedDate);
```



ШАГ 16 · 2. КЛАСС ЧТЕНИЯ И ПРЕОБРАЗОВАНИЯ СТРОК · 1/2

Преобразуйте поля ответа

```
public static class Notes
{
    public static List<Note> Read(int? userId = null)
    {
        const string source = """
            SELECT n.id, n.title, u.login, n.content, n.created_at
            FROM notes n
            JOIN users u ON u.id = n.id_user
            """;
        string sql = source + (userId.HasValue ? " WHERE n.id_user=@p0" : "") + " ORDER BY n.id";
        using var c = Database.Open();
        using var cmd = Database.Command(c, sql, userId.HasValue ? new object?[] { userId.Value } : []);
        using var r = cmd.ExecuteReader();
        var result = new List<Note>();
```



ШАГ 16 · 2. КЛАСС ЧТЕНИЯ И ПРЕОБРАЗОВАНИЯ СТРОК · 2/2

Преобразуйте поля ответа

```
        while (r.Read())
        {
            var date = r[4] is DateOnly day ? day : DateOnly.FromDateTime(Convert.ToDateTime(r[4],
CultureInfo.InvariantCulture));
            result.Add(new Note(
                Convert.ToInt32(r[0]),
                r.GetString(1) + " - " + r.GetString(2),
                r.GetString(3),
                date.ToString("dd.MM.yyyy", CultureInfo.InvariantCulture)));
        }
        return result;
    }
}
```

ШАГ 16 · ОЖИДАЕМЫЙ ЭЛЕМЕНТ JSON ДЛЯ КОНТРОЛЬНОЙ СТРОКИ

Преобразуйте поля ответа

```
{
  "id": 1,
  "title_user": "Заметка 1 - student",
  "content": "Содержание заметки 1",
  "formatted_date": "15.03.2027"
}
```

ШАГ 17 ИЗ 24

Добавьте GET /notes через MapGet

MapGet направляет GET-запрос по выбранному пути в ReadNotes. Results.Ok передаёт список в HTTP-ответ 200, а ASP.NET Core записывает его в JSON.

1. В Program.cs замените шаблонный код блоком ниже. В обработчике без параметров Notes.Read() читает все заметки. Для второго адреса /api/notes можно добавить такую же регистрацию с тем же ReadNotes.
2. Запустите проект с профилем http и откройте http://127.0.0.1:5272/notes в браузере или Postman. Найдите контрольную запись по её id и сравните четыре значения с ожидаемыми.
3. Остановите свой API перед следующей правкой. Теперь известно, что связка БД, преобразования и HTTP работает до добавления фильтра.

ЧТО ПРОВЕРИТЬ

GET /notes возвращает 200 и массив. У контрольной строки совпадают id, title_user, content и formatted_date.

ШАГ 17 · ПЕРВЫЙ РАБОТАЮЩИЙ PROGRAM.CS

Добавьте GET /notes через MapGet

```
using Api;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

IResult ReadNotes() => Results.Ok(Notes.Read());

app.MapGet("/notes", ReadNotes);
app.Run();
```

ШАГ 18 ИЗ 24 · 1/3

Обработайте неверные параметры

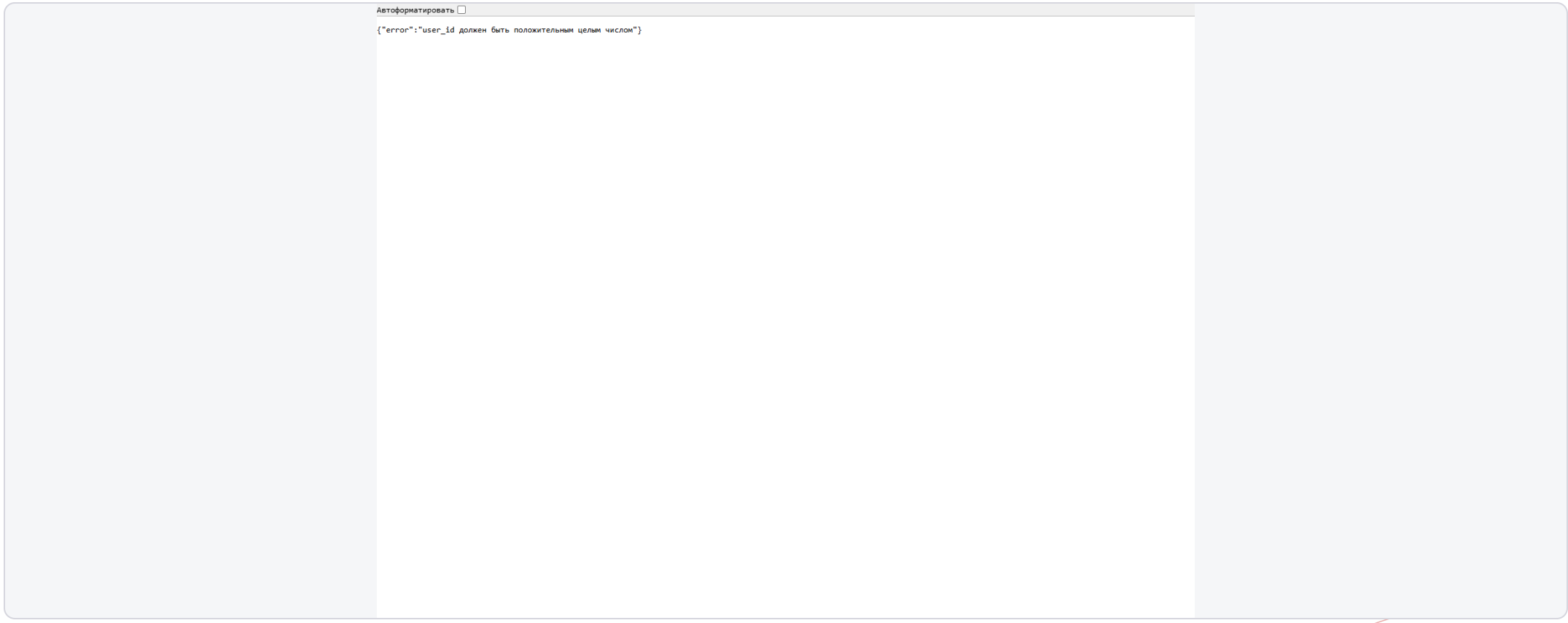
Фильтр user_id — необязательный параметр учебного примера. Его проверяют до вызова БД, чтобы запрос с неверным значением получил 400.

1. Замените только метод ReadNotes в Program.cs. null означает отсутствие фильтра. Если user_id передан, обработчик проверяет, что значение одно, оно целое и больше нуля. Неизвестное имя параметра тоже приводит к 400.
2. При допустимом id передайте его в Notes.Read. В SQL фильтр применяется к n.id_user, а число передаётся параметром @p0. Фильтр ищет заметки по автору, поэтому id заметки здесь не используется.

ШАГ 18 ИЗ 24 · 2/3

Обработайте неверные параметры

Покажите на экране



Настоящий JSON-ответ на **user_id=abc**. Статус **400** отдельно подтверждён **HTTP**-тестами. Маршрут: **WPF / PostgreSQL**.

ШАГ 18 ИЗ 24 · 3/3

Обработайте неверные параметры

1 Запустите API снова. Сравните **/notes** и **/notes?user_id=<id student>**, подставив числовой **id** без угловых скобок. При тестовых данных останутся две заметки **student**. Запрос **/notes?user_id=abc** должен вернуть **400**.

ЧТО ПРОВЕРИТЬ

Без параметра видны все заметки, с **id** — заметки выбранного автора, с **abc** — **HTTP 400** и объект **error**.



ШАГ 18 · READNOTES С ПРОВЕРКОЙ ПАРАМЕТРА · 1/2

Обработайте неверные параметры

```
IResult ReadNotes(HttpRequest request)
{
    int? userId = null;
    if (request.Query.Keys.Any(k => k != "user_id"))
        return Results.BadRequest(new
        {
            error = "Неизвестный параметр. Допустим user_id."
        });
    if (request.Query.ContainsKey("user_id"))
    {
        if (request.Query["user_id"].Count != 1 || !int.TryParse(request.Query["user_id"], out var parsed) || parsed <=
0)
            return Results.BadRequest(new
            {
                error = "user_id должен быть положительным целым числом"
            });
```

ШАГ 18 · READNOTES С ПРОВЕРКОЙ ПАРАМЕТРА · 2/2

Обработайте неверные параметры

```
        userId = parsed;
    }
    return Results.Ok(Notes.Read(userId));
}
```

ШАГ 19 ИЗ 24

Обработайте пустой результат и сбой

В списке Notes.Read может быть ноль записей. Если БД ответила без строк, запрос всё равно выполнен. При сбое БД обработчик возвращает 500.

1. Внутри ReadNotes замените строку return Results.Ok(Notes.Read(userId)); блоком ниже. Проверки query-параметров сохраните перед try.
2. Перезапустите API и запросите id автора без заметок. Цикл чтения не добавит элементов, поэтому Results.Ok вернёт 200 с [].
3. В случае исключения при чтении catch запишет подробности в журнал. Клиент получит HTTP 500 и JSON с полем error. Проверьте этот путь позже на отдельном тестовом профиле.

ЧТО ПРОВЕРИТЬ

Для допустимого фильтра без записей получается 200 и []. Сбой чтения обрабатывается отдельно.



ШАГ 19 · БЛОК ЧТЕНИЯ И ОБРАБОТКИ ОШИБКИ ВНУТРИ READNOTES

Обработайте пустой результат и сбой

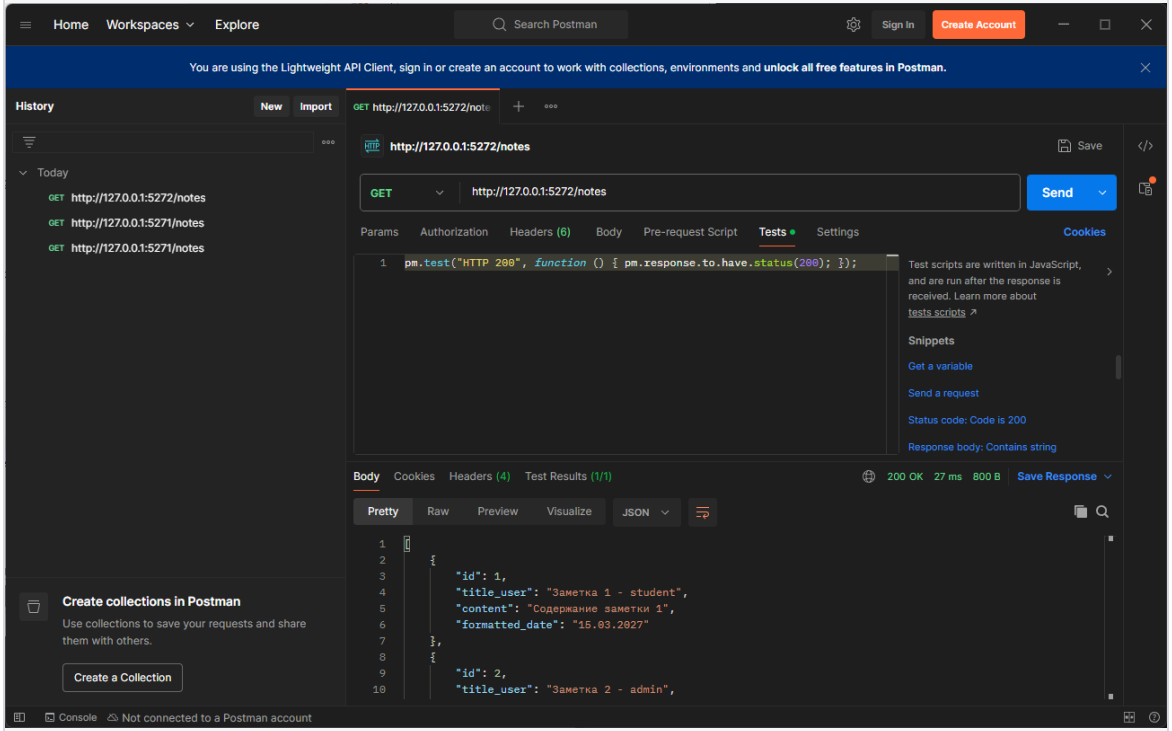
```
try
{
    return Results.Ok(Notes.Read(userId));
}
catch (Exception ex)
{
    // Подробности остаются в журнале сервера. Клиент получает только сообщение.
    app.Logger.LogError(ex, "Ошибка обработки API");
    return Results.Json(
        new { error = "Ошибка подключения к базе данных или обработки запроса" },
        statusCode: StatusCodes.Status500InternalServerError);
}
```

ШАГ 20 ИЗ 24 · 1/2

Запустите сервер API

API работает отдельным приложением. Пока оно остановлено, браузер и Postman не получат данные.

1. Откройте свой Api.csproj через Файл → Открыть → Проект или решение.
2. Выделите Api → Отладка → Свойства отладки Api.
3. В профиле http задайте DKIP_CONNECTION и DKIP_STACK=postgresql в Переменных среды.



Postman Lightweight: настоящий ответ локального API. Маршрут: WPF / PostgreSQL.

ШАГ 20 ИЗ 24 · 2/2

Запустите сервер API

1. В App URL укажите http://127.0.0.1:5272; сохраните профиль.
2. Выберите этот профиль рядом с зелёной кнопкой запуска → Отладка → Запуск без отладки (Ctrl+F5).
3. В браузере откройте http://127.0.0.1:5272/notes. Оставьте приложение работающим.

ЧТО ПРОВЕРИТЬ

Браузер показывает JSON; последующие запросы выполняются к этому адресу.

ШАГ 21 ИЗ 24 · 1/2

Создайте коллекцию Postman

Каждый запрос проверяет один сценарий. Переменные коллекции отделяют адрес основного API от адреса тестового профиля.

1. В Postman создайте коллекцию для своего API. В Variables добавьте base_url с адресом рабочего профиля и error_url с адресом тестового профиля.
2. Задайте base_url = http://127.0.0.1:5272. Сохраните GET {{base_url}}/notes, запрос с ?user_id=abc и запрос с id автора без заметок, например ?user_id=2147483647.
3. Для проверки сбоя сохраните GET {{error_url}}/notes. При необходимости сравните свои запросы с коллекцией выбранного стека.

ЧТО ПРОВЕРИТЬ

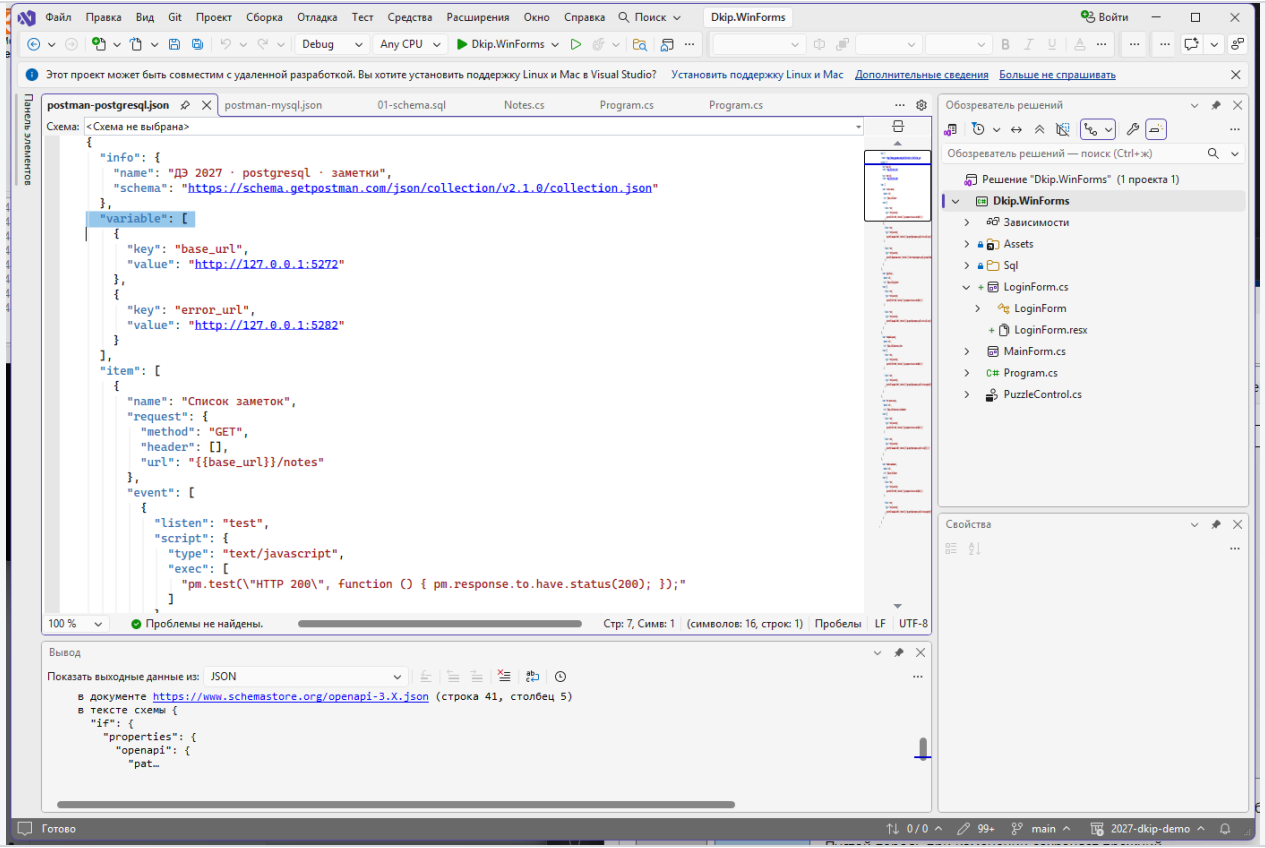
У каждого запроса есть адрес и ожидаемый сценарий.

Скачать коллекцию для сверки

ШАГ 21 ИЗ 24 · 2/2

Создайте коллекцию Postman

Покажите на экране



Файл коллекции **Postman**: адреса **base_url** и **error_url** перед импортом. Маршрут: **WPF / PostgreSQL**.

ШАГ 22 ИЗ 24 · 1/3

Выполните проверки 200 и JSON

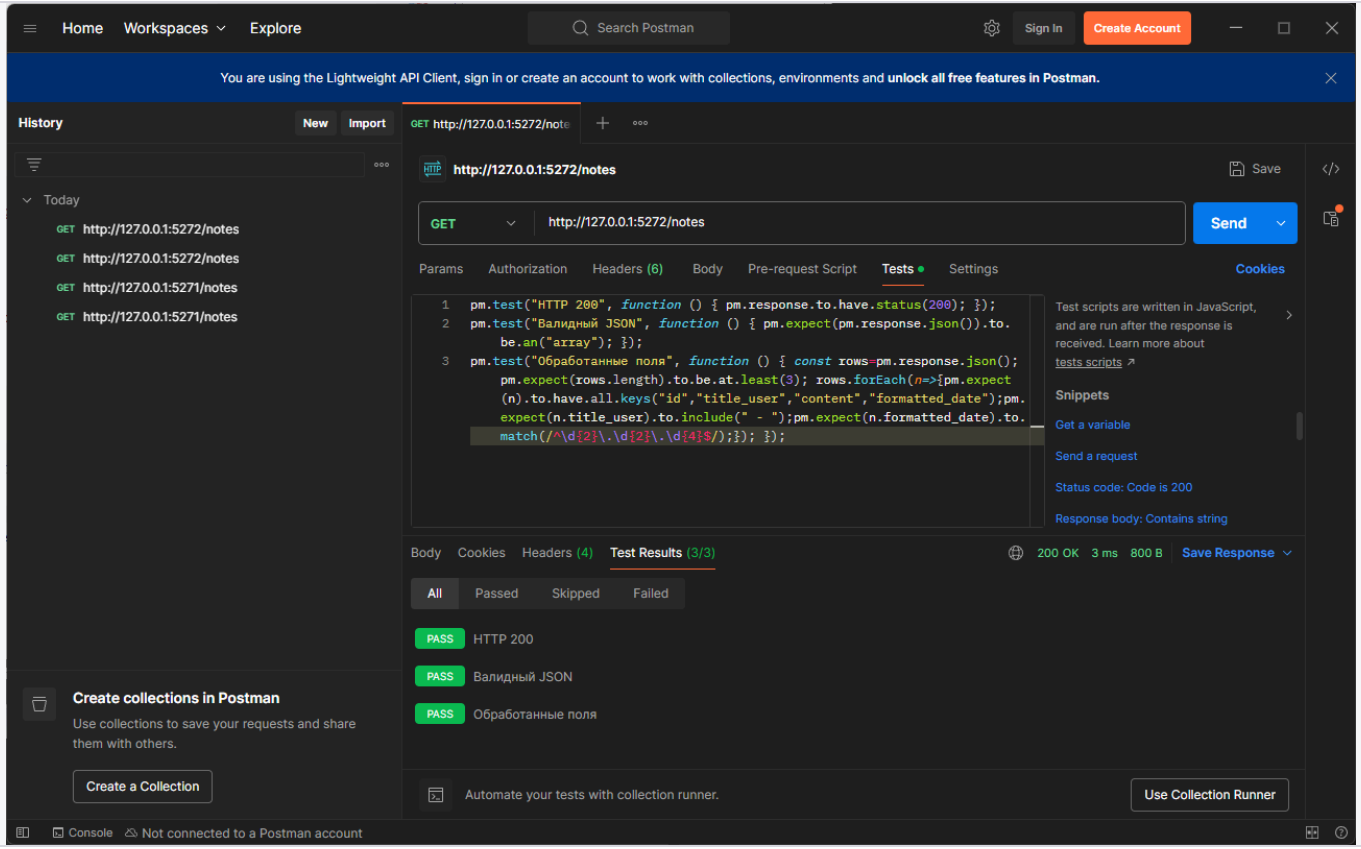
Проверка статуса и формы массива показывает, что ответ обработан. Сравнение одной заметки по id проверяет, что преобразованы данные нужной строки.

1. В Variables коллекции создайте expected_note. Вставьте JSON-объект из первого блока и подставьте значения, которые рассчитали по контрольной строке БД. Если id отличается от 1, укажите присвоенный id.
2. У GET {{base_url}}/notes откройте Scripts → Post-response. Вставьте второй блок и отправьте запрос. Сначала проверяются статус и массив. Затем тест находит запись по id и сравнивает все четыре значения, включая логин в title_user.

ШАГ 22 ИЗ 24 · 2/3

Выполните проверки 200 и JSON

Покажите на экране



Postman: выполненные проверки HTTP 200, JSON и преобразованных полей. Маршрут: WPF / PostgreSQL.

ШАГ 22 ИЗ 24 · 3/3

Выполните проверки 200 и JSON

1 В Test Results проверьте три результата. Для контрольного опыта измените ожидаемый логин в expected_note, отправьте запрос снова и увидьте отказ сравнения. Верните правильный логин и повторите запрос.

ЧТО ПРОВЕРИТЬ

Тест формы проходит для массива. Тест контрольной заметки проходит только при совпадении фактических значений с ожидаемыми.

ШАГ 22 · 1. ЗНАЧЕНИЕ EXPECTED_NOTE В VARIABLES

Выполните проверки 200 и JSON

```
{
  "id": 1,
  "title_user": "Заметка 1 - student",
  "content": "Содержание заметки 1",
  "formatted_date": "15.03.2027"
}
```

ШАГ 22 · 2. ТЕСТЫ ЗАПРОСА В POST-RESPONSE

Выполните проверки 200 и JSON

```
pm.test("HTTP 200", () => pm.response.to.have.status(200));
pm.test("Ответ — массив", () => {
  pm.expect(pm.response.json()).to.be.an("array");
});

pm.test("Контрольная заметка совпадает со строкой БД", function () {
  const expected = JSON.parse(pm.collectionVariables.get("expected_note"));
  const actual = pm.response.json().find(note => note.id === expected.id);
  pm.expect(actual).to.deep.equal(expected);
});
```

ШАГ 23 ИЗ 24 · 1/2

Воспроизведите 500 на отдельном сервере

Тестовый профиль запускает API с другим портом HTTP и заведомо недоступным адресом БД. Так HTTP-сервер может принять запрос и вернуть 500.

1. Остановите тестовую копию Api, если она запущена. В её свойствах отладки создайте отдельный профиль **DkipError** с запуском проекта.
2. Скопируйте **DKIP_CONNECTION** рабочего профиля в **DkipError**. В копии задайте порт БД 1 и **Timeout=2**. В **App URL** задайте **http://127.0.0.1:5282**. Сохраните профиль.
3. Выберите **DkipError** рядом с кнопкой запуска и нажмите **Ctrl+F5**. В **Postman** укажите этот HTTP-адрес в **error_url**. В запросе «Ошибка подключения» откройте **Scripts** → **Post-response** и вставьте блок ниже.

ШАГ 23 ИЗ 24 · 2/2

Воспроизведите 500 на отдельном сервере

1 Нажмите **Send**. Проверьте **Test Results** и сохраните статус и **Body** для задания 6. Оставьте оба экземпляра API запущенными до завершения прогона коллекции.

ЧТО ПРОВЕРИТЬ

Работающий API возвращает **500** и **JSON** с полем **error**. Обе проверки **Postman** проходят.

ШАГ 23 · ПРОВЕРКА СТАТУСА И JSON В POST-RESPONSE

Воспроизведите 500 на отдельном сервере

```
pm.test("HTTP 500", function () { pm.response.to.have.status(500); });
pm.test("Валидный JSON", function () { pm.expect(pm.response.json()).to.have.property("error"); });
```

ШАГ 24 ИЗ 24 · 1/3

Сохраните результаты тестирования

Подготовьте два адреса и запустите коллекцию

Коллекция хранит запросы и проверки. Отчёт прогона показывает, какие проверки прошли на запущенном API.

1. В настольном Postman откройте Variables коллекции. В base_url укажите http://127.0.0.1:5272, в error_url — http://127.0.0.1:5282. Оба экземпляра API должны работать одновременно.
2. В меню ··· коллекции выберите Run collection. Оставьте включёнными все запросы и нажмите Run с названием коллекции. Дождитесь результата.

ШАГ 24 ИЗ 24 · 2/3

Сохраните результаты тестирования

Прочитайте результаты прогона

1. Откройте результаты запросов. У обычного списка ожидается HTTP 200 с заметками, у пустого — HTTP 200 и []. У неверного параметра ожидается 400, у «Ошибки подключения» — 500. Проверки этих статусов должны пройти.
2. В готовой коллекции один прогон даёт 5 запросов и 12 успешных проверок. В собственной коллекции их число может отличаться: проверьте, что отработали все ваши сценарии.
3. Убедитесь, что нет упавших тестов и ошибок отправки. Если Postman сообщает ECONNREFUSED, проверьте, запущен ли профиль для нужного адреса, затем повторите прогон.

ШАГ 24 ИЗ 24 · 3/3

Сохраните результаты тестирования

Выгрузите отчёт и коллекцию отдельно

1. В результатах прогона откройте ··· → **Export run results** и сохраните файл «Результаты-тестов-API.json». Эта выгрузка доступна в настольном **Postman**.
2. Вернитесь к меню ··· самой коллекции, выберите **Export** → **Collection v2.1** и сохраните отдельный **JSON**. После выгрузки остановите **DkipError**; обычный профиль оставьте запущенным для задания 6.

ЧТО ПРОВЕРИТЬ

Сохранены два **JSON**: отчёт успешного прогона и коллекция, которую можно импортировать и запустить повторно.

РЕЗУЛЬТАТ ЭТАПА

Что должно остаться после этапа

Сохранены проект Api, коллекция Postman и результаты тестов. Сервис возвращает поля и формат из приложения к КИМ.

ЧТО ПРОВЕРИТЬ

HTTP-сервис и коллекция Postman с тестами.

ЗАДАНИЕ 6 · 20 МИН

Документация API

Документацию пишем по работающему сервису. Точный запрос и фактический ответ берём из Postman.

ЧТО ПРОВЕРИТЬ

Готов документ Прил_6_ОЗ_КИМ_09.02.07-5-2027.docx, заполненный по шаблону КИМ.

УСЛОВИЕ ИЗ КИМ

Разработка проектной документации

Разработать документацию на реализованный API в точном соответствии с шаблоном из Приложения 3.

В документации описать все доступные методы, параметры к ним и привести примеры запросов/ответов.

Приложение: Прил_6_ОЗ_КИМ_09.02.07-5-2027.docx

КИМ · стр. 32; Приложение 3 Исходные файлы · ZIP

ШАГ 1 ИЗ 7

Откройте исходный шаблон Word

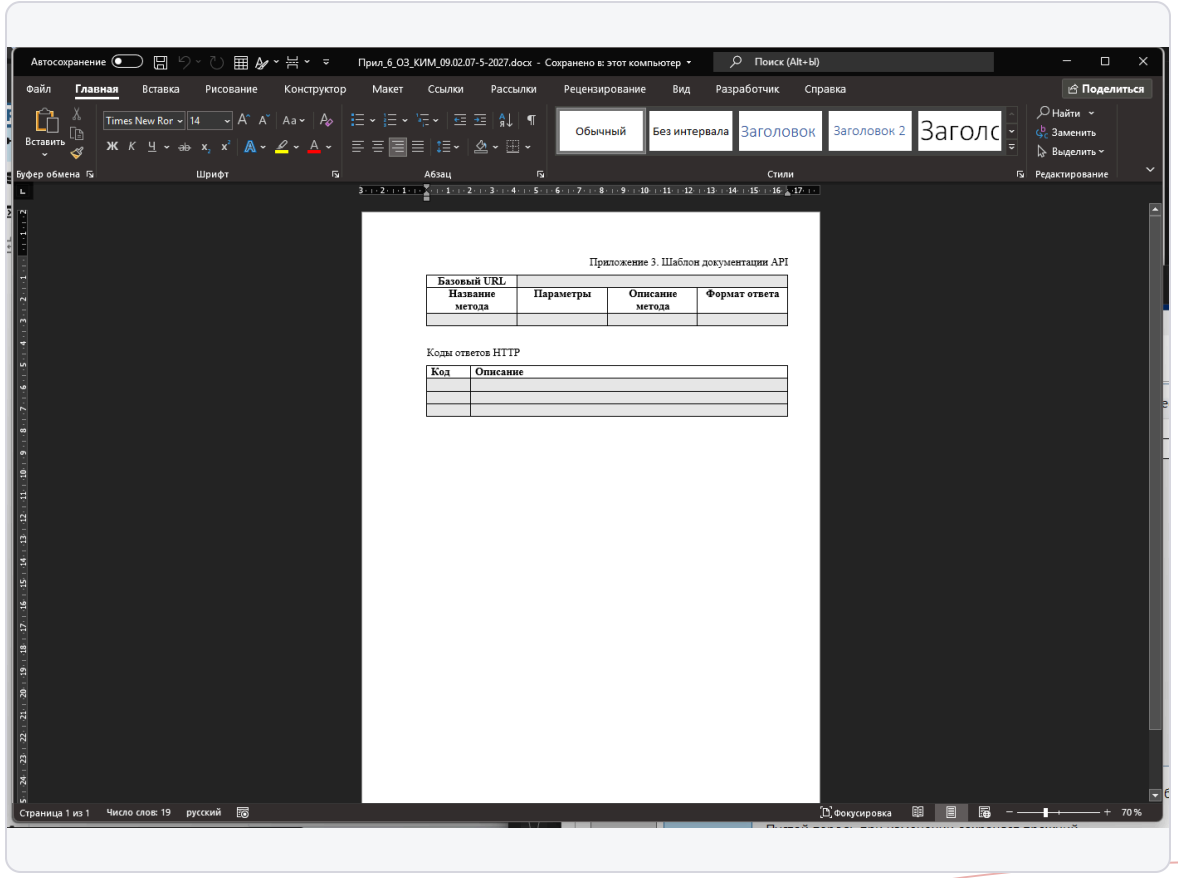
Задание 6 описывает ASP.NET Core Minimal API, который вы уже написали в задании 5.

1. Скачайте Приложение 3 к заданию 6. Сделайте копию DOCX и откройте её в Word.
2. Найдите поля базового URL, методов, параметров, описания и формата ответа. Ниже найдите таблицу кодов HTTP.

ЧТО ПРОВЕРИТЬ

Открыта копия исходного шаблона с таблицами для методов и кодов HTTP.

КИМ, стр. 32; Приложение 3



Исходный шаблон Приложения 3 открыт в Microsoft Word. Маршрут: WPF / PostgreSQL.

ШАГ 2 ИЗ 7

Найдите методы в Program.cs

Строка **MapGet** связывает путь и обработчик. По ней можно определить, что написать в колонке «Название метода».

1. Откройте **Program.cs** в своём проекте **Api**. Найдите строки **app.MapGet** для заметок.
2. Для каждой строки выпишите **GET** и путь. Если один обработчик привязан к **/notes** и **/api/notes**, укажите оба пути.

ЧТО ПРОВЕРИТЬ

Список методов в черновике совпадает с маршрутами **MapGet** в вашем проекте.

ШАГ 2 · PROGRAM.CS · 1/3

Найдите методы в Program.cs

```
using Dkip;
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
app.UseStaticFiles();
IResult ReadNotes(HttpRequest request)
{
    int? userId = null;
    if (request.Query.Keys.Any(k => k != "user_id"))
        return Results.BadRequest(new
        {
            error = "Неизвестный параметр. Допустим user_id."
        });
    if (request.Query.ContainsKey("user_id"))
    {
        if (request.Query["user_id"].Count != 1 || !int.TryParse(request.Query["user_id"], out var parsed) || parsed <=
0)
```

ШАГ 2 · PROGRAM.CS · 2/3

Найдите методы в Program.cs

```
            return Results.BadRequest(new
            {
                error = "user_id должен быть положительным целым числом"
            });
        userId = parsed;
    }
    try
    {
        return Results.Ok(Notes.Read(userId));
    }
    catch (Exception ex)
    {
        // Подробности остаются в журнале сервера. Клиент получает только сообщение.
        app.Logger.LogError(ex, "Ошибка обработки API");
        return Results.Json(
            new { error = "Ошибка подключения к базе данных или обработки запроса" },
```



ШАГ 2 · PROGRAM.CS · 3/3

Найдите методы в Program.cs

```
            statusCode: StatusCodes.Status500InternalServerError);
    }
}
app.MapGet("/notes", ReadNotes);
app.MapGet("/api/notes", ReadNotes);
app.MapGet("/", () => Results.Redirect("/notes"));
app.Run();
```

ШАГ 3 ИЗ 7

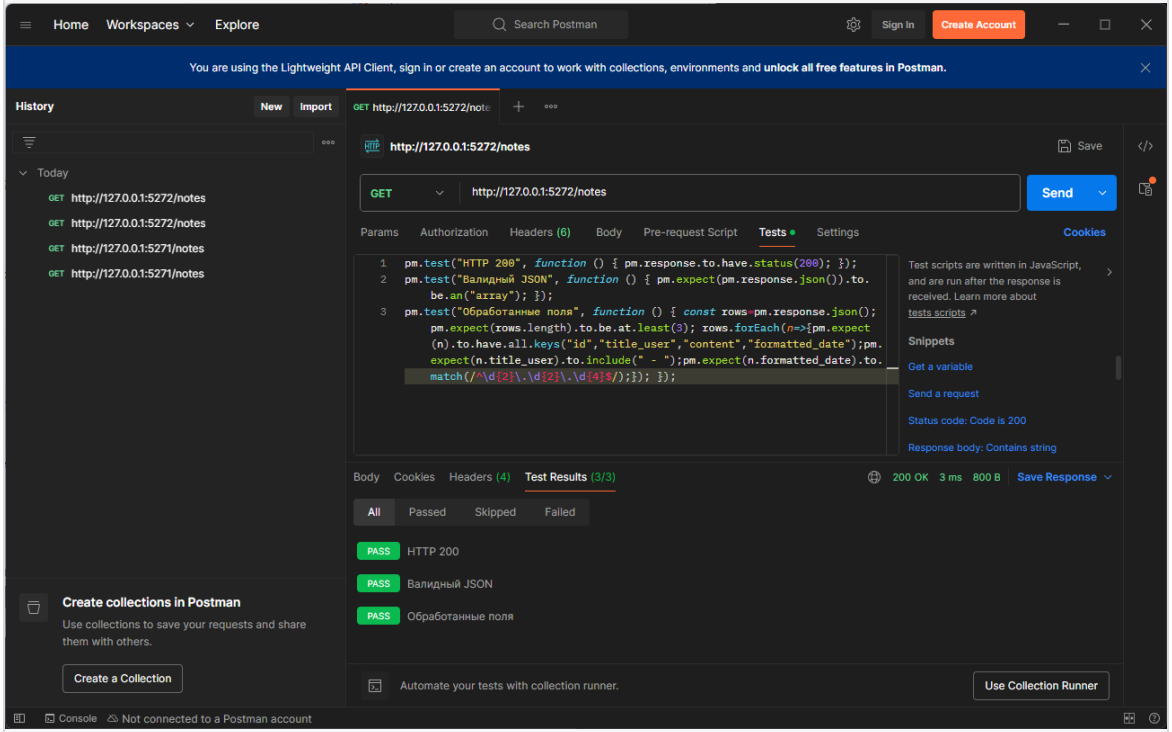
Получите успешный ответ в Postman

Документация должна показывать ответ работающего сервиса с вашими данными.

1. Запустите свой Api из профиля **http** в **Visual Studio**. Проверьте, что выбран адрес **http://127.0.0.1:5272**.
2. В **Postman** откройте коллекцию из задания 5. Выберите запрос списка заметок и нажмите **Send**.
3. Запишите адрес, статус и JSON из **Body**. Для примера можно привести одну заметку с подписью «Один элемент ответа».

ЧТО ПРОВЕРИТЬ

Postman показывает статус **200 OK**. В **JSON** видны **id**, **title_user**, **content** и **formatted_date**.



Postman: выполненные проверки **HTTP 200**, **JSON** и преобразованных полей. Маршрут: **WPF / PostgreSQL**.

ШАГ 4 ИЗ 7 · 1/2

Уточните параметр и пустой результат

Параметр user_id фильтрует заметки по автору. LEFT JOIN сохраняет в таблице авторов и тех, у кого нет заметок. COUNT(n.id) считает только существующие заметки, поэтому для такого автора получится 0.

1. Выполните запрос ниже в своей базе. Найдите id и login автора, у которого notes_count больше нуля, и id автора с notes_count = 0. Если у всех есть заметки, возьмите свободный положительный id, которого нет в колонке id.

2. В Postman отправьте GET http://127.0.0.1:5272/notes?user_id=<id>: подставьте id автора с заметками. Например, при notes_count = 2 в ответе HTTP 200 должно быть два объекта. В обоих title_user логин после « - » совпадает с колонкой login.

ШАГ 4 ИЗ 7 · 2/2

Уточните параметр и пустой результат

1. Подставьте второй id и повторите запрос. Ожидайте HTTP 200 и тело []. Сохраните оба фактических запроса и ответа. В DOCX укажите, что user_id — необязательное положительное целое.

ЧТО ПРОВЕРИТЬ

В документации есть пример фильтра с найденными заметками и пример пустого ответа 200 с [].

ШАГ 4 · НАЙДИТЕ АВТОРОВ С ЗАМЕТКАМИ И БЕЗ НИХ

Уточните параметр и пустой результат

```
SELECT u.id, u.login, COUNT(n.id) AS notes_count
FROM users u
LEFT JOIN notes n ON n.id_user = u.id
GROUP BY u.id, u.login
ORDER BY u.id;
```

ШАГ 5 ИЗ 7

Возьмите примеры ответов 400 и 500

Статус 400 относится к неверному запросу. Статус 500 относится к сбою работающего сервиса.

1. Откройте в коллекции запрос с неверным значением **user_id** и нажмите **Send**. Сохраните статус и объект **error**.
2. Возьмите результат **500**, сохранённый в задании **5**. Для повторения запустите профиль **DkipError** и отправьте запрос «Ошибка подключения» по его адресу.
3. Сохраните точный запрос, условие воспроизведения, статус и JSON ответа.

ЧТО ПРОВЕРИТЬ

В **Postman** получены ответы **400** и **500** с полем **error**. Адрес и условие теста записаны.

ШАГ 6 ИЗ 7 · 1/4

Перенесите методы, параметры и ответы в DOCX

Возьмите адрес и метод из своего проекта

Заполним одну строку шаблона для GET /notes. По ней читатель сможет повторить запрос и узнать ожидаемый ответ.

1. В поле «Базовый URL» впишите адрес из профиля http, для этого маршрута — http://127.0.0.1:5272. В Program.cs найдите app.MapGet("/notes", ReadNotes). В колонке «Название метода» напишите GET /notes.
2. В колонке «Описание метода» напишите, что GET /notes возвращает заметки и добавляет логин автора к заголовку. Пример запроса из Postman — GET http://127.0.0.1:5272/notes.

ШАГ 6 ИЗ 7 · 2/4

Перенесите методы, параметры и ответы в DOCX

Заполните параметр по правилам обработчика

1 В ReadNotes посмотрите, как читается user_id из query. Запишите в «Параметры» положительное целое, необязательное. Без него обработчик читает все заметки, с ним фильтрует по id автора. Добавьте запрос с реальным id пользователя.

ШАГ 6 ИЗ 7 · 3/4

Перенесите методы, параметры и ответы в DOCX

Перенесите формат и фактический ответ

1. В «Формат ответа» впишите JSON-массив. Укажите id, title_user, content и formatted_date с их смыслом. Из Body успешного запроса скопируйте реальный JSON. Если приводите одну запись, подпишите её «Один элемент ответа».
2. В таблице кодов укажите 200 для массива с заметками и массива [], 400 для неверного параметра и 500 для сбоя. Для каждого сценария добавьте свой запрос и фактический ответ из Postman. Для второго пути добавьте строку, если он зарегистрирован в вашем Program.cs.

ШАГ 6 ИЗ 7 · 4/4

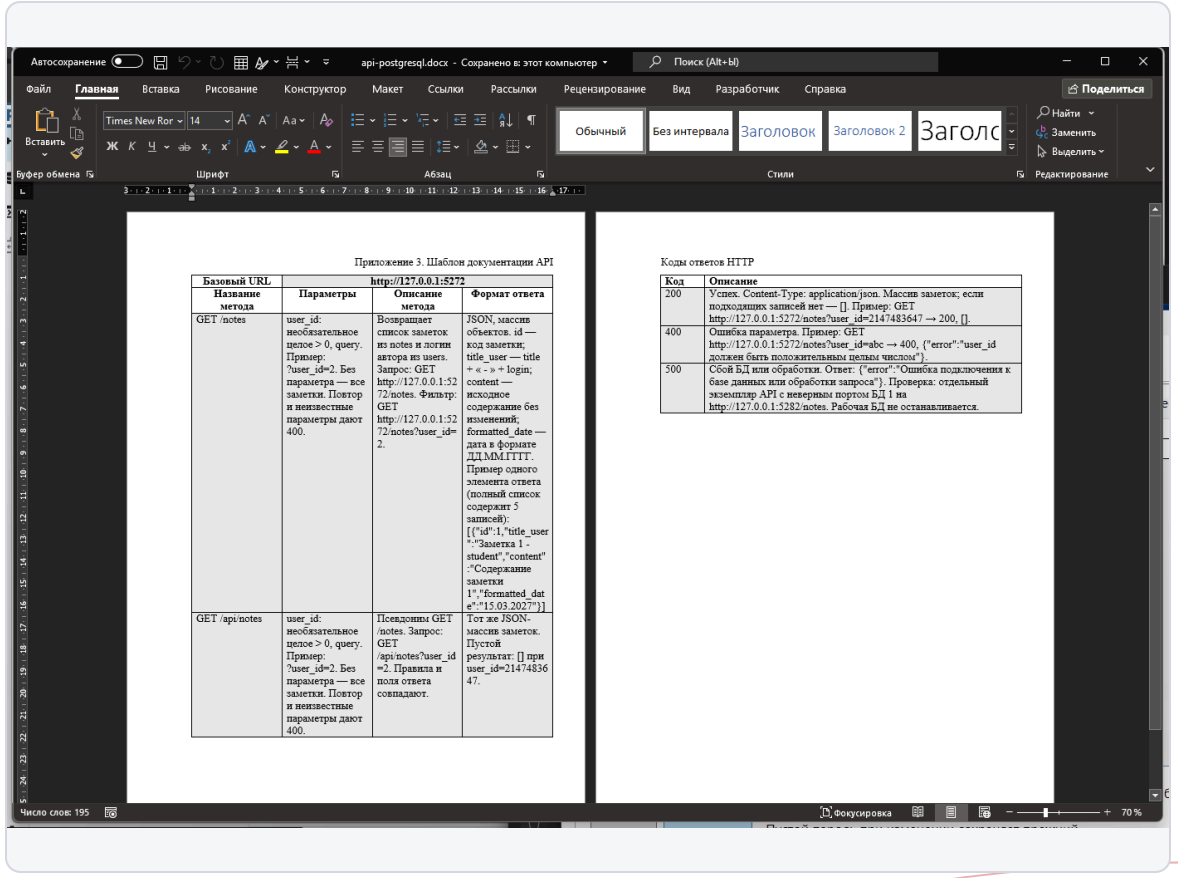
Перенесите методы, параметры и ответы в DOCX

Покажите результат на экране

ЧТО ПРОВЕРИТЬ

В строке **GET /notes** заполнены четыре колонки. Адрес, параметр, статус и **JSON** совпадают с вашим запросом в **Postman**.

Скачать заполненный шаблон для сверки



Приложение 3. Шаблон документации API

Базовый URL	http://127.0.0.1:5272		
Название метода	**Параметры**	**Описание метода**	**Формат ответа**
GET /notes	user_id: необязательное целое > 0, query. Пример: ?user_id=2. Без параметра — все заметки. Повтор и неизвестные параметры дают 400.	Возвращает список заметок из notes и логин автора из users. Запрос: GET http://127.0.0.1:5272/notes. Фильтр: GET http://127.0.0.1:5272/notes?user_id=2.	JSON, массив объектов. id — код заметки; title_user — title + « - » + login; content — исходное содержание без изменений; formatted_date — дата в формате ДД.ММ.ГГГГ. Пример одного элемента ответа (полный список содержит 5 записей): [{"id":1,"title_user":"Заметка 1 - student","content":"Содержание заметки 1","formatted_date":"15.03.2027"}]
GET /api/notes	user_id: необязательное целое > 0, query. Пример: ?user_id=2. Без параметра — все заметки. Повтор и неизвестные параметры дают 400.	Псевдоним GET /notes. Запрос: GET /api/notes?user_id=2. Правила и поля ответа совпадают.	Тот же JSON-массив заметок. Пустой результат: [] при user_id=2147483647.

Коды ответов HTTP

Код	Описание
200	Успех. Content-Type: application/json. Массив заметок; если подходящих записей нет — []. Пример: GET http://127.0.0.1:5272/notes?user_id=2147483647 → 200, [].
400	Ошибка параметра. Пример: GET http://127.0.0.1:5272/notes?user_id=abc → 400, {"error":"user_id должен быть положительным целым числом"}.
500	Сбой БД или обработки. Ответ: {"error":"Ошибка подключения к базе данных или обработки запроса"}. Проверка: отдельный экземпляр API с неверным портом БД 1 на http://127.0.0.1:5282/notes. Рабочая БД не останавливается.

Заполненный исходный шаблон документации API в **Microsoft Word**. Маршрут: **WPF / PostgreSQL**.

ШАГ 6 · HTTP 200 · ОДИН ЭЛЕМЕНТ ОТВЕТА ДЛЯ СВЕРКИ

Перенесите методы, параметры и ответы в DOCX

```
[
  {
    "id": 1,
    "title_user": "Заметка 1 - student",
    "content": "Содержание заметки 1",
    "formatted_date": "15.03.2027"
  }
]
```

ШАГ 7 ИЗ 7 · 1/2

Сверьте и сохраните документацию

Сверьте метод с OpenAPI

DOCX объясняет вызов читателю. Статический OpenAPI JSON описывает тот же контракт в структурированном виде.

1. Откройте пример **openapi.json**. Сопоставьте запись **paths** → **/notes** → **get** с **GET /notes**. В **parameters** у **user_id** должны совпасть **in** = **query**, **type** = **integer**, **minimum** = **1** и **required** = **false**. В **responses** проверьте **200**, **400** и **500**.
2. У **200** сверьте массив и схему **Note** с четырьмя полями из **Postman**. Если ваш сервис реализует один путь, в **paths** оставьте только его. Сохраните свой **openapi.json** в папку **wwwroot** проекта **Api**.

ШАГ 7 ИЗ 7 · 2/2

Сверьте и сохраните документацию

Проверьте документ и файл на работающем сервере

1. Если в **Program.cs** ещё нет **UseStaticFiles**, добавьте строку из блока после **builder.Build()**. Перезапустите API и откройте **http://127.0.0.1:5272/openapi.json**. Сервер должен вернуть **JSON**.
2. Возьмите адрес **GET** из **DOCX** и повторите запрос в **Postman**. Сравните метод, статус и поля ответа. Сохраните заполненный **DOCX** в «Результаты», затем откройте его и проверьте читаемость таблиц.

ЧТО ПРОВЕРИТЬ

Запрос из **DOCX** воспроизводится, а открытый **/openapi.json** совпадает с работающими путями, параметрами и ответами.

Открыть пример **OpenAPI JSON**



ШАГ 7 · ОТДАЧА ФАЙЛОВ ИЗ WWWROOT В PROGRAM.CS

Сверьте и сохраните документацию

```
app.UseStaticFiles();
```

РЕЗУЛЬТАТ ЭТАПА

Что должно остаться после этапа

Готов документ Прил_6_ОЗ_КИМ_09.02.07-5-2027.docx, заполненный по шаблону КИМ.

ЧТО ПРОВЕРИТЬ

Заполненный **DOCX** по шаблону КИМ с фактическими примерами ответов.

ГОТОВЫЙ ПРИМЕР

От исходных файлов до работающей системы

WPF + PostgreSQL

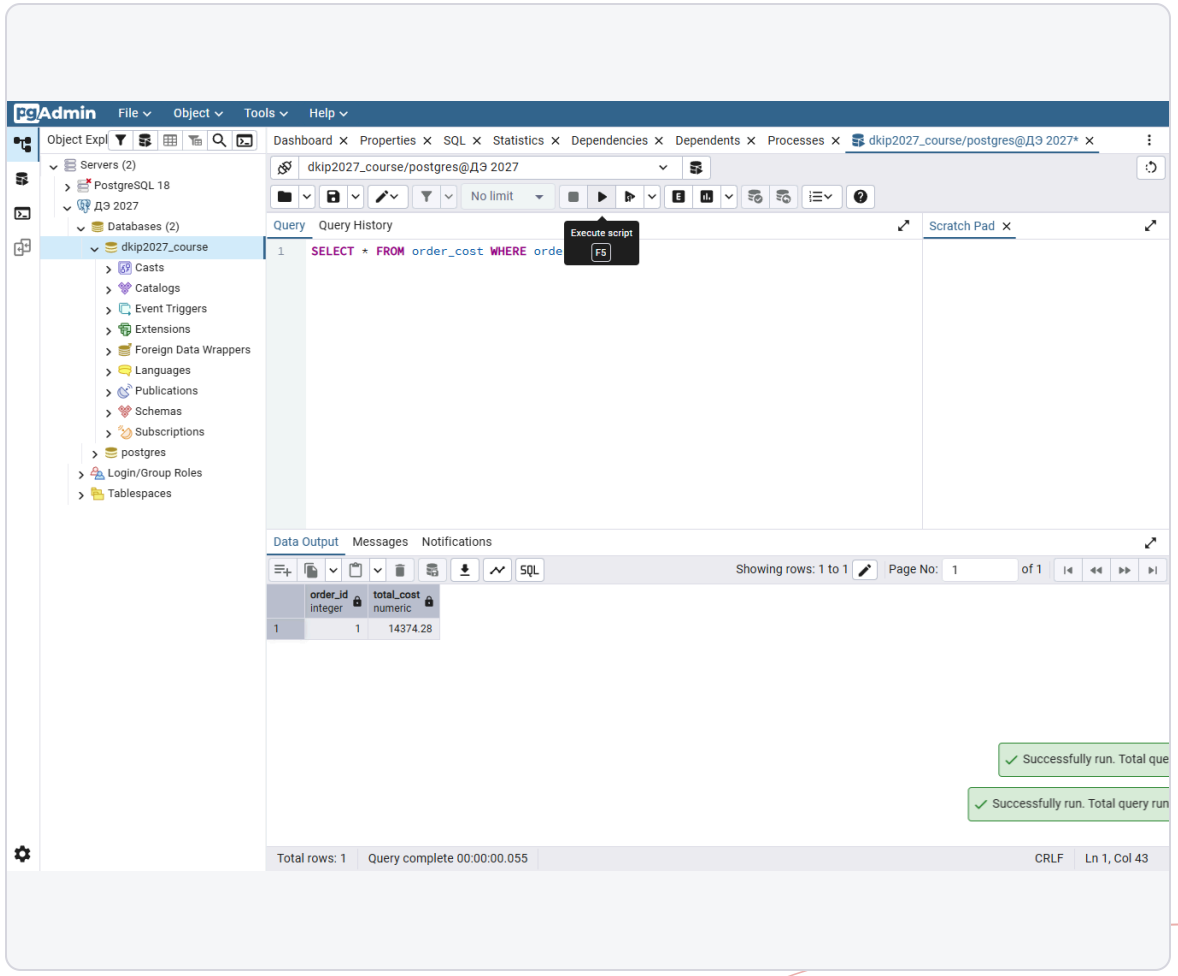
Готовый пример · ZIP

База и расчёт заказа

1. В pgAdmin откройте **counterparty** и сравните первую запись с «Заказчики.json».
2. Откройте представление **order_cost** и найдите заказ с order_id = 1.

ЧТО ПРОВЕРИТЬ

Первый код **000000001** сохранился. Нормативная стоимость — 14 374,28.



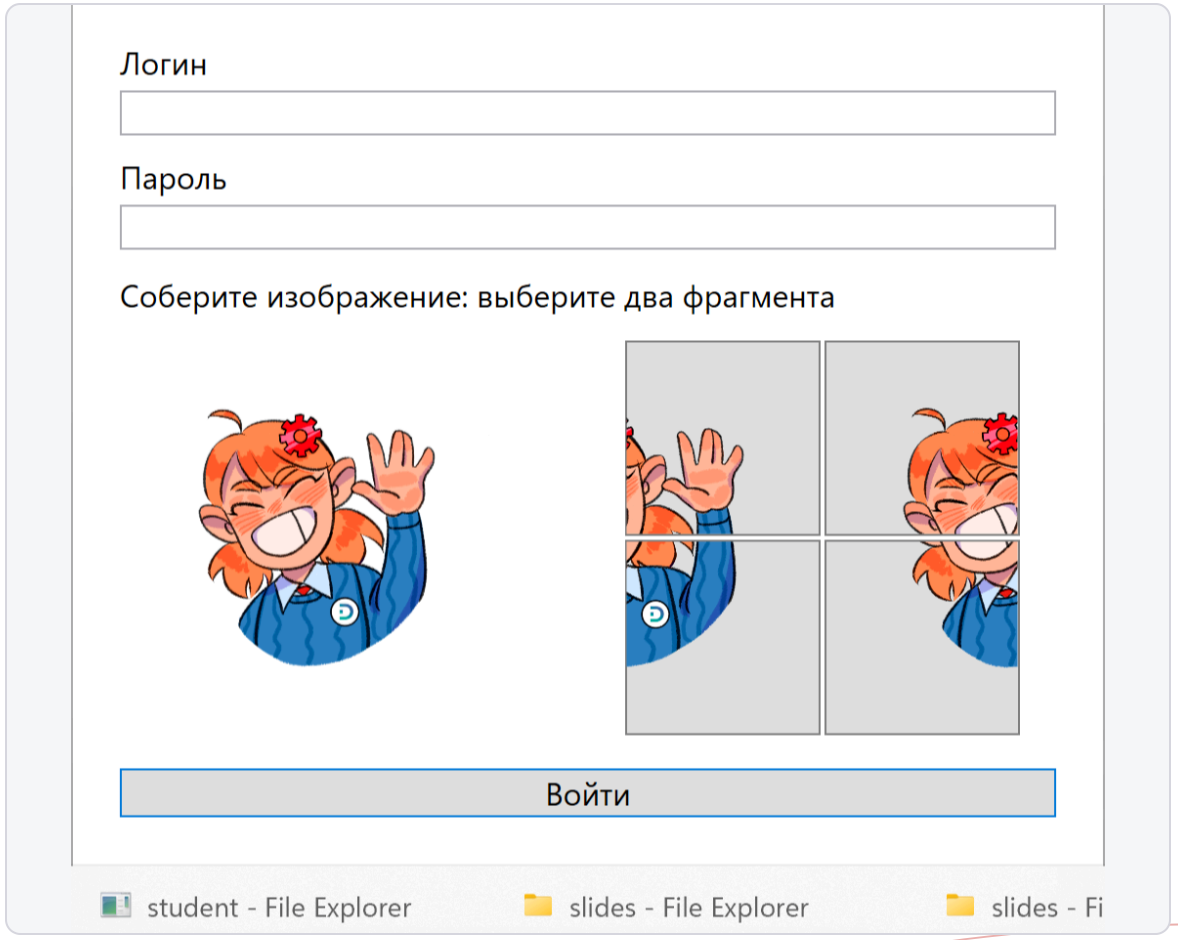
Результат расчёта в СУБД: **14 374,28**. Маршрут: **WPF / PostgreSQL**.

Вход в настольное приложение

1. Запустите **WPF**. В окне «Вход» введите логин и пароль, затем соберите пазл.
2. Покажите по очереди вход пользователя и администратора. Учебный пароль обеих записей: **Demo2027!**.

ЧТО ПРОВЕРИТЬ

Логин **student** открывает окно пользователя. Логин **admin** — управление учётными записями.



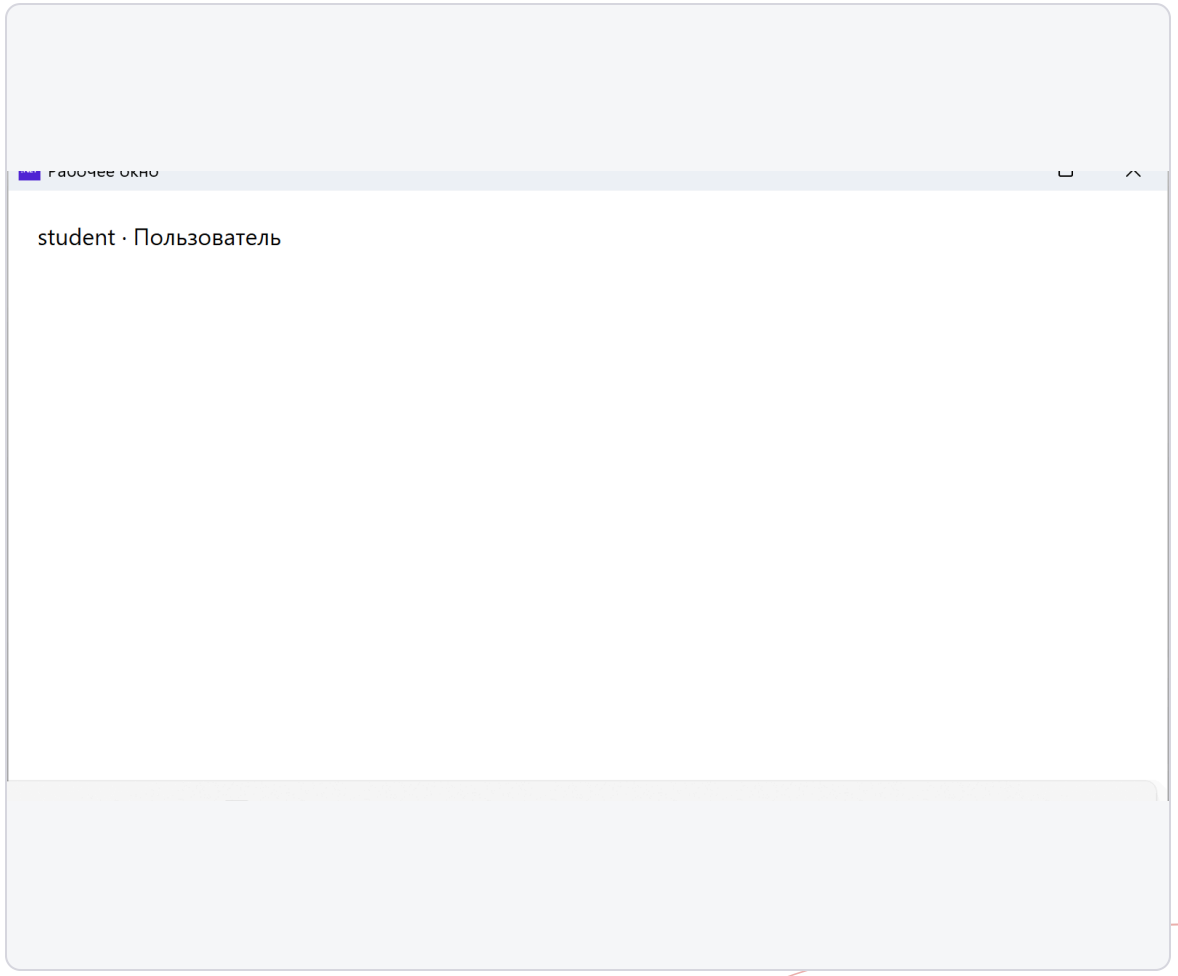
WPF. Окно «Вход»: поля логина и пароля, образец пазла слева и четыре фрагмента справа.

Что видит пользователь

1 Войдите как **student**. Найдите имя и роль в рабочем окне.

ЧТО ПРОВЕРИТЬ

Есть роль «Пользователь». Управление учётными записями недоступно.



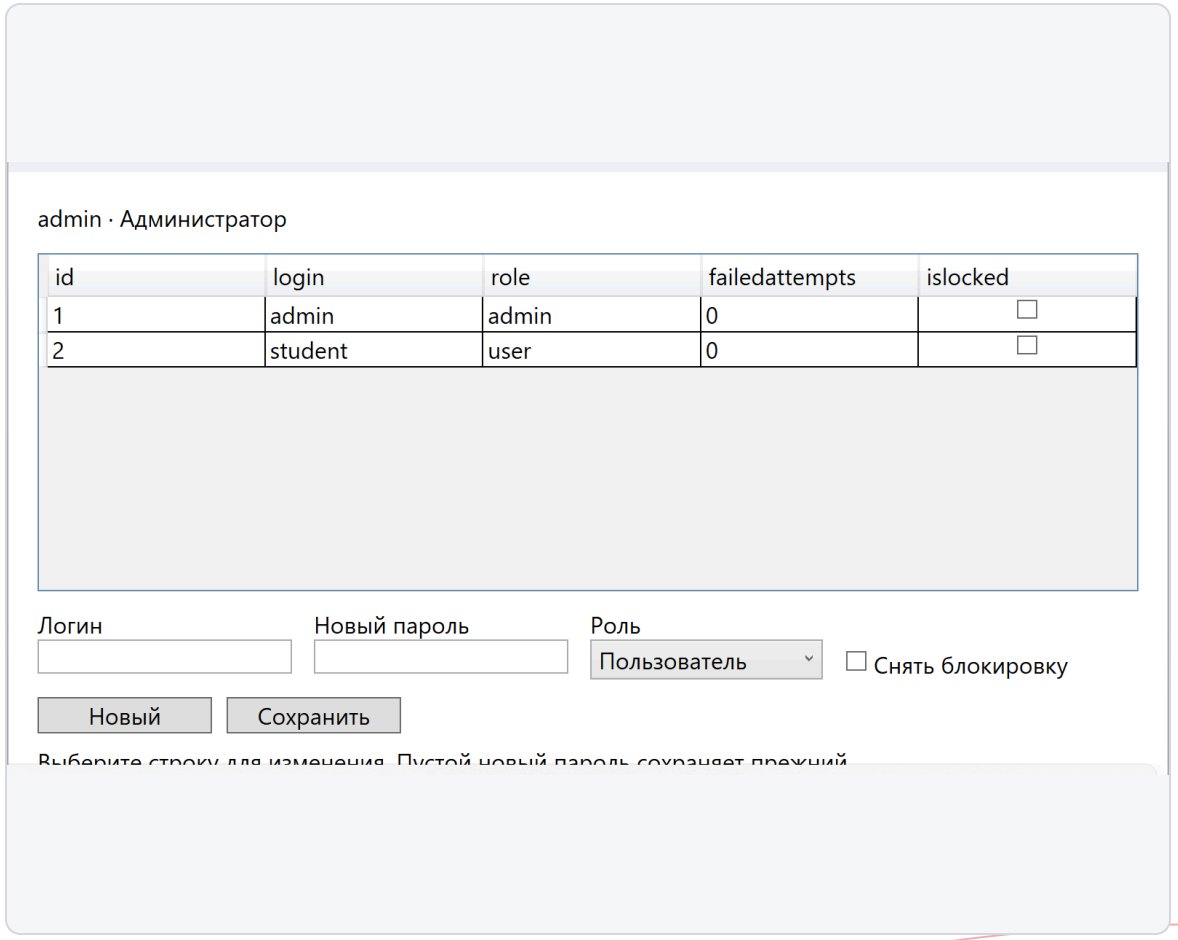
WPF. Рабочее окно **student**: видны логин и роль «Пользователь».

Что видит администратор

1. Войдите как **admin**. Покажите список, поля учётной записи, флажок блокировки и кнопки.
2. Создайте тестовую запись, проверьте блокировку и снимите её из окна администратора.

ЧТО ПРОВЕРИТЬ

Администратор может добавить, изменить и разблокировать учётную запись.

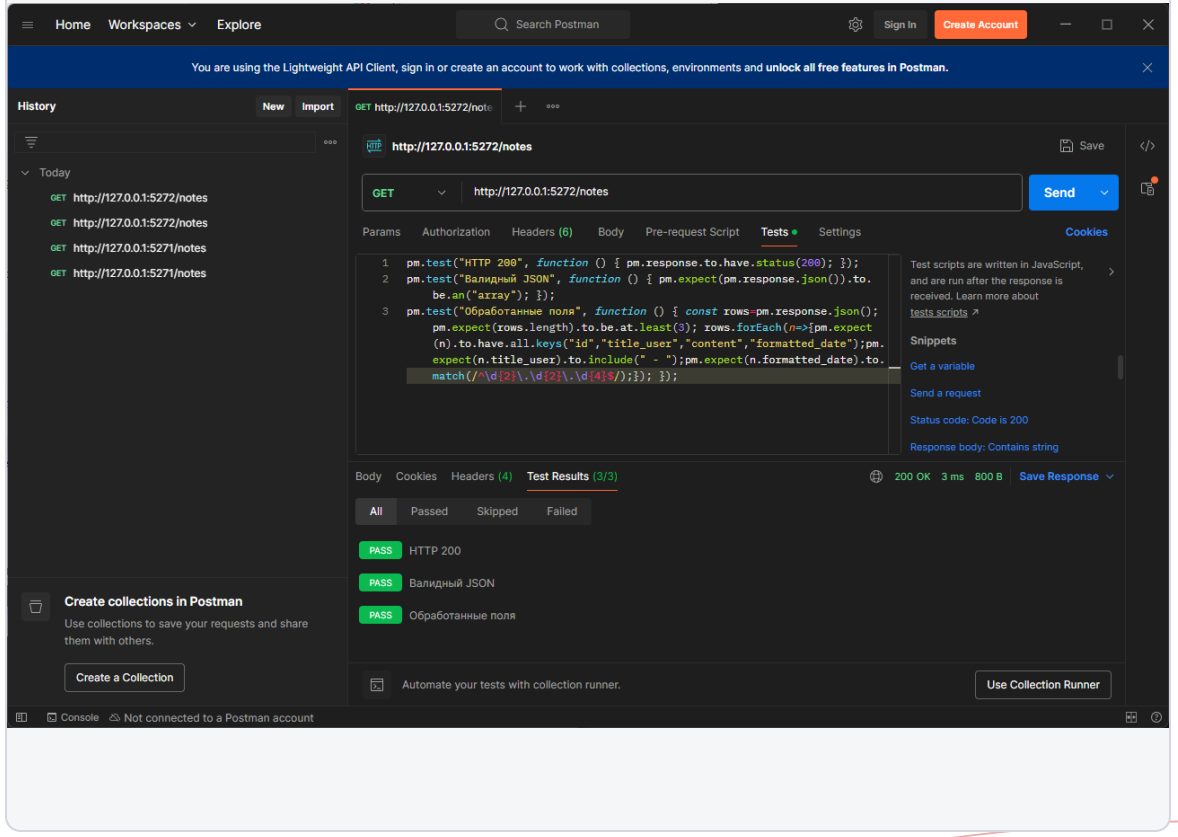


WPF. Рабочее окно **admin**: таблица учётных записей, поля редактирования и кнопки «Новый» и «Сохранить».

Запрос, тесты и документ Word

1. В Postman отправьте GET /notes. Покажите обработанный ответ и запустите тесты коллекции.
2. Сравните пример ответа в заполненном DOCX с результатом этого запроса.

Коллекция Postman Заполненный DOCX



Postman: выполненные проверки **HTTP 200**, **JSON** и преобразованных полей. Маршрут: **WPF / PostgreSQL.**

КИМ · ТАБЛИЦА 7

За что начисляют баллы

Критерий	Баллы
Осуществление администрирования отдельных компонент серверов	6
Выявление технических проблем при эксплуатации баз данных и серверов	11
Сбор исходных данных для проектной документации	6
Разработка модулей информационной системы по техническому заданию	10
Разработка подсистем безопасности	8
Разработка проектной документации по требованиям заказчика	6
Поиск, анализ и интерпретация информации	3
Разработка тестовых наборов и сценариев	9
Интеграция модулей в программное обеспечение	15
Отладка программного модуля	1

Открыть КИМ · стр. 13